

赛区评阅编号（由赛区组委会填写）：

2026 高教社杯全国大学生数学建模竞赛

承 诺 书

我们仔细阅读了《全国大学生数学建模竞赛章程》和《全国大学生数学建模竞赛参赛规则》（以下简称“竞赛章程和参赛规则”，可从 <http://www.mcm.edu.cn> 下载）。

我们完全清楚，在竞赛开始后参赛队员不能以任何方式，包括电话、电子邮件、“贴吧”、QQ 群、微信群等，与队外的任何人（包括指导教师）交流、讨论与赛题有关的问题；无论主动参与讨论还是被动接收讨论信息都是严重违反竞赛纪律的行为。

我们完全清楚，在竞赛中必须合法合规地使用文献资料和软件工具，不能有任何侵犯知识产权的行为。否则我们将失去评奖资格，并可能受到严肃处理。

我们以中国大学生名誉和诚信郑重承诺，严格遵守竞赛章程和参赛规则，以保证竞赛的公正、公平性。如有违反竞赛章程和参赛规则的行为，我们将受到严肃处理。

我们授权全国大学生数学建模竞赛组委会，可将我们的论文以任何形式进行公开展示（包括进行网上公示，在书籍、期刊和其他媒体进行正式或非正式发表等）。

我们参赛选择的题号（从 A/B/C/D/E 中选择一项填写）： C

我们的报名参赛队号（12 位数字全国统一编号）： 202619055013

参赛学校（完整的学校全称，不含院系名）： 广东以色列理工学院

参赛队员（打印并签名）：1. 唐睿

2. 李想

3. 章文心

指导教师或指导教师组负责人（打印并签名）： 无

（指导教师签名意味着对参赛队的行为和论文的真实性负责）

日期： 2026 年 09 月 13 日

（请勿改动此页内容和格式。此承诺书打印签名后作为纸质论文的封面，注意电子版论文中不得出现此页。以上内容请仔细核对，如填写错误，论文可能被取消评奖资格。）

赛区评阅编号：
(由赛区填写)

全国评阅编号：
(全国组委会填写)

2026 高教社杯全国大学生数学建模竞赛

编 号 专 用 页

赛区评阅记录（可供赛区评阅时使用）：

评 阅 人						
备 注						

送全国评阅统一编号：
(赛区组委会填写)

(请勿改动此页内容和格式。此编号专用页仅供赛区和全国评阅使用，参赛队打印后装订到纸质论文的第二页上。注意电子版论文中不得出现此页。)

基于预测与滚动优化的微网购电与储能调控策略

摘要

针对分布式光伏渗透率不断提高、购电与储能需在不确定环境下协调的问题，本文以某非孤岛式微网为对象，围绕日前计划购电、日内偏差处理与波动电价适应三项任务，建立四种信息条件下的购电与储能优化模型，并给出全年滚动求解结果。

针对问题一，在每天电价与小区负载相同、光伏预测已知的条件下，以各 10 min 时段的购电功率、储能充放电功率与弃光功率为决策变量，以功率平衡、储能容量与功率限额、储电量日循环为约束，建立全天购电费用最小的线性规划模型并求得全局最优解。取附录 1 给定的初始储电量（6000 kWh）时，全天购电量 59482.70 kWh，购电费用 **35126.95 元**，无弃光；较不配置储能（48052.05 元）日节省 12925.10 元（26.9%）。首末储电量自由确定的对照情形（8550 kWh，35118.60 元）仅差 8.35 元（附录 A）。

针对问题二，考虑负载与光伏逐日变化、供电不足按 5 倍电价紧急购电的情形，构建“日前预测—优化计划—日内纠偏”的全年滚动调控模型。数据分析发现负载具有星期型双状态结构（周日至周四约 124 MWh、周五周六约 78 MWh），据此采用上周同星期实际值预测，并依据报童模型确定购电余量（负载 $\times 1.05$ 、光伏 $\times 0.98$ ）；每天 0:00 求解日循环线性规划得到计划购电量，日内由储能优先吸收预测偏差，不足时触发紧急购电。2025 年 2 月 1 日至 12 月 31 日（334 天）总费用为 **1454.65 万元**，紧急购电量 14.5 万 kWh（102 天），较持续性预测基准（2244.8 万元）降低 35.2%。

针对问题三，对四个发布时刻的光伏预报作精度诊断，发现同一目标时刻的预报精度并不随发布时间改善，且存在上午低估、午后高估的系统性偏差。据此采用 MOS 方法融合官方预报与历史同期信息，并以当日已实现残差修正，建立 6:00、12:00、18:00 的双向调整线性规划模型（偏离计划按 0.5 倍或 1.5 倍电价结算），引入日末最低备用约束（ ≥ 4000 kWh）抑制储能短视放电。结果表明，全年总费用降至 **1368.75 万元**，较问题二策略再降 5.9%，紧急购电费由 87.0 万元降至 30.4 万元；后续预报必须经标定后使用，调整机制本身价值显著。

针对问题四，将固定电价替换为逐日波动的实时电价，在问题二、三框架下重新求解。两种策略全年总费用分别为 **1526.36 万元** 与 **1431.56 万元**，较固定电价上升 4.93% 与 4.59%，问题三策略仍优 6.21%。费用上升并非源于电价均值变化（两种情形全年均价均为 0.7662 元/kWh），而源于电价与负载的强正相关（0.898）形成的“价—荷同峰”效应；波动电价还使储能周转量提升 5%~7%，调整策略对价格波动表现出更好的适应性。

模型通过能量守恒、跨日衔接与费用分解等全链路数值校验；余量系数、残差更新权重与末端备用水平在合理范围内变化时总费用变化小于 1%，参数选择稳健。

关键词：微网；储能系统；购电策略；滚动优化；线性规划

一、问题重述

1.1 问题背景

微网是消纳光伏等分布式能源、减少并网冲击的重要形式。由于光伏出力具有明显的间歇性与波动性，小区负载也随时间变化，非孤岛式微网可在外部电网（下称外网）电价较低、或分布式能源有剩余电量的时段为储能设备充电，并在电价较高的时段释放电能，从而在保证供电可靠的同时降低购电成本。题目给出了某微网一天的分时电价、小区负载与光伏发电预测功率，以及储能设备的运行参数：最大容量 12000 kWh、最大充放电功率 5000 kW、充放电效率 90%，且运行过程中储电量须保持在 1200~10800 kWh 之间。在此基础上，需要针对电价、负载与光伏信息在不同时间尺度上的已知程度，分别建立微网日前购电与储能充放电的优化调控模型，并给出相应的计算结果。

1.2 问题提出

问题一：若每天的电价与小区负载相同，且已知光伏发电功率一天的预测数据，在微网提供的电能不低于小区负载、储能设备 0:00 与 24:00 储电量相同的条件下，建立每天 0:00 制定当天计划购电策略的数学模型，并给出指定时段的购电量、全天购电量与购电费用，以及储能设备的充放电量与 0:00、24:00 的储电量。

问题二：若每天的电价相同，而小区负载与光伏发电功率随时间变化，当微网提供的电能低于小区负载时，需按交易时刻电价的 5 倍向外网紧急购电。要求建立每天 0:00 制定计划购电策略的数学模型，并给出指定日期的购电量、储能设备充放电量与紧急购电情况。

问题三：若每天 0:00、6:00、12:00、18:00 可获得未来 24 小时整点的光伏发电功率预报，计划购电量与调整购电量之间的偏差分别按交易时刻电价的 50%（少购）与 150%（多购）结算，购电总费用还包括紧急购电费用。要求建立当天计划与调整购电策略的数学模型，分析是否需要引入其他时刻的预报制定调整策略，并给出指定日期的计算结果。

问题四：实际上外网电价也是实时波动的。要求针对波动电价，重新建立问题二与问题三条件下的微网购电策略模型，并给出相应的计算结果。

二、问题分析

2.1 针对问题一的分析

问题一在电价、负载与光伏出力均为已知固定日内规律的条件下，确定当天各 10 min 时段的购电量与储能充放电策略。由于光伏出力高峰与负载高峰在时间上并不重合，核心思路是利用储能实现电能在不同时段之间的转移：在低价与光伏富余时段充电，在高价时段放电以减少外网购电。将一天离散为 144 个时段后，各时段须满足功率平衡，储能受储电量上下限、充放电功率限额、充放电效率及日循环约束；目标函数与全部约束均为线性关系，故该问题可归结为以全天购电费用最小为目标的线性规划问题。

2.2 针对问题二的分析

问题二取消了“日内规律每天相同”的假设：负载与光伏逐日变化，计划购电按照付不议结算，供电不足部分需按当时电价的 5 倍紧急购电——购电过多造成浪费，购电不足承担高昂紧急购电成本，关键在于提高日前预测精度并合理权衡两者。首先分析附件 2 的全年数据：小区负载呈现显著的星期型双状态特征（周日至周四高、周五周六低），光伏具有较强的季节性与相邻日持续性；据此采用“上周同星期实际值”作为预测基础 [1]，避免持续性预测在星期状态切换日失效。其次，紧急购电单价是正常电价的 5 倍，预测偏低的代价远高于偏高，故依据报童模型对预测施加安全余量（抬高负载预测、下压光伏预测）[2]，并用全年仿真确定余量系数。日内利用储能实时纠偏：缺额时依次减少计划充电、增加储能放电，最后才紧急购电；富余时依次减少计划放电、增加储能充电，储能达到极限后才弃光；日末实际储电量作为次日初始状态，形成跨日连续运行的滚动调控策略。

2.3 针对问题三的分析

问题三在问题二的基础上增加 0:00、6:00、12:00、18:00 的光伏预报，可据此滚动调整购电策略；但偏离计划需付出代价（少购按 0.5 倍、多购按 1.5 倍结算），关键在于权衡“预报更新的收益”与“调整成本、紧急购电风险”。先对附件 3 预报作精度诊断：同一目标时刻的预报精度并不随发布时间改善，且存在上午低估、午后高估的系统性偏差，故不宜直接用最新预报替换原计划，而应结合历史同期信息对其标定，并利用当日已实现残差修正后续时段估计。在此基础上，以 0:00 计划为基准，在 6:00、12:00、18:00 用更新后的估计重新优化剩余时段，目标函数同时计入正常购电费用、调整违约/溢价费用与 5 倍紧急购电费用，由模型自行判断“维持原计划”与“调整购电”何者更经济。此外，若日末储电量完全自由，模型会在傍晚短视放空储能，故在调整模型中设置日末最低备用约束，最终形成“预报标定—日内修正—滚动调整—末端备用”的调控策略。

2.4 针对问题四的分析

问题四进一步放开电价，使不同日期、不同时段电价均可变化，购电决策除受负载、光伏与储能状态影响外，还取决于电价自身的时序变化，因此需在问题二、三的框架内将各项费用中的固定电价 c_t 替换为当日实际电价 $c_{d,t}$ 重新求解。先分析附件 4 的电价特征：全年均值与附件 1 相同（0.7662 元/kWh），但取值范围明显扩大、日内价差更大，说明费用变化不能仅由均价解释，还需考察电价与负载、光伏在时间上的匹配关系。其次，电价波动改变了储能的经济价值与预测偏差的代价：日内价差越大，低充高放的时移收益越高；而紧急购电按 5 倍结算时，偏差的代价也随电价上行而放大，两种策略对价格波动的适应性存在差异，需要比较其全年运行费用并归因。

2.5 本文总体建模思路

四个问题共用同一套“微网功率平衡 + 储能约束 + 购电费用最小”的优化内核，差异仅在于电价、负载与光伏信息的已知程度：问题一求解日循环线性规划；问题二引入日前预测、安全余量与日内实时纠偏；问题三利用整点光伏预报滚动调整购电计划并设置日末备用约束；问题四替换价格项并比较两类策略的经济性。总体技术路线如图 1 所示。

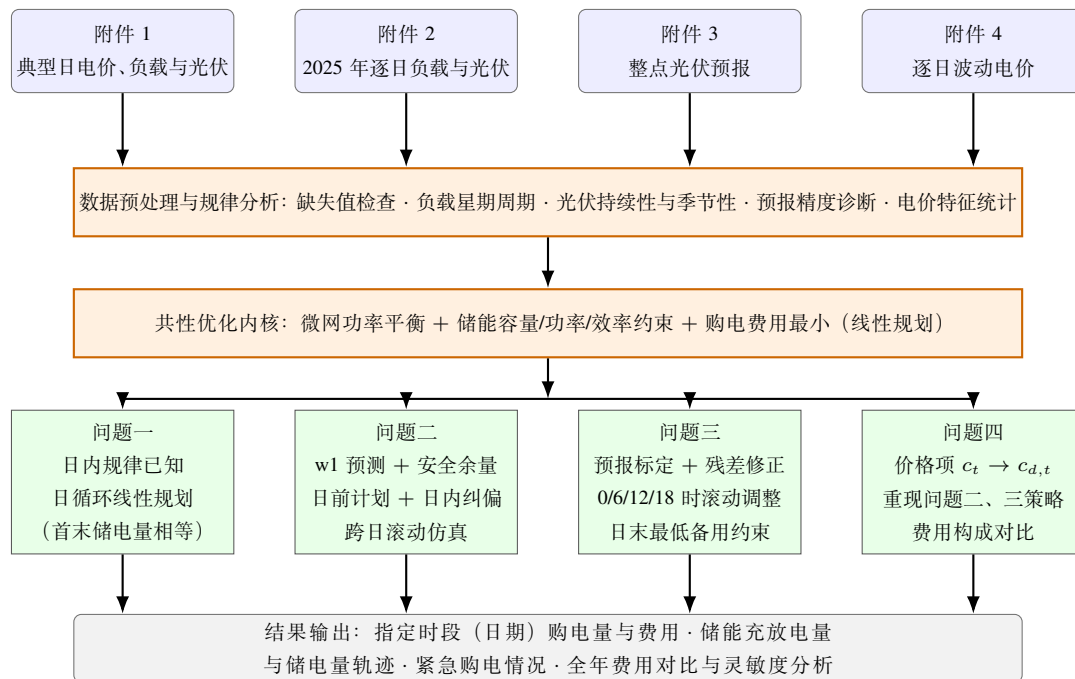


图 1 本文总体建模与求解技术路线

三、模型假设

- (1) 问题一为确定性问题：假设题目给出的光伏发电预测值即为当天的实际出力，**不考虑预测误差**；该假设**仅适用于问题一**。
- (2) 每个 10 min 离散时段内，小区负载、光伏发电功率与外网电价**保持不变**——三者在该尺度上变化很小，故用时段平均值代表其功率水平。
- (3) 储能设备的充电效率与放电效率均为 **90%**，即充电时只有 90% 的电存入储能，放电时需额外释放 1/0.9 倍电量；除充放电损耗外，**不考虑自放电及其他能量损失**。
- (4) 研究时段内储能设备的容量（12000 kWh）与最大充放电功率（5000 kW）**保持稳定**，不随使用过程衰减。
- (5) **问题二至问题四中**，每日 0:00 制定计划时**仅使用该时刻之前可获得的历史数据**及题目给出的预报，不使用当日实际运行数据，以保证策略的**因果性**。
- (6) 微网**仅能从外部电网购电**，不考虑向外部电网反向售电；当光伏发电富余且储能无法继续接纳时，**允许弃光**。
- (7) **问题二**按“**照付不议**”方式结算计划购电量，计划一经确定**不再调整**，供电不足时按当时电价的 5 倍紧急购电；**问题三、问题四**的调整机制按题目规定的 **0.5 倍 / 1.5 倍**电价另行结算。
- (8) 日内实际运行中，储能设备可根据实际负载与光伏相对预测的偏差**实时调节充放电功率**，并始终满足**容量、功率与效率约束**。
- (9) (**问题三、问题四**) 光伏预报与调度时段**口径一致**：发布时刻后第 k 小时的预报表征该整点小时内 6 个 10 min 区间的平均功率，实际值按同一口径统计。
- (10) (**问题四**) 每日 0:00 制定计划时，当日逐时电价**已由日前市场公布**，即电价**可知但逐日、逐时段波动**；电价未知情形作为模型推广方向讨论。
- (11) 忽略预测计算、优化求解与控制指令传输造成的**时间延迟**，认为计划与调度指令可即时执行。

四、符号说明

表 1 符号说明

符号	含义	单位
参数		

表 1 符号说明 (续)

符号	含义	单位
t	时段索引, $t = 1, 2, \dots, 144$ (每段 10 min)	—
$c_t, c_{d,t}$	第 t 时段 (第 d 天第 t 时段) 的外网电价	元 $\cdot \text{kWh}^{-1}$
L_t	小区负载功率	kW
P_t^{pv}	光伏发电功率 (实际值、标定后估计值与官方预报值)	kW
$E_{\min}, E_{\max}$	储电量运行下限、上限	kWh
$P_{\max}$	储能最大充放电功率	kW
η_c, η_d	储能的充电、放电效率	—
γ, δ	负载、光伏预测的余量系数	—
λ_L, λ_{pv}	当日已实现残差对负载、光伏估计的更新权重	—
$S_{\min}^{end}$	调整计划要求的日末最低备用储电量	kWh
决策变量		
p_t	购电功率	kW
p_t^{plan}, p_t^{adj}	日前计划、调整后的购电功率	kW
u_t^+, u_t^-	相对计划增加、减少的购电量	kW
u_t^c, u_t^d	储能的充电、放电功率	kW
q_t, e_t	弃光功率、紧急购电功率	kW
S_t	第 t 时段末的储电量	kWh
D_t	实际净负载功率 ($D_t = L_t - P_t^{pv}$)	kW
指标		
Z	全天 (全年) 购电总费用	元
$C^{plan}, C^{adj}, C^{em}$	计划购电费、调整费用、紧急购电费	元
MAE	光伏预报的平均绝对误差	kW
r	相关系数	—

五、问题一模型的建立与求解

5.1 模型建立

将一天按 10 min 离散为 144 个时段，记时间步长 $\Delta t = 1/6 \text{ h}$ 。以各时段的外网购电功率、储能充电功率、储能放电功率及弃光功率为决策变量，以各时段末的储电量描述储能状态变化。

(1) 功率平衡约束。外网购电、光伏发电及储能放电构成微网的电力供给，小区负载、储能充电及弃光构成电力消耗。对任一时段 t ，有

$$p(t) + P_{pv}(t) + u_d(t) = L(t) + u_c(t) + q(t), \quad t = 1, 2, \dots, 144 \quad (1)$$

其中 $q(t)$ 为弃光功率，且 $p(t) \geq 0$ 、 $q(t) \geq 0$ ； $p(t) \geq 0$ 表明微网仅从外网购电，不反向售电。

(2) 储能运行约束。考虑充放电损耗，储能状态转移方程为

$$S(t) = S(t-1) + \eta_c u_c(t) \Delta t - \frac{u_d(t) \Delta t}{\eta_d}, \quad t = 1, 2, \dots, 144 \quad (2)$$

其中 $\eta_c = \eta_d = 0.9$ ：充电时仅 90% 电能存入储能，放电时需多释放 $1/0.9$ 倍电量。由题目给定的储电量运行范围与充放电功率限额，

$$1200 \leq S(t) \leq 10800, \quad t = 0, 1, \dots, 144; \quad 0 \leq u_c(t) \leq 5000, \quad 0 \leq u_d(t) \leq 5000. \quad (3)$$

问题一要求 0:00 与 24:00 的储电量相同，设置日循环约束

$$S(144) = S(0). \quad (4)$$

(3) 目标函数。第 t 时段购电量为 $p(t)\Delta t$ ，费用为 $c(t)p(t)\Delta t$ ，故

$$\min Z = \sum_{t=1}^{144} c(t) p(t) \Delta t. \quad (5)$$

上述模型共 721 个变量（ 4×144 个功率变量与 145 个储能状态变量）、289 个等式约束，目标与约束均为线性，属小型线性规划，可用单纯形法求得全局最优解。

5.2 模型求解

将附件 1 的分时电价、小区负载及光伏预测功率代入模型，采用 Python 调用 HiGHS 线性规划求解器（`scipy.optimize.linprog` 接口）求解，并用 MATLAB `linprog` 独立复算，两者结果逐位一致。购电功率按 $Q_t = p(t)\Delta t$ 换算为各时段购电量输出。

关于日循环边界的取值：附录 1 给定 2025 年 1 月 1 日 0:00 的储电量为 6000 kWh，本文在主文中据此取 $S(0) = S(144) = 6000 \text{ kWh}$ ；作为对照，将首末储电量改由模型自由确定的情形（得 $S(0) = S(144) = 8550 \text{ kWh}$ ）完整求解并列于附录 A——两情形全天购电量与指定时段购电量完全相同，费用仅相差 8.35 元（0.024%），结论对该边界的两种解读均稳健。

5.3 求解结果与分析

最优策略下，全天外网购电量为 **59482.70 kWh**，全天购电费用为 **35126.95 元**，0:00 与 24:00 储电量均为 6000.00 kWh，全天弃光量为 0。题目要求的指定时段购电量如表 2 所示，储能分时段充放电量与首末储电量如表 3 所示，全天调度过程如图 2 所示。

表 2 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.0000	12:00–12:10	486.4029	14:00–14:10	0.0000
16:00–16:10	394.9315	18:00–18:10	636.9826	20:00–20:10	0.0000
全天购电量/kWh: 59482.70			全天购电费/元: 35126.95		

表 3 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	4500.0000	0.0000	4:00–8:00	833.3333	5947.4197
8:00–12:00	3954.6310	2121.4184	12:00–16:00	6119.3685	91.1014
16:00–20:00	0.0000	5068.6869	20:00–24:00	5333.3333	3571.3131
0:00 储电量/kWh: 6000.0000			24:00 储电量/kWh: 6000.0000		

由图 2 可见，储能充放电行为与分时电价、光伏出力高度协同：(i) 夜间低谷（22:00 至次日 6:00，电价约 0.37~0.44 元/kWh）低价购电满足负载并对储能充电，0:00–4:00 与 20:00–24:00 两段合计充电 9833.33 kWh；(ii) 早高峰（6:00–10:00，电价 0.69~1.17 元）储能大功率放电（4:00–8:00 段放电 5947.42 kWh），压低高价购电；(iii) 中午光伏富余且电价处于日内低谷，富余光伏全部存入储能，并以低价购电充满充电通道（12:00–12:10 购电 486.40 kWh），供晚高峰使用——存储套利的经济性条件 $0.9 \times 0.9 \times 1.35 > 0.44$ 成立；(iv) 傍晚峰段（18:00–21:00，电价 1.23~1.40 元）储能能量稀缺，优先留给最高价的 18:50–20:50 放电（该时段购电为 0），其余峰段由购电覆盖净负载。全天共有约 20.7 MWh 电量经储能周转，每 1 kWh 夜间电量经储能（往返效率 0.81）在峰时释放净收益约 $0.81 \times 1.35 - 0.43 \approx 0.66$ 元；与不配置储能的情形（48052.05 元，见第九章）相比，储能使全天购电费下降 **12925.10 元（26.9%）**。

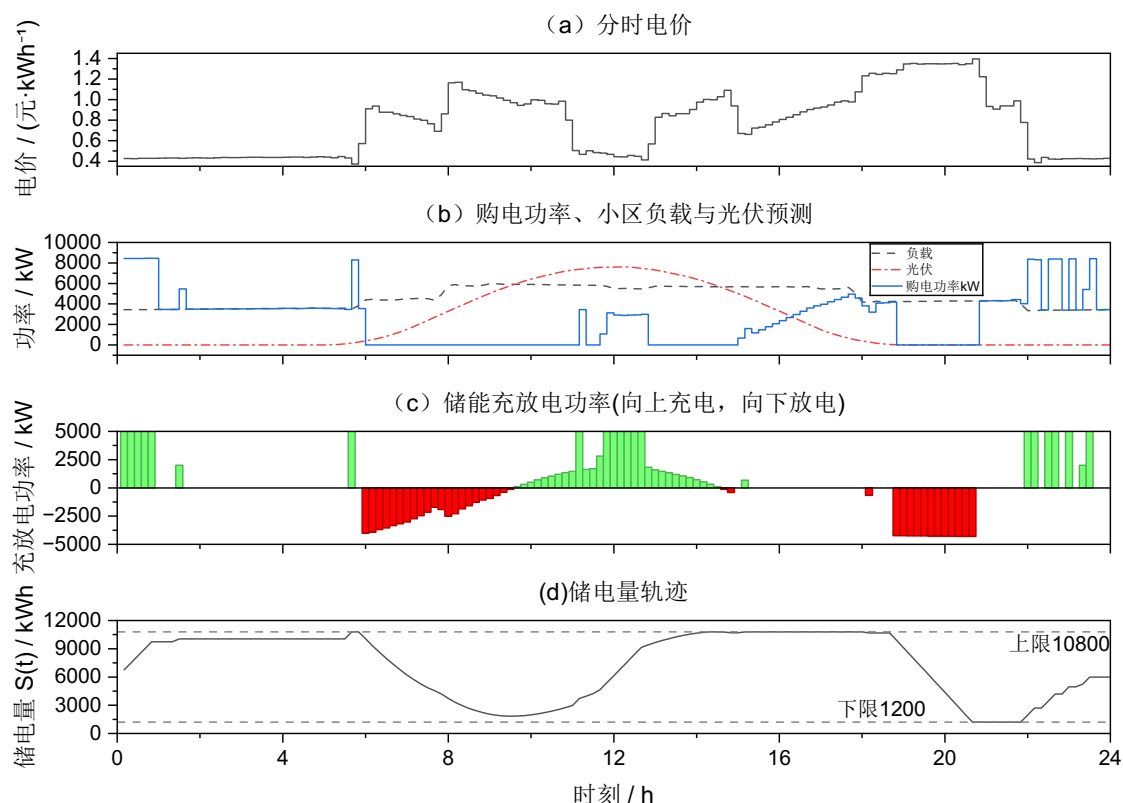


图2 问题一最优日前调度结果 ($S(0) = S(24:00) = 6000 \text{ kWh}$): (a) 分时电价; (b) 购电功率与小区负载; (c) 储能充放电功率 (向上为充电、向下为放电); (d) 储电量轨迹 (虚线为运行上下限 1200/10800 kWh)

六、问题二模型的建立与求解

6.1 建模思路

问题二中负载与光伏出力不再具有固定日内规律,日前计划与实际运行之间存在预测偏差,供电不足需以正常电价的 5 倍紧急购电。因此在问题一优化内核的基础上,引入负载与光伏预测、预测余量及日内实时纠偏机制,构建“日前预测—计划优化—实时纠偏—跨日滚动”的调控模型:先由全年数据分析选择预测方法,再经余量修正后求解日前计划线性规划,实际运行时由储能优先吸收预测偏差、不足时紧急购电,日末实际储电量传递至次日。

6.2 数据规律分析与日前预测模型

对附件 2 中 2025 年实际负载和光伏数据进行统计分析,结果如图 3 所示。

由图 3 (左) 可知,小区日负载具有显著的星期周期特征:周日至周四日负载均值约 124.0 MWh,周五、周六仅约 78.3 MWh (比值 0.63),呈稳定的星期型双状态结构;若直接采用前一日数据预测当日负载,在周四至周五、周六至周日的状态切换时将产生

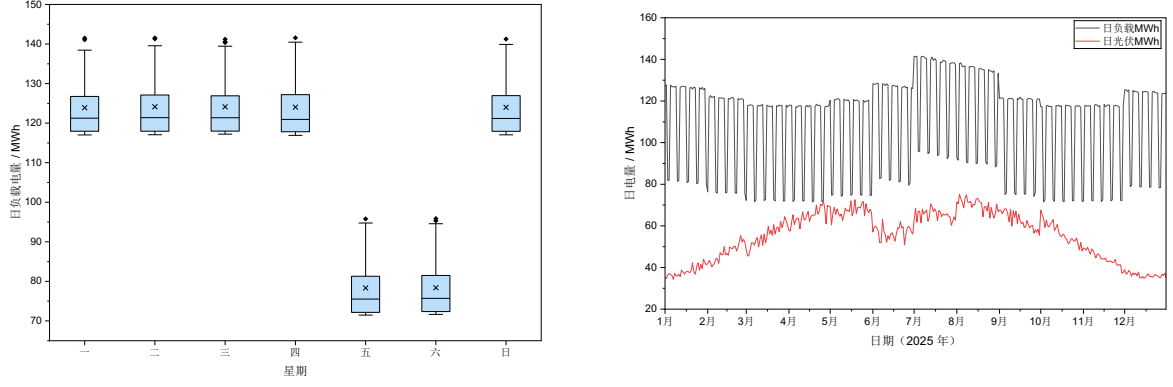


图3 2025 年负载与光伏出力时序特征：（左）按星期分组的日负载电量箱线图；（右）全年逐日负载与光伏电量

很大偏差。由图 3（右）可知，负载在星期周期之上还存在温和的季节波动；光伏日发电量呈明显季节性，且相邻日自相关系数约 0.964[1]，近期历史光伏数据对当日出力有较强参考价值。

综合上述特征，本文采用上周同星期日预测法（w1）[1] 构造日前预测：对第 d 日第 t 个时段，

$$\hat{L}_{d,t} = L_{d-7,t}, \quad \hat{P}_{d,t}^{pv} = P_{d-7,t}^{pv} \quad (6)$$

该预测仅使用计划时刻已获得的历史数据，避免未来信息泄漏，同时保留负载的星期周期结构与光伏的近期变化特征。作为对照，本文同时构建持续性预测、近 2 个和近 4 个同星期日均值预测（w2、w4）与 7 日移动平均预测（ma7），以全年运行总费用为指标比较（结果见第九章表 17）。

6.3 预测余量设置与参数寻优

日前预测不可避免存在误差：当实际净负荷高于预测时产生 5 倍电价的紧急购电费用，而适度多购仅增加约 1 倍电价的正常购电成本——预测偏低与偏高的经济损失明显**不对称**，恰为报童模型结构 [2]：临界分位数 $(5-1)/5 = 0.8$ ，即日前计划应覆盖净负荷约 80% 分位数而非中位数。据此引入负载余量系数 γ 与光伏余量系数 δ 对预测作保守修正：

$$L_{d,t}^* = \gamma \hat{L}_{d,t}, \quad P_{d,t}^{pv,*} = \delta \hat{P}_{d,t}^{pv}, \quad \gamma \geq 1, \quad 0 < \delta \leq 1. \quad (7)$$

以全年计划购电费用与紧急购电费用之和为评价指标，对各候选预测方法与余量参数组合网格搜索与全年滚动仿真，最终选取 w1 预测与

$$\gamma = 1.05, \quad \delta = 0.98 \quad (8)$$

即负载预测上调 5%、光伏预测下调 2%。按全年均值折算，其对净负荷的等效余量约 12%，与报童 0.8 分位所需安全水平量级一致；总费用曲面在 $\gamma \in [1.03, 1.07]$ 为平坦平

台（详见第九章）。

6.4 日前计划购电优化模型

以余量修正后的预测作为每日 0:00 的已知信息，在问题一线性规划的基础上建立第 d 日的日前计划模型：

$$\min C_d^{plan} = \sum_{t=1}^{144} c_t p_{d,t} \Delta t \quad (9)$$

满足计划功率平衡与储能约束

$$p_{d,t} + P_{d,t}^{pv,*} + u_{d,t}^{dis} = L_{d,t}^* + u_{d,t}^{ch} + q_{d,t}, \quad (10)$$

$$S_{d,t} = S_{d,t-1} + \eta_c u_{d,t}^{ch} \Delta t - \frac{u_{d,t}^{dis} \Delta t}{\eta_d}, \quad (11)$$

$$1200 \leq S_{d,t} \leq 10800, \quad 0 \leq u_{d,t}^{ch}, u_{d,t}^{dis} \leq 5000. \quad (12)$$

问题二同时涉及**日前计划状态与实际运行状态**。每日 0:00 以储能实际储电量 $S_{d,0}$ 为初始状态规划全天储能轨迹，为避免模型通过消耗已有储能人为压低当日计划费用，在计划层面设置首末储电量相等约束

$$S_{d,144}^{plan} = S_{d,0} \quad (13)$$

该约束仅作用于计划轨迹；实际运行因预测偏差由储能纠偏，实际日末储电量 $S_{d,144}^{real}$ 可与计划值不同，并作为次日初始状态 $S_{d+1,0} = S_{d,144}^{real}$ 实现跨日连续传递。

实际运行中产生的紧急购电费用按当日真实数据与纠偏结果计算：

$$C_d^{em} = \sum_{t=1}^{144} 5 c_t e_{d,t} \Delta t, \quad C_{total} = \sum_d (C_d^{plan} + C_d^{em}). \quad (14)$$

需要指出， C_{total} 并非日前线性规划直接优化的目标，而是评价不同预测方法与余量参数下全年策略实际经济效果的指标。

6.5 日内实时纠偏策略

定义执行日前计划后第 t 时段的实时功率缺口：

$$D_{d,t} = L_{d,t}^{real} - [p_{d,t}^{plan} + P_{d,t}^{pv,real} + u_{d,t}^{dis,plan} - u_{d,t}^{ch,plan}] \quad (15)$$

$D_{d,t} > 0$ 表示按计划执行后仍有供电缺口， $D_{d,t} < 0$ 表示电力富余。纠偏严格按成本由低到高的优先级进行（流程见图 4）：

(1) 供电不足 ($D_{d,t} > 0$)： 减少计划充电 → 增加储能放电 → 紧急购电。先减少该时段计划充电功率将电能直接用于负荷；仍有缺口则在功率与最低储电量约束内增加放电；储能达调节极限后，剩余缺口按当时电价 5 倍紧急购电。

(2) 电力富余 ($D_{d,t} < 0$): 减少计划放电 → 增加储能充电 → 弃置剩余电量。

(3) 状态更新。每时段纠偏后按实际充放电功率更新储电量 $S_{d,t}^{real} = S_{d,t-1}^{real} + \eta_c u_{d,t}^{ch,real} \Delta t - u_{d,t}^{dis,real} \Delta t / \eta_d$ ，日末传至次日。

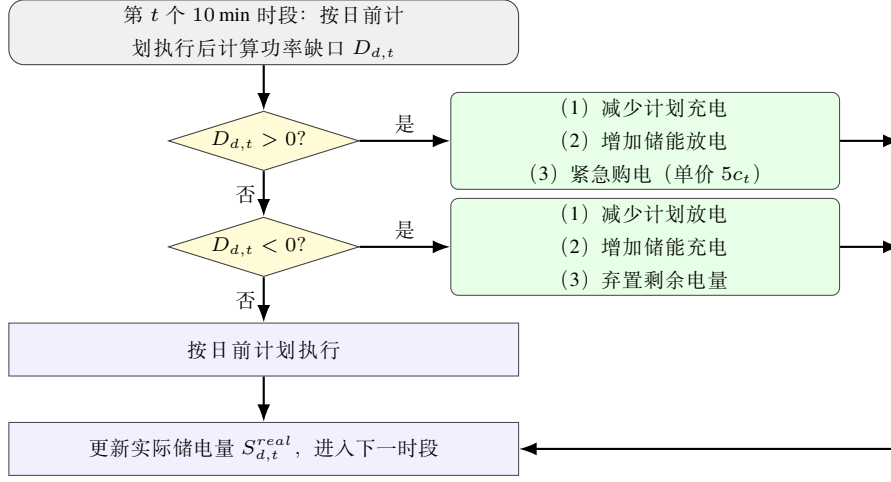


图 4 日内实时纠偏决策流程 (第 d 日第 t 时段)

6.6 全年滚动求解流程

按题目给定条件设 2025 年 1 月 1 日 0:00 储能初始储电量 $S_{1,0}^{real} = 6000 \text{ kWh}$ ，随后按“日前预测—余量修正—计划优化—实时纠偏—状态传递”逐日滚动求解全年。由于 w1 预测需要 7 天历史数据，1 月为预热期；按题目要求，最终统计 **2025 年 2 月 1 日至 12 月 31 日共 334 天**的计划购电量、计划购电费用及紧急购电费用。

6.7 求解结果与分析

全年滚动求解结果为：报告期内计划购电总量 22320727.5 kWh，计划购电费用 13676311.49 元；全年紧急购电总量 145133.8 kWh (102 天发生)，紧急购电费用 870192.48 元。实际购电总费用

$$C_{total} = 13676311.49 + 870192.48 = \mathbf{14546503.97 \text{ 元}} \approx 1454.65 \text{ 万元}. \quad (16)$$

紧急购电量仅占外网购电量的 0.6%，说明日前预测与安全余量覆盖了大部分负荷偏差，储能进一步承担了日内误差的实时调节。全年每日紧急购电量分布如图 5 所示。

(1) 指定日期购电结果。按题目要求统计 2025 年 3 月 20 日、6 月 21 日、9 月 23 日和 12 月 21 日的结果，如表 4-6 所示 (表 5 的时段块按钟点日划分：“0:00-4:00”块为当日 0:00-0:10 至 0:10-4:00 共 24 个区间，与同排 0:00 储电量严格对应)。

(2) 典型日实际运行结果分析。选取负载状态切换日 2025 年 12 月 21 日 (周六 → 周日) 分析日前计划与日内纠偏的实际运行过程，如图 6 所示。

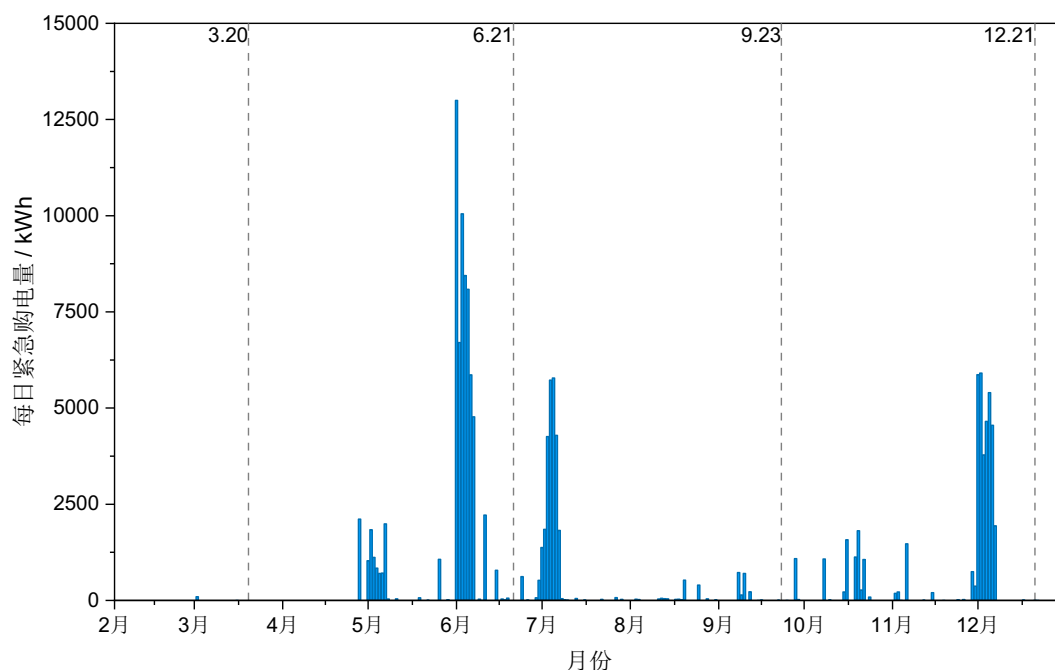


图 5 2025 年 2-12 月每日紧急购电量分布（虚线为题目指定的四个日期）：全年共 102 天发生紧急购电，合计 14.5 万 kWh，集中于 6 月与 12 月的负载状态切换期

表 4 四个指定日期微网在指定时间段的计划购电量及全天的购电量和购电费

日期	10:00-10:10	12:00-12:10	14:00-14:10	16:00-16:10	18:00-18:10	20:00-20:10	全天/kWh	购电费/元
2025-03-20	0.0000	822.2488	0.0000	493.8229	759.4335	0.0000	78872.98	48644.97
2025-06-21	0.0000	0.0000	0.0000	139.1066	451.7126	0.0000	38441.60	21995.11
2025-09-23	0.0000	627.7958	0.0000	401.8831	833.4051	9.9357	70972.59	44123.14
2025-12-21	0.0000	1023.8313	0.0000	857.8885	753.0828	0.0000	100592.96	65063.32

由图 6(a) 可知，计划购电功率并不简单跟随电价或负载：在 8:00-11:00 与 17:00-21:00 高电价时段被压至较低水平，在 0:00-8:00 低电价时段维持较高购电功率，为储能补充电量，实现电能的“低储高放”；w1 预测较好地跟踪了实际负载与光伏的整体变化，仅局部时段存在偏差。由图 6(b) 可知，实际储电量轨迹与计划总体一致，但在预测偏差累积时段明显偏离：10:30-11:00 与 21:00 前后实际储电量一度接近运行下限 1200 kWh，表明储能调节能力已被充分调用；全天始终未越限。当日仅在 8:00-8:10 出现 5.62 kWh 短时紧急购电：该时段计划购电功率仅约 600~700 kW 并由储能满功率放电顶峰，任何微小偏差都会立即触发紧急购电——这正是“预测已很准为何仍有紧急购电”的机理答案。作为对照，持续性预测基准在当日产生紧急购电 42.9 MWh（状态切换日预测失效），

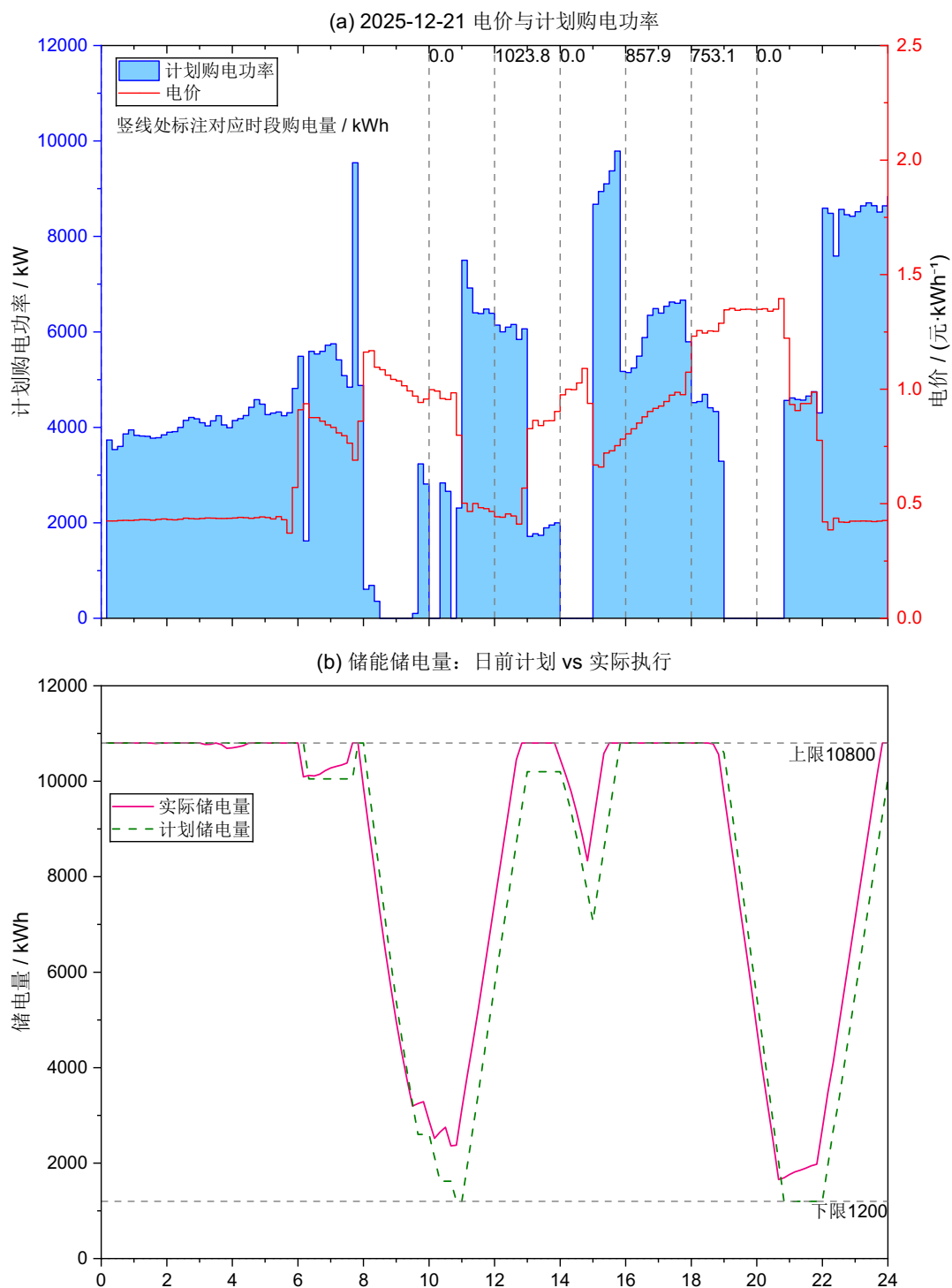


图6 2025年12月21日日前计划与实际运行对照：(a) 分时电价与计划购电功率（竖线处标注题目指定6时段的购电量，kWh）；(b) 日前计划储电量与实际储电量轨迹（虚线为储电量上下限1200/10800 kWh）

表 5 四个指定日期储能设备在指定时间段的充、放电量（实际值，单位：kWh）

时间段	2025-03-20		2025-06-21		2025-09-23		2025-12-21	
	充电量	放电量	充电量	放电量	充电量	放电量	充电量	放电量
0:00–4:00	119.4939	96.7901	209.2249	2844.3902	177.3126	11.5808	272.7706	132.1232
4:00–8:00	1317.6609	3157.2466	252.6738	5322.7460	34.2821	6151.4626	919.6894	644.8460
8:00–12:00	5604.5696	3265.5568	9620.9855	0.0000	4414.4148	2387.9403	5182.9578	7883.6687
12:00–16:00	1321.6289	254.7228	0.0000	0.0000	6638.1387	440.9339	7290.8424	2220.1094
16:00–20:00	16.6683	4501.1868	150.9449	4915.2439	342.0831	5001.9810	0.0000	4606.0024
20:00–24:00	10054.6548	3656.5848	9694.1410	3059.2756	10077.1714	3606.4323	10159.0080	3622.7941

表 6 四个指定日期的 0:00 与 24:00 储电量（单位：kWh）及紧急购电情况

日期	0:00 储电量	24:00 储电量	紧急购电时段与电量
2025-03-20	10800.00	10800.00	无
2025-06-21	10800.00	10800.00	无
2025-09-23	10653.29	10612.43	无
2025-12-21	10590.08	10800.00	8:00–8:10, 5.6161 kWh

w1 预测将其基本消除——这正是数据规律分析的价值所在。

(3) 结果小结。所建立的调控策略形成“日前优化—储能纠偏—紧急购电兜底”的分层调控机制：全年总购电费用 1454.65 万元，较持续性预测基准（2244.8 万元）降低 35.2%，较完美预测理论下界（1224.5 万元）的残余预测误差代价为 230.2 万元（15.8%），为后续策略比较与模型检验提供了基础。

七、问题三模型的建立与求解

7.1 求解思路

问题三在问题二的基础上增加每天 0:00、6:00、12:00、18:00 发布的未来 24 小时整点光伏预报（附件 3），并允许在 6:00、12:00、18:00 依据新预报调整当天尚未执行的购电计划：少购部分按交易时刻电价的 50% 结算，多购部分按 150% 结算，供电缺口仍按 5 倍电价紧急购电。因此关键不在于“用最新预报直接替换原计划”，而在于回答**后续时刻的预报是否值得用于制定调整购电策略**。本文先对附件 3 的预报质量作诊断，再建立“预报标定—当日残差修正—双向滚动调整—末端备用”的两阶段调度模型，通过策

略变体对比拆解“预报质量”与“调整机制”各自的贡献，定量回答上述问题。求解流程如图 7 所示。

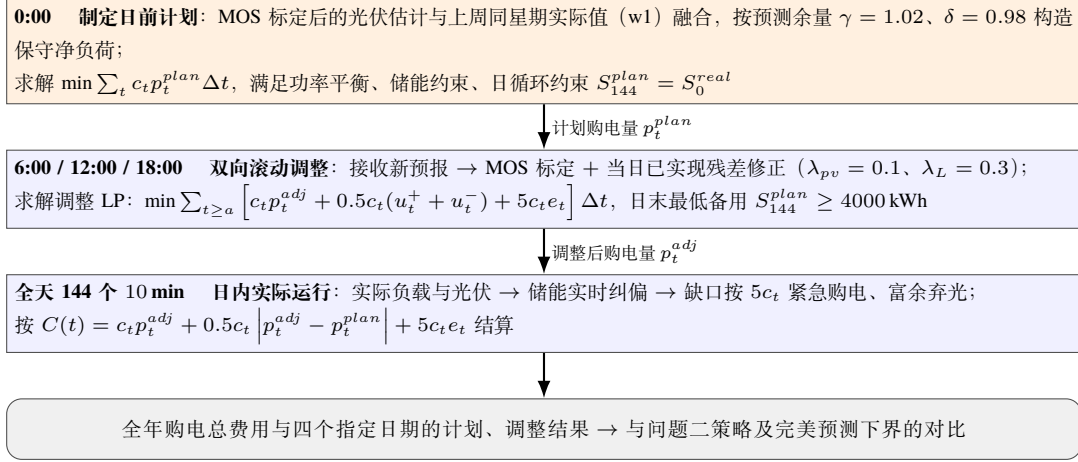


图 7 问题三“计划—调整—结算”两阶段滚动调度流程

7.2 模型建立

7.2.1 附件 3 预报质量诊断

将附件 3 全年 365 天、每天 4 次发布的整点预报与附件 2 实际光伏出力按同一口径逐时对照 (“预报第 k 小时” 对应发布时刻后第 k 个整点, 实际值取该小时内 6 个 10 min 区间均值), 统计各目标时刻的平均绝对误差 (MAE) [1] 与系统性偏差, 如图 8 所示。

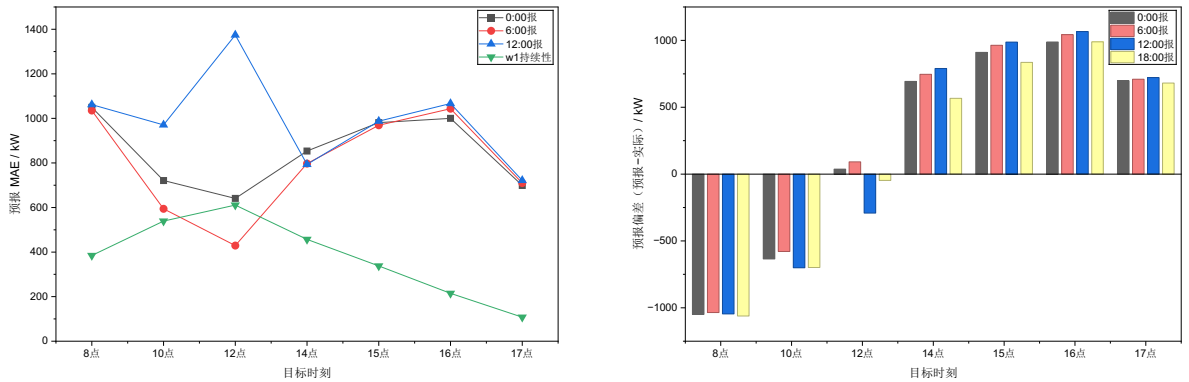


图 8 附件 3 预报质量诊断: (左) 不同发布时刻的预报 MAE 随目标时刻的变化; (右) 预报偏差 (预报值 - 实际值) 随目标时刻的变化

由图 8 (左) 可得两点结论: 其一, 同一目标时刻的预报精度几乎不随发布时间改善——目标时刻 16:00 时, 0:00、6:00、12:00 发布的预报 MAE 分别为 1000.2、1043.6、1066.5 kW, 反而略有上升; 14:00 之后上周同星期实际值 (w1) 的 MAE 显著更低 (16:00

时仅 214.7kW)。其二，“更新”的预报并不必然优于早前预报——对目标时刻 12:00，12:00 发布序列中对应的恰为 24 小时前作出的预报，其 MAE 高达 1374.1kW，明显劣于当日 0:00 预报 (640.9kW) 与 6:00 预报 (429.4kW)。

由图 8 (右) 可知，预报存在明显的**系统性偏差**：上午时段预报偏低 (8:00 约 -1050kW，10:00 约 -580 ~ -700kW)，午后时段预报偏高 (14:00 约 +567 ~ +790kW，15:00 约 +836 ~ +988kW，16:00 约 +988 ~ +1067kW)，且不同发布时刻的偏差方向高度一致，属预报本身的系统性偏差；预报峰值时刻 (13:00) 与实际峰值时刻 (12:00) 亦存在错位。此外，12:00 报的修订量与当日上午“实际 - 预报”偏差的相关系数仅 0.02~0.04，说明**后续预报几乎不含当日天气的新信息**。

尽管信息增量有限，预报值与实际出力仍具有较高的相关性 (0:00 报逐时相关系数 0.78 以上)，其系统性偏差可用历史数据标定修正；且当日上午实际出力与午后实际出力的相关系数达 0.914，w1 残差的日内自相关系数光伏为 0.464、负载为 0.592，说明**后续预报的正确用法是“经标定的输入 + 当日已实现残差修正”，而非当作真值直接使用**。

7.2.2 光伏出力估计模型

采用模型输出统计 (MOS) 方法 [3] 对光伏出力标定，并以当日已实现残差修正后续时段。

(1) **0:00 计划用估计值**。逐小时建立线性回归 [1]，融合当日 0:00 官方预报与 w1：

$$\widehat{PV}(d, h) = a_h + b_h \widehat{PV}_{0:00}^{off}(d, h) + c_h \widehat{PV}^{w1}(d, h) \quad (17)$$

回归系数由最近 30 天的 (官方预报，w1，实际值) 三元组按最小二乘拟合并逐日滚动更新 (保证因果性)，历史不足 5 天时退化为 0.5/0.5 等权融合。以 14:00 为例， $\widehat{PV} \approx 232 + 0.34 \widehat{PV}_{0:00}^{off} + 0.58 \widehat{PV}^{w1}$ ，MAE 为 373.0kW，优于单独使用 w1 (456.8kW) 与原始预报 (793.4kW)。逐时估计再按上周同日的日内形状系数分解到 144 个时段。

(2) **调整时刻的更新估计值**。在调整时刻 $\tau \in \{6, 12, 18\}$ ，将 τ 时刻发布的官方预报代入式 (17) 得基准估计，并叠加当日已实现残差：

$$\widehat{PV}_\tau(h) = \widehat{PV}(h) + \lambda_{pv} \overline{(PV^{real} - \widehat{PV})}_{h' < \tau}, \quad \widehat{L}_\tau(h) = \widehat{L}(h) + \lambda_L \overline{(L^{real} - \widehat{L})}_{h' < \tau} \quad (18)$$

其中 $\overline{(\cdot)}_{h' < \tau}$ 对 τ 之前已实现的整点时段取平均。由于残差日内自相关为正但不强， λ 宜取中小值；以全年总费用为目标网格寻优，最终取 $\lambda_{pv} = 0.1$ 、 $\lambda_L = 0.3$ (见表 7)。

7.2.3 计划与调整两阶段优化模型

(1) **费用结算口径**。按题意，第 t 个时段的购电费用为

$$C(t) = c_t p_t^{adj} + 0.5 c_t |p_t^{adj} - p_t^{plan}| + 5 c_t e_t \quad (19)$$

三项依次为按调整购电量实付的电费、相对计划的违约与溢价费用、紧急购电费用；它精确等价于“计划内按 c_t 、超出计划按 $1.5c_t$ 、少购部分按 $0.5c_t$ ”结算。引入非负变量 u_t^+, u_t^- 表示上调与下调量以便线性化：

$$p_t^{adj} - p_t^{plan} = u_t^+ - u_t^-, \quad u_t^+ \geq 0, \quad u_t^- \geq 0. \quad (20)$$

(2) 0:00 计划模型。结构与问题二一致：余量取 $\gamma = 1.02$ 、 $\delta = 0.98$ （调整机制提供了追索权，余量较问题二的 1.05 下降，寻优见第 7.3.1 节），以当日计划购电费用最小为目标，满足功率平衡、储能约束与日循环约束 $S_{144} = S_0$ 。

(3) 调整模型。在调整时刻 τ 对尚未执行的时段 $t \geq a$ 重新优化：

$$\min \sum_{t=a}^{144} \left[c_t p_t^{adj} + 0.5 c_t u_t^+ + 0.5 c_t u_t^- + 5 c_t e_t \right] \Delta t \quad (21)$$

约束包括更新估计下的功率平衡、以该时刻实际储电量为初值的储能状态转移与容量、功率约束、式 (20) 的调整量分解，以及紧急购电松弛变量 $e_t \geq 0$ 。 e_t 作为“安全阀”保证任意更新情形下模型可行，并使模型在“ $1.5c_t$ 上调”与“ $5c_t$ 紧急购电”之间自动比价。此外，为避免模型在傍晚短视放空储能，设置**日末最低备用约束**

$$S_{144}^{plan} \geq S_{\min}^{end} = 4000 \text{ kWh} \quad (22)$$

该约束仅作用于调整计划的规划口径；日内实际偏差下储能仍可放电至运行下限 (1200 kWh)。

(4) 日内实际运行与结算。日内仍采用与问题二相同的储能实时纠偏机制（缺额：减充电 → 加放电 → 5 倍紧急购电；富余：减放电 → 加充电 → 弃光），并按式 (19) 对调整后的实际购电量结算。

7.3 模型求解与分析

7.3.1 参数寻优

调整机制使预测偏差的代价由 5 倍紧急购电降为 0.5 倍调整费用，余量参数重新寻优后负载余量由 1.05 降至 $\gamma = 1.02$ ，光伏余量保持 $\delta = 0.98$ 。当日残差更新权重的寻优结果如表 7 所示：不用当日信息 ($\lambda = 0$) 时总费用 1376.3 万元，最优点 (0.1, 0.3) 处 1368.8 万元，说明当日已实现偏差确有信息增量； λ 偏大 (0.3/0.4 以上) 时调整模型追逐残差噪声、支付多余调整费用，总费用回升；最优点附近曲面平坦，结果对 λ 取值不敏感。

日末最低备用约束的取值经变体实验确定：完全自由末端虽比强制当日闭环略省 (1382.6 万元对 1384.3 万元)，但储能每天傍晚放空至下限附近、几乎不留备用，紧急购电天数明显增加；取 $S_{\min}^{end} = 4000 \text{ kWh}$ (约可用容量的 42%) 后费用与稳健性同时改善，且在 2000~7000 kWh 范围内总费用变化平缓（详见第九章）。

表 7 当日残差更新权重的寻优结果（全年总费用，单位：万元）

$(\lambda_{pv}, \lambda_L)$	(0, 0)	(0.1, 0.2)	(0.1, 0.3)	(0.15, 0.2)	(0.3, 0.4)	(0.45, 0.6)	(0.6, 0.8)
全年总费用	1376.3	1369.5	1368.8	1370.6	1377.2	1393.0	1414.4

7.3.2 策略变体对比

为定量回答“是否需要引入其他时刻的预报”，设计一组策略变体从“预报质量”与“调整机制”两个维度逐步改进，全年滚动仿真结果如表 8 与图 9 所示。

表 8 不同购电策略的全年总费用对比（报告期 2025 年 2 月 1 日–12 月 31 日，共 334 天）

策略	说明	总费用/万元	费用构成/万元	相对问题二
V0	问题二策略（w1 光伏预测、无调整）	1454.7	实付 1367.6 + 紧急 87.0	—
V1	原始 0:00 预报、无调整	1809.5	实付 1335.7 + 紧急 473.8	+24.4%
V2	原始预报 + 双向调整	1586.0	实付 1458.3 + 调整 110.1 + 紧急 17.7	+9.0%
V3	预报标定、无调整	1410.7	实付 1315.0 + 紧急 95.6	−3.0%
V4D	标定 + 仅下调调整	1407.0	实付 1301.4 + 违约 17.5 + 紧急 88.1	−3.3%
V4	标定 + 双向调整（强制当日闭环）	1384.3	实付 1312.8 + 调整 33.7 + 紧急 37.8	−4.8%
V4F	标定 + 双向调整 + 自由末端	1382.6	实付 1303.6 + 调整 36.3 + 紧急 42.6	−5.0%
V4FR	标定 + 双向调整 + 末端备用 + λ 寻优	1368.8	实付 1306.2 + 调整 32.1 + 紧急 30.4	−5.9%
—	完美预测（理论下界）	1224.5	—	−15.8%

由表 8 可知：直接用原始预报替代 w1 预测使总费用由 1454.7 万元升至 1809.5 万元（+24.4%），说明**原始预报的偏差远大于 w1 预测**；但仅依靠调整机制（V2）即可挽回其中 63% 的损失，说明**调整机制本身具有可观的经济价值**——它把 5 倍电价的紧急购电转化为 1.5 倍溢价、把过配的全部损失削减为 50% 违约费用。引入预报标定（V3）后总费用降至 1410.7 万元，说明**标定是发挥预报价值的前提**；再引入双向调整、末端备用与残差权重寻优（V4FR），总费用最终降至 1368.8 万元，较问题二策略降低 5.9%。对比 V4D 与 V4 可知，仅允许下调会因无法应对光伏低估而多支付约 50 万元紧急购电费（88.1 万元对 37.8 万元），说明**双向调整是必要的**。

7.3.3 求解结果与分析

最终策略（V4FR）的全年运行指标如表 9 所示。报告期内实际购电总费用为 **1368.75 万元**，较问题二策略（1454.65 万元）降低 85.90 万元（−5.9%）；紧急购电费用 30.40 万

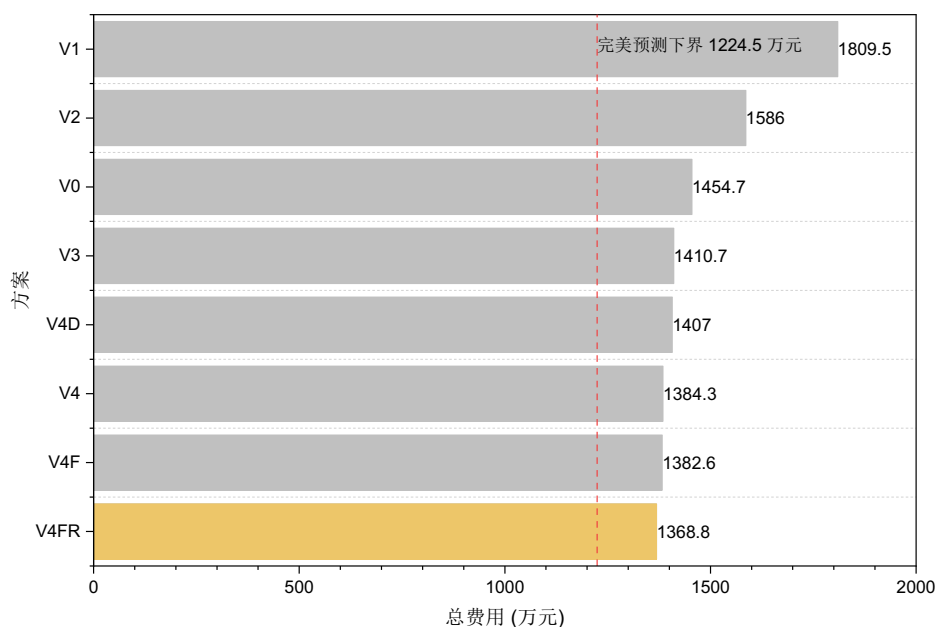


图 9 各策略变体的全年总费用对比（虚线为完美预测下的理论下界 1224.5 万元）

元，较问题二的 87.02 万元下降 65.1%，紧急购电量由 14.51 万 kWh 降至 5.59 万 kWh。这表明日内滚动调整有效吸收了预测偏差，把原本需 5 倍电价兜底的缺口转化为代价较低的调整购电。

表 9 问题三最终策略的年度运行指标（报告期 2025 年 2 月 1 日–12 月 31 日）

指标	数值	占比
计划购电费用（0:00 计划口径）	12863249.43 元	—
按调整购电量实付费用	13062110.51 元	100.0%
调整违约与溢价费用	321415.34 元	2.5%
紧急购电费用（5 倍电价）	303964.84 元	2.3%
全年购电总费用	13687490.69 元（1368.75 万元）	—

四个指定日期的计划与调整结果如表 10 与表 11 所示。调整主要集中在 12:00–12:10 与 18:00–18:10 两个时段：3 月 20 日 12:00 由计划 629.79 kWh 上调至 839.81 kWh；6 月 21 日 18:00 由 445.00 kWh 微调至 479.30 kWh；9 月 23 日 12:00 由 529.25 kWh 上调至 852.32 kWh；12 月 21 日 12:00 由 996.21 kWh 上调至 1256.55 kWh，16:00 由 830.71 kWh 上调至 842.74 kWh，18:00 由 731.57 kWh 微调至 736.34 kWh。四个日期 0:00 与 24:00 的储电量均处于 4115~5206 kWh 的健康备用区间，说明日末备用约束有效避免了储能被放空。

表 10 四个指定日期的计划购电量、调整后购电量、购电费用与紧急购电情况

日期	计划全天/kWh	调整全天/kWh	计划购电费/元	调整后结算/元	紧急购电
2025-03-20	70931.49	71641.61	43191.79	43970.52	无
2025-06-21	35382.09	35651.35	20188.83	21095.57	无
2025-09-23	67776.12	69583.76	42138.53	44294.40	20:30–20:40, 6.38 kWh
2025-12-21	96674.20	98469.77	61965.50	63563.48	8:00–8:10, 29.55 kWh

表 11 四个指定日期在题目指定时间段内的计划购电量 → 调整后购电量 (单位: kWh)

日期	口径	10:00–10:10	12:00–12:10	14:00–14:10	16:00–16:10	18:00–18:10	20:00–20:10
2025-03-20	计划	0.0000	629.7876	0.0000	490.5334	737.7354	0.0000
	调整	0.0000	839.8117	0.0000	490.5334	737.7354	0.0000
2025-06-21	计划	0.0000	0.0000	0.0000	123.9504	444.9959	0.0000
	调整	0.0000	0.0000	0.0000	123.9504	479.2983	0.0000
2025-09-23	计划	0.0000	529.2523	0.0000	422.6452	821.9636	0.0000
	调整	0.0000	852.3236	0.0000	422.6452	821.9636	0.0000
2025-12-21	计划	0.0000	996.2066	0.0000	830.7113	731.5661	0.0000
	调整	0.0000	1256.5490	0.0000	842.7382	736.3372	0.0000

7.3.4 是否需要引入其他时刻的预报

综合上述诊断与对比，回答是：需要，但必须以正确的方式引入。

(1) 后续预报本身的信息增量有限。同一目标时刻的预报精度并不随发布时间改善，其修订量与当日已实现偏差几乎不相关，不宜用最新预报直接替换原计划——这正是 V1 策略费用比问题二策略高 24.4% 的原因。

(2) 后续预报的价值主要体现在“触发调整”与“参与标定”。即使原始预报质量不佳，以其为输入的双向调整机制 (V2) 仍能挽回 63% 的预报损失；经 MOS 标定并与 w1 融合后 (V3、V4FR)，后续预报才真正转化为经济收益。

(3) 当日已实现信息是主要的日内信息增量。以已实现偏差按适度权重修正后续估计 ($\lambda_{pv} = 0.1$ 、 $\lambda_L = 0.3$) 可使总费用再降 7.5 万元；权重过大则追逐噪声、增大调整摩擦，残差更新应当保守使用。

(4) 调整机制的收益来自费率结构的改善。0.5 倍的双向调整费率使模型在“少购违约 ($0.5c_t$)”“多购溢价 ($1.5c_t$)”与“紧急购电 ($5c_t$)”三个通道之间自动比价，把问题二中被动承担的高成本缺口转化为主动的低成本计划调整。

综上，应当引入其他时刻的预报，但应作为经标定与融合后的输入，并配合当日已实现残差的适度修正与日末备用约束统一纳入滚动调整模型，而不是直接以最新预报取代原计划。

八、问题四模型的建立与求解

8.1 建模思路与电价假设

问题四将电价从“每天相同”放宽为逐日实时波动（附件 4，365 天 $\times$ 144 点），要求在波动电价下重新建立问题二与问题三条件下的购电策略模型。本文采用电力市场的通行惯例并显式声明：

假设（日前电价可知）：每日 0:00 制定计划时，当日 144 点电价曲线已由日前市场公布（可知但逐日不同）；紧急购电按交易时刻实际电价的 5 倍结算，调整购电按附件 4 实际价格结算（超额 1.5 倍、违约 0.5 倍，同问题三）。日前电价未知的情形作为推广方向讨论。

在此假设下，问题二、三已建立的策略框架（预测 $\rightarrow$ 日前 LP 计划 $\rightarrow$ 滚动调整 $\rightarrow$ 日内实时调度）**结构完全复用**，唯一变化是所有价格项（计划目标、紧急费率、调整结算）由固定曲线 c_t 换成当日曲线 $c_{d,t}$ 。

8.2 附件 4 电价特征分析

对附件 4 全年电价作统计，结果如表 12 所示。

表 12 附件 4（波动电价）与附件 1（固定电价）的统计特征对比

指标	附件 4（波动）	附件 1（固定）
全年均值/（元 $\cdot$ kWh $^{-1}$ ）	0.7662	0.7662
取值范围/（元 $\cdot$ kWh $^{-1}$ ）	[0.0076, 1.7936]	[0.3713, 1.3952]
日均价范围/（元 $\cdot$ kWh $^{-1}$ ）	[0.5378, 0.9290]	0.7662（每日相同）
日内价差（max-min）均值/（元 $\cdot$ kWh $^{-1}$ ）	1.1073	1.0239

全年均值与附件 1 **完全相同**——附件 4 是“均值不变、只打散波动”的设计，排除了“电价整体上移”的解释，费用变化全部来自结构性因素。进一步考察电价与负载、光伏的日尺度相关结构（图 10）：

（1）电价与负载强正相关： $\text{corr}(\text{日均价}, \text{日负载电量}) = 0.898$ 。高负载日几乎必然是高电价日——能量必须在最贵的日子买入，无法跨日回避。

(2) 电价与光伏负相关：corr（日均价，日光伏电量）= -0.343。光伏大发时电价偏低、缺发时电价偏高，光伏替代购电的“保值”作用被进一步削弱。

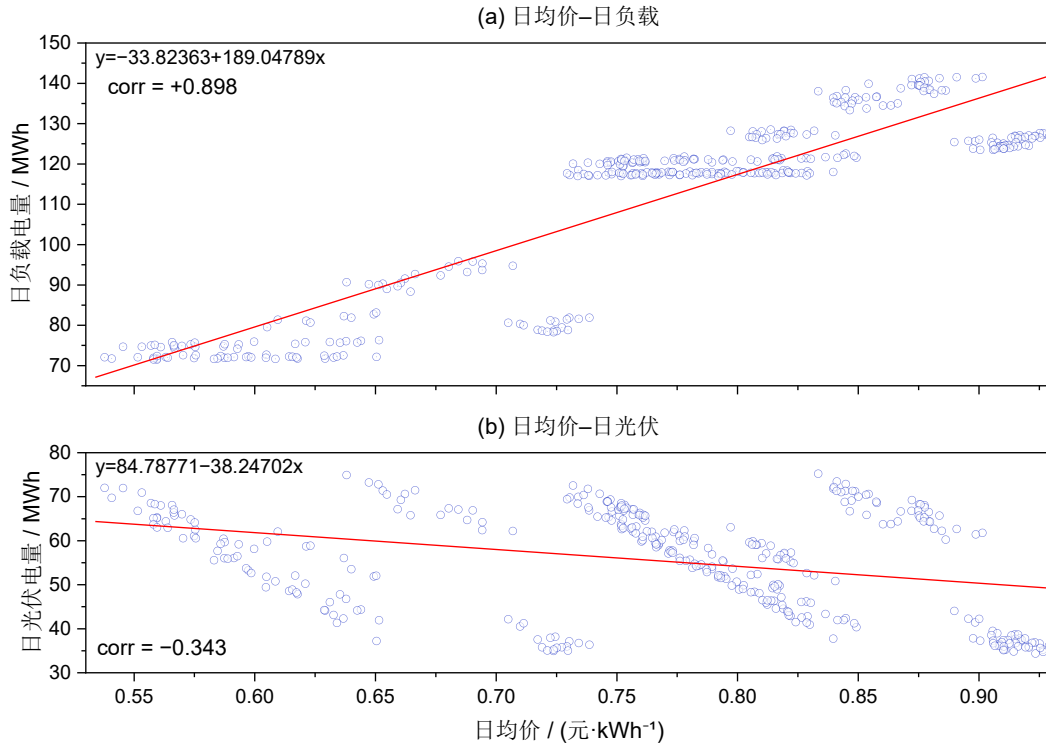


图 10 附件 4 日均价与日负载电量、日光伏电量的散点及回归线：(a) 价—荷相关系数 +0.898；(b) 价—光相关系数 -0.343

8.3 波动电价下的购电策略模型

(1) Q4-2（对应问题二）。w1 预测 + 余量（负载 $\times 1.05$ 、光伏 $\times 0.98$ ）+ 日前 LP ($S_{144} = S_0$ 日循环) + 日内储能纠偏 + 5 倍紧急购电；日前 LP 目标为 $\min \sum_t c_{d,t} p_{d,t} \Delta t$, $c_{d,t}$ 为附件 4 第 d 日曲线。余量参数在附件 4 上复扫后仍最优（详见第九章）。

(2) Q4-3（对应问题三）。标定光伏估计 + 当日残差更新 + 0:00 计划 LP + 6:00/12:00/18:00 双向调整 LP ($c_{d,t}$ 计入目标与安全阀) + 日内纠偏；结算同问题三： $\sum c_{d,t} p_{d,t}^{adj} \Delta t + \sum 0.5 c_{d,t} |p_{d,t}^{adj} - p_{d,t}^{plan}| \Delta t + \sum 5 c_{d,t} e_{d,t} \Delta t$ 。余量参数在波动电价下重新寻优： $\gamma = 1.02$ 保持最优，光伏余量由 $\delta = 0.98$ 调整为 $\delta = 1.00$ （波动电价下光伏边际价值下降，过度下压光伏预测反而抬高购电成本，见第九章表 18）；调整 LP 沿用日末最低备用约束 $S_{144} \geq 4000 \text{ kWh}$ 。

8.4 求解结果

两种策略在波动电价下的全年滚动仿真结果如表 13 与表 14 所示。

表 13 固定电价与波动电价下的全年总费用对比（报告期 334 天）

策略	固定电价（附件 1）	波动电价（附件 4）	变化
Q4-2（问题二策略）	14546503.97 元	15263635.75 元	+717131.78（+4.93%）
Q4-3（问题三策略 V4FR）	13687490.69 元	14315636.38 元	+628145.69（+4.59%）
Q4-3 相对 Q4-2	-5.91%	-6.21%	优势扩大

表 14 波动电价下的费用构成（单位：元）

策略	计划/实付购电费	调整违约/溢价	紧急购电费	紧急购电量/天数
Q4-2	14336659.77	—	926975.98	146798.3 kWh / 105 天
Q4-3	13568042.27	364362.21	383231.90	70819.5 kWh / 199 天

四个指定日期在波动电价下的计算结果如表 15 所示（各日期指定时段购电量与储能充放电量明细见结果文件 result4-2.xlsx 与 result4-3.xlsx）。

表 15 四个指定日期在波动电价下的购电结果

日期	Q4-2（问题二策略）		Q4-3（问题三策略）		
	计划购电量/kWh	购电费/元	计划 → 调整/kWh	计划费 → 结算费/元	紧急购电
2025-03-20	79420.62	49650.82	69723.56→70905.44	43137.99→44365.27	无
2025-06-21	38848.12	19143.71	35919.35→36269.04	17318.23→18024.19	无
2025-09-23	71264.87	45711.78	66530.09→69322.40	42714.01→46352.32	合计 156.48 kWh
2025-12-21	100592.96	76223.89	95937.48→98112.10	71680.77→74388.56	8:00–8:10, 34.76 kWh

Q4-2 仅 12 月 21 日发生紧急购电（8:00–8:10，5.62 kWh），其余三天无；Q4-3 的 9 月 23 日紧急购电为三段合计（9:20–10:00 共 149.02 kWh、18:30–18:40 共 1.08 kWh、20:30–20:40 共 6.38 kWh）。

8.5 结果分析：费用上升的来源与策略适应性

(1) 费用上升的主因是“价—荷同峰”。全年均值不变的前提下，高电价恰好落在高负载日：负载加权的购电均价由固定电价的 0.7955 元/kWh 升至 0.8190 元/kWh（+2.95%），构成费用上升的主要部分；在“每日 0:00 计划 + 日循环”的框架下无法跨日转移负载以规避（跨日套利的价值见下述敏感性实验）。

(2) 光伏的避险价值被削弱。价—光负相关（-0.343）意味着发电时段电价低、缺发

时段电价高，光伏替代购电所避免的单位成本下降；这也是 Q4-3 最优光伏余量由 0.98 变为 1.00 的原因。

(3) 问题三策略对价格波动的适应性更好。V4FR 在波动电价下依然最优，且优势由 5.91% 扩大至 6.21% (节省 94.8 万元)；其上调购电量与当日日内价差正相关 (0.194)——高价差日更多使用 1.5 倍上调通道规避 5 倍高价紧急购电。尾部风险也显著压平：Q4-2 最贵单个 10 min 紧急购电达 8514 元 (7 月 3 日 20:40, 949.34 kWh, 电价 1.7936 元)、最贵一天 81112 元 (6 月 1 日)，紧急购电费的 92.5% 落在电价 > 1.0 元的时段；Q4-3 对应指标降为 5931 元、17792 元与 67.4%。

(4) 波动电价激活了更多储能套利周转。全年储能充电量由固定电价下的 6653.2 MWh (Q4-2) /6502.4 MWh (Q4-3) 升至 7123.4 MWh (+7.1%) /6840.1 MWh (+5.2%)，约每日 2.0 次等效循环；但日维度“日内价差—当日充电量”相关仅 +0.18，说明周转提升主要来自价格形态的整体变化，容量与功率限额仍是主要约束，与问题一中“扩容有正边际收益”的结论一致。

(5) 敏感性：跨日套利的价值。附件 4 日均价在 0.54~0.93 元之间波动，跨日价差甚至大于日内价差。放开 Q4-2 的日循环约束 (计划 LP 末端自由) 重算，总费用降至 15031398 元，再省 232238 元 (-1.5%)——跨日套利是波动电价下真实的价值来源；但代价是储能频繁贴限 (日末均值 2355 kWh)、削弱次日备用，实际运用中须在备用水平与套利收益之间权衡，可作为后续改进方向。

综上，波动电价通过“价—荷同峰”结构使两种策略费用分别上升 4.93% 与 4.59%，问题三的“标定 + 双向调整”框架在波动电价下保持了优势并压平了尾部风险，模型结构无需改变即可量化价格波动的影响。

九、灵敏度分析与误差分析

为检验模型结果对关键参数与输入误差的依赖程度，本章从灵敏度分析与误差分析两方面进行检验：前者考察关键参数变化时最优方案与购电费用的变化幅度，后者考察预测值与实际值的偏差及其经济后果。

9.1 灵敏度分析

9.1.1 问题一：日循环边界与充放电效率

问题一的灵敏度检验结果如表 16 所示。

日循环边界的两种解读对费用影响仅 0.024%，结论稳健；充放电效率由 81% 往返提升至 90% 往返可使费用进一步下降 3.8%，说明效率参数是储能经济性的重要来源；无储能时费用上升 36.8%，再次确认储能的核心价值。约束取等情况：储能上限 10800 kWh 在 14:30–18:00 与 5:40–5:50 取等、下限 1200 kWh 在 20:50–22:00 取等，充电

表 16 问题一灵敏度分析（全天购电费用）

情形	全天购电费/元	与基准差
基准 ($S(0) = S(144) = 6000 \text{ kWh}$)	35126.95	—
首末储电量自由 ($\rightarrow 8550 \text{ kWh}$, 附录 A)	35118.60	-8.35 (-0.024%)
往返效率 90% ($\eta_c = \eta_d = \sqrt{0.9}$)	33793.78	-3.8%
无储能设备	48052.05	+36.8%

功率 5000 kW 在 19 个时段满功率取等——对偶意义下扩大储能容量与充电通道具有正的边际收益。此外，无爬坡成本时 LP 最优解呈“bang-bang”特征（购电曲线相邻区间最大跳变 4994 kW、>500 kW 跳变 22 次）；对购电功率加微小爬坡惩罚 $\varepsilon \sum_t |p(t) - p(t-1)|$ （仍为 LP）， $\varepsilon = 0.005 \text{ 元/kW}$ 时费用仅增加 67.12 元（+0.19%），跳变降至 9 次，实际申报时可采用该平滑化方案。

9.1.2 预测方法与余量参数

模型中不确定性最强、对结果影响最大的是预测方法与预测余量系数。表 17 给出问题二在不同预测方法与余量组合下的全年总费用与紧急购电费用。

表 17 不同预测方法与余量组合下的全年运行费用（问题二，报告期 334 天）

预测方法	(γ, δ)	总费用/万元	紧急购电费/万元
持续性预测（当日 = 昨日）	(1.00, 1.00)	2244.8	1021.1
持续性预测	(1.05, 1.00)	2139.4	793.6
上周同星期（w1）	(1.00, 1.00)	1602.4	378.7
上周同星期（w1, 本文采用）	(1.05, 0.98)	1454.7	87.0
近 4 个同星期均值	(1.05, 1.00)	1574.7	229.7
7 日滑动平均	(1.08, 1.00)	2059.0	698.6

由表 17 可知，预测方法的选取对费用的影响远大于余量系数：持续性预测因周末前后的负载状态切换而严重失效（2244.8 万元）；w1 预测不加余量时为 1602.4 万元，加入余量后降至 1454.7 万元（降 147.7 万元），说明余量策略是应对 5 倍紧急购电惩罚的关键。w1 预测下 γ 在 [1.03, 1.07] 内构成平坦平台（偏移 ± 0.02 时总费用变化不足 1%）；将报告期拆分为 2–8 月与 9–12 月两段分别寻优（时间序列交叉验证 [1]），最优参数与全年结果一致，说明参数并非对全年数据过拟合。

9.1.3 策略参数的稳健性

(1) **储能配置的价值。**同一预测与余量下取消储能设备，问题二全年总费用由 1454.7 万元升至 1893.9 万元，储能年价值 439.2 万元（-23.2%）。

(2) **当日残差更新权重与日末备用。**问题三中 $\lambda = 0$ 时总费用 1376.3 万元，最优 (0.1, 0.3) 处 1368.8 万元，增大至 (0.6, 0.8) 回升至 1414.4 万元（表 7），最优点附近平缓；日末最低备用在 2000~7000 kWh 范围内总费用变化平缓，不改变“标定 + 双向调整优于无调整”的结论，取 4000 kWh 主要出于备用稳健性考量。

(3) **波动电价下的参数复核。**引入波动电价（问题四）后重新扫描余量参数，结果如表 18 所示：Q4-2 最优值仍在 $\gamma = 1.05$ 附近，继续沿用 (1.05, 0.98)；Q4-3 负载侧最优值仍为 $\gamma = 1.02$ ，光伏侧最优余量由 $\delta = 0.98$ 变为 $\delta = 1.00$ ——波动电价下光伏边际价值下降，过度下压光伏预测反而抬高购电成本。电价波动改变了最优光伏余量，但未改变模型结构与整体结论。

表 18 波动电价下余量参数复核（全年总费用，单位：万元）

Q4-2: γ ($\delta = 0.98$)	1.03	1.05	1.07
全年总费用	1530.2	1526.4	1548.6
Q4-3: δ ($\gamma = 1.02$)	0.96	0.98	1.00
全年总费用	1440.3	1433.6	1431.6

9.2 误差分析

9.2.1 光伏预报误差

对附件 3 整点预报与实际出力的对照（图 8）表明，白天时段预报 MAE 最高达 1374.1 kW，并呈现稳定的**系统性偏差**：上午偏低（8:00 约 -1050 kW）、午后偏高（16:00 约 +988 ~ +1067 kW），预报峰值时刻与实际峰值时刻错位。经 MOS 标定后，14:00 的估计 MAE 由原始预报的 793.4 kW 降至 373.0 kW，也优于单独使用上周同星期实际值的 456.8 kW，说明**标定有效削弱了系统性偏差**。由于预报误差不可能完全消除，本文并未追求更高的点预测精度，而是通过安全余量与日内滚动调整吸收剩余误差。

9.2.2 预测误差的经济代价

以“完美预测”为理论下界：在与最终策略完全相同的框架下（含储能、日循环约束与跨日滚动），仅将预测值替换为当日实际值并将余量置 1，重新求解全年调度。完美预测下全年总费用为 1224.5 万元，本文问题二策略为 1454.7 万元，二者相差 230.2 万

元，即**残余预测误差的代价约为总费用的 15.8%**；问题三引入标定与滚动调整后，该差距缩小至 144.3 万元（1368.8 万元对 1224.5 万元），说明滚动调整机制确实降低了预测误差的经济损失。进一步考察误差代价的分布：问题二紧急购电的平均结算电价为 1.094 元/kWh，约为全天均价的 1.4 倍——供电缺口较多落在傍晚高电价时段，5 倍惩罚在最贵时段被放大，这正是问题三滚动调整收益的主要来源。

9.2.3 余量策略的固有代价与数值校验

日前计划施加安全余量的代价是部分已付费电量最终无法使用：问题二全年弃电量约 326 万 kWh（占计划购电量 14.6%），其中“已付费却被迫丢弃”的电量下限约 81 万 kWh，按均价估算浪费不低于 60 万元（约占总费用 4%）。这是“日前承诺 + 不确定性对冲”的必然成本，与所避免的约 790 万元紧急购电费用相比净收益显著。

最后，对求解结果作全链路数值校验：（1）各时段功率平衡与储能状态递推均成立；（2）储能轨迹全程位于 [1200, 10800] kWh，充放电功率不超过 5000 kW；（3）跨日储电量按实际末态连续传递，无跳变；（4）各日费用按“计划购电费 + 调整费用 + 紧急购电费”分解后与总费用一致；（5）调整模型的解可由重新求解线性规划复现。上述校验在 334 天滚动仿真中均未发现违规；全部数值结果经 Python 与 MATLAB 双平台独立求解逐位一致，说明求解结果自洽且可复现。

十、模型评价与推广

10.1 模型的优点

（1）预测模型由数据规律驱动，可解释性强。本文并未直接套用通用时间序列方法，而是先对全年数据作统计分析，发现负载的星期型双状态结构（周日至周四约 124.0 MWh，周五、周六约 78.3 MWh），据此选择“上周同星期实际值”作为预测基础，避免持续性预测在状态切换日失效；光伏出力具有强季节性与日间持续性（自相关系数 0.964），支持使用近期历史数据。模型变量与参数均有明确的实际含义。

（2）关键参数具有理论依据而非经验试凑。5 倍电价惩罚使购电决策具有报童模型结构，临界分位数 0.8 要求计划覆盖净负荷约 80% 分位点，据此设置的安全余量在全年扫描中形成平坦平台（ γ 在 [1.03, 1.07] 内总费用变化不足 1%），分段验证结论一致，参数稳健、无过拟合。

（3）模型分层递进，四个问题共用同一优化内核。从问题一的日循环线性规划，到问题二“预测—余量—日前计划—日内纠偏—跨日滚动”，再到问题三“预报标定—残差修正—双向调整—末端备用”与问题四的价格项替换，模型结构始终一致、各层功能清晰，便于横向比较不同策略的经济性。

(4) 求解可靠、结果可复现且具有经济含义。日前计划与调整计划均为线性规划，可求得全局最优；日内纠偏规则透明、物理可行。全链路数值校验在 334 天滚动仿真中零违规，Python 与 MATLAB 双平台求解结果逐位一致；单次全年仿真在数分钟内完成，使参数大规模扫描寻优成为可能。从结果看，储能设备年价值 439.2 万元；问题三把 5 倍紧急购电转化为 0.5 倍与 1.5 倍调整结算后，全年费用较问题二下降 5.9%，紧急购电量由 14.51 万 kWh 降至 5.59 万 kWh，与费率结构改善的机理吻合。

10.2 模型的不足

(1) 预测仍为点预测加比例余量。未建立负载与光伏的联合概率分布或场景集，余量按全天统一设置、未按时段细化，存在固有代价（问题二全年弃电约 326 万 kWh，其中已付费却被迫丢弃的电量下限约 81 万 kWh）。采用分布鲁棒优化 [4] 或分时段余量有望进一步压缩该损失。

(2) 日内调度采用贪心规则。储能纠偏按固定优先级执行，未在日内逐时段重新优化，与模型预测控制（MPC）[5] 相比仍留有优化空间（估计 1%~2% 量级）。

(3) 电价不确定性与参数寻优方式尚不完善。问题四假设计划制定时当日电价已知，电价未知时应引入价格场景集或不确定集，将模型扩展为两阶段随机规划；余量系数、残差权重与备用水平均以全年数据寻优，虽已通过平台分析与分段验证排除过拟合，严格意义上仍应采用滚动交叉验证。

(4) 目标函数未考虑储能循环损耗。模型以购电费用最小为唯一目标，未计入充放电次数带来的寿命损耗；引入单位循环成本后储能周转量将相应下降。

10.3 模型的推广

本文的核心思想——**在不对称惩罚下用保守余量制定日前计划、用日内滚动调整校正预测偏差**——不依赖具体场景，可推广到其他“日前承诺、实时考核”型决策问题。

(1) 同类电力系统与设备。风电场与光伏电站的日前出力申报、独立储能电站的峰谷套利、园区综合能源系统调度、需求响应中可中断负荷的报价等，均具有“计划量按合同结算、偏差按惩罚电价结算”的结构，可直接沿用本文两阶段框架，只需替换考核系数（5 倍、1.5 倍、0.5 倍）与设备的容量—功率—效率参数。

(2) 数据与机制变化下的调整。预报发布更频繁时可加密调整时刻；电价未知时可将固定电价替换为价格场景集，扩展为两阶段随机规划或鲁棒优化；充放电效率随荷电状态变化时可将线性储能模型替换为分段线性模型。

(3) 方法层面的改进方向。预测环节可用 LSTM、Transformer 等机器学习模型或概率预测方法 [6] 替代 MOS 线性回归；日内调度可由贪心规则升级为模型预测控制；还可引入储能寿命模型，把循环损耗纳入目标函数。

十一、AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码调试与结果复算、图表绘制、论文写作辅助与语言润色，详细使用情况见支撑材料。

十二、参考文献

- [1] Hyndman R J, Athanasopoulos G. Forecasting: Principles and Practice[M/OL]. 3rd ed. Melbourne: OTexts, 2021. <https://otexts.com/fpp3/> (访问时间: 2026-09-13) . 本文引用第 2 章 (时间序列图形与自相关)、第 5 章 (预测工具箱: 5.2 简单预测方法、5.4 点预测精度评估、5.6 时间序列交叉验证)、第 7 章 (时间序列回归模型) .
- [2] Arrow K J, Harris T, Marschak J. Optimal inventory policy[J]. Econometrica, 1951, 19(3): 250-272.
- [3] Glahn H R, Lowry D A. The use of Model Output Statistics (MOS) in objective weather forecasting[J]. Journal of Applied Meteorology, 1972, 11(8): 1203-1211.
- [4] Bertsimas D, Litvinov E, Sun X A, et al. Adaptive robust optimization for the security constrained unit commitment problem[J]. IEEE Transactions on Power Systems, 2013, 28(1): 52-63.
- [5] Mayne D Q, Rawlings J B, Rao C V, et al. Constrained model predictive control: Stability and optimality[J]. Automatica, 2000, 36(6): 789-814.
- [6] Hong T, Pinson P, Fan S, et al. Probabilistic energy forecasting: Global Energy Forecasting Competition 2014 and beyond[J]. International Journal of Forecasting, 2016, 32(3): 896-913.

A 问题一对照情形：首末储电量由模型自由确定

问题一题面仅要求“储能设备 0:00 与 24:00 的储电量相同”，附录 1 另给出 2025 年 1 月 1 日 0:00 的初始储电量 6000 kWh。主文按附录 1 的给定值取 $S(0) = S(144) = 6000$ kWh 求解；本附录给出另一种解读的对照情形：仅保留首末相等约束 $S(144) = S(0)$ ，循环起点储电量由模型自由优化确定（记为情形 A，主文情形相应记为情形 B）。

求解结果：情形 A 的最优循环起点为 $S(0) = S(144) = 8550.00$ kWh，全天购电费 35118.60 元，较情形 B（35126.95 元）仅降低 8.35 元（0.024%）；两情形全天购电量均

为 59482.70 kWh，题目指定的六个时段购电量完全相同。差异仅在于夜间充电在“20:40 储能触底后的 21:00–24:00”与“次日 0:00–6:00”两个低价窗口之间的分配：情形 B 起点低 2550 kWh，需在 0:00–4:00 段多充 2833.33 kWh，20:00–24:00 段相应少充；两窗口电价略有差异（21:00–24:00 约 0.39~0.44 元，0:00–4:00 约 0.42~0.43 元）导致 8.35 元费用差。6:00 之后两情形轨迹完全重合（均于 6:00 充满至 10800 kWh）。两情形的储能分时段充放电量对照如表 19 所示，情形 A 的调度过程如图 11 所示。

表 19 问题一两种日循环边界情形的结果对照

时间段	情形 A ($S(0)$ 自由 $\rightarrow$ 8550 kWh)		情形 B ($S(0) = 6000$ kWh, 主文)	
	充电量/kWh	放电量/kWh	充电量/kWh	放电量/kWh
0:00–4:00	1666.6667	0.0000	4500.0000	0.0000
4:00–8:00	833.3333	5947.4197	833.3333	5947.4197
8:00–12:00	3954.6310	2121.4184	3954.6310	2121.4184
12:00–16:00	6119.3685	91.1014	6119.3685	91.1014
16:00–20:00	0.0000	5068.6869	0.0000	5068.6869
20:00–24:00	8166.6667	3571.3131	5333.3333	3571.3131
全天购电费	35118.60 元		35126.95 元 (+8.35 元)	

两种解读下购电策略与费用实质一致：主文采用附录 1 给定初始值的情形 B，情形 A 作为稳健性佐证列于此；对应结果文件分别为 result1.xlsx（情形 A）与 result1_fix6000.xlsx（情形 B，主文口径）。

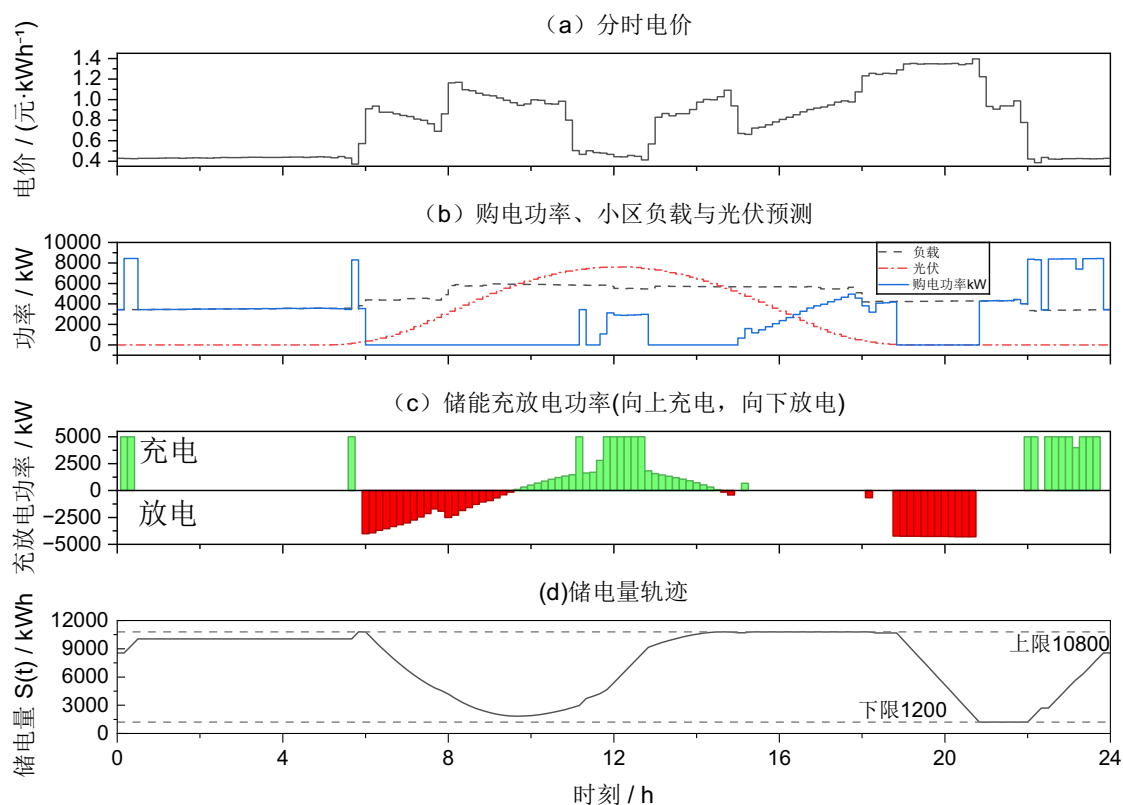


图 11 情形 A (首末储电量由模型自由确定为 8550.00 kWh, 全天购电费 35118.60 元) 的最优日前调度结果, 各子图含义同正文图 2

B 支撑材料文件列表

文件/目录	内容说明
结果文件/result1.xlsx	问题一结果 (情形 A: $S(0)$ 自由 $\rightarrow$ 8550 kWh)
结果文件/result1_fix6000.xlsx	问题一结果 (情形 B: $S(0) = 6000$ kWh, 主文口径)
结果文件/result2.xlsx	问题二结果 (计划购电量、充放电量、紧急购电量三表)
结果文件/result3.xlsx	问题三结果 (计划/调整购电量、充放电量、紧急购电量四表)
结果文件/result4-2.xlsx	问题四结果 (对应问题二策略, 波动电价)
结果文件/result4-3.xlsx	问题四结果 (对应问题三策略, 波动电价)
源程序/matlab/Q1-Q4/problem*.m	问题一至四的 MATLAB 求解程序 (提交复算用)
源程序/python/Q1-Q4/*.py	对应的 Python 参考实现 (与 MATLAB 结果逐位一致)
源程序/README.pdf	程序运行环境、依赖与运行方式说明
所有图表/	正文全部图件的源文件 (按问题分目录)
人工智能使用情况/AI_工具使用详情.pdf	AI 工具使用情况说明

C 源程序

以下给出四个问题的全部可运行源程序。MATLAB 版与 Python 版为同一算法的双实现，二者结果逐位一致（运行环境、数据路径说明见支撑材料 README）。

3.1 MATLAB 程序

3.1.1 问题一：problem1.m（日循环线性规划，双情形）

```
1 %% CUMCM2026 C题 问题1 —— 微网日前计划购电策略（线性规划，双情形）
2 % 约定：附件1 第 i 行(时刻戳) = 以该时刻为起点的 10 min 区间，
3 % 与 result1.xlsx 模板"计划购电量"表行序一一对应。
4 % 变量：x = [p; ch; dis; q; s]
5 % p(t) 购电功率 kW, ch(t)/dis(t) 充/放电功率 kW, q(t) 弃光功率 kW
6 % s(k) 第 k 段结束时的储电量 kWh, k = 0..144（共145个）
7 % 双情形：A: S(0)自由(最优8550, 费用35118.60元); B: S(0)=6000(费用35126.95元)
8 % 参考解(Python/HiGHS)：两情形购电量均 59482.70 kWh —— 用于交叉验证。
9 clear; clc; close all;
10
11 %% ----- 1. 读取数据 -----
12 thisdir = fileparts(mfilename('fullpath'));
13 fdata = fullfile(thisdir, '..', '附件', '附件1.xlsx');
14 M = readmatrix(fdata, 'Range', 'B2:D145'); % 电价/负载/光伏（跳过时刻列）
15 c = M(:,1); % 电价 元/kWh
16 L = M(:,2); % 小区负载 kW
17 pv = M(:,3); % 光伏发电预测功率 kW
18 T = numel(c); assert(T == 144, '数据行数应为144');
19 dt = 1/6; % 10 min = 1/6 h
20
21 %% ----- 2. 储能参数(附录1) -----
22 eta_c = 0.9; eta_d = 0.9; % 充/放电效率
23 Pmax = 5000; % 最大充放电功率 kW
24 Smin = 1200; Smax = 10800; % 运行储电量上下限 kWh
25
26 %% ----- 3. 变量索引 -----
27 ip = 1:T; ich = T+1:2*T;
28 idi = 2*T+1:3*T; iq = 3*T+1:4*T;
29 idxs = 4*T+1:5*T+1; % s(0)..s(144)
30 nx = 5*T + 1;
31
32 %% ----- 4. 目标函数: min sum c(t)*p(t)*dt -----
33 fobj = zeros(nx,1); fobj(ip) = c*dt;
34
35 %% ----- 5. 等式约束(公共部分) -----
36 % (a) 功率平衡: p(t) + pv(t) + dis(t) = L(t) + ch(t) + q(t)
37 % (b) 储能动态: s(t+1) = s(t) + eta_c*ch(t)*dt - dis(t)*dt/eta_d
38 % (c) 周期约束: s(144) = s(0) (即 0:00 与 24:00 储电量相同)
39 Ii = [1:T, 1:T, 1:T, 1:T, ... % (a)
40       T+(1:T), T+(1:T), T+(1:T), T+(1:T), ... % (b)
41       2*T+1, 2*T+1]; % (c)
42 Jj = [ip, idi, ich, iq, ...
43       idxs(2:T+1), idxs(1:T), ich, idi, ...
44       idxs(1), idxs(T+1)];
45 Vv = [ones(1,T), ones(1,T), -ones(1,T), -ones(1,T), ...
46       ones(1,T), -ones(1,T), -eta_c*dt*ones(1,T), (dt/eta_d)*ones(1,T), ...
47       1, -1];
48 Aeq0 = sparse(Ii, Jj, Vv, 2*T+1, nx);
49 beq0 = [L - pv; zeros(T,1); 0];
50
51 %% ----- 6. 上下界 -----
```

```

52 lb = zeros(nx,1); ub = inf(nx,1);
53 ub(ich) = Pmax; ub(idi) = Pmax;
54 lb(idxs) = Smin; ub(idxs) = Smax;
55
56 %% ----- 7. 双情形求解 -----
57 % 情形A: S(0) 自由优化 (稳态循环解读) -> result1.xlsx
58 % 情形B: S(0)=6000 kWh 固定 (附录1初始值解读) -> result1_fix6000.xlsx
59 cases = {'A', [], 'result1.xlsx';
60          'B', 6000, 'result1_fix6000.xlsx'};
61 opts = optimoptions('linprog', 'Algorithm', 'dual-simplex', 'Display', 'off');
62
63 for ic = 1:size(cases,1)
64     tag = cases{ic,1};
65     s0fix = cases{ic,2};
66     fout = fullfile(thisdir, cases{ic,3});
67
68     Aeq = Aeq0; beq = beq0;
69     if ~isempty(s0fix) % (d) 固定 S(0)=s0fix
70         r = sparse(1, nx); r(idxs(1)) = 1;
71         Aeq = [Aeq; r]; beq = [beq; s0fix];
72     end
73
74     [x, fval, exitflag] = linprog(fobj, [], [], Aeq, beq, lb, ub, opts);
75     assert(exitflag == 1, 'linprog 未收敛, exitflag = %d', exitflag);
76     fprintf('\n===== 情形 %s =====\n最优购电费用 = %.4f 元\n', tag, fval);
77
78     %% 后处理
79     p = x(ip); ch = x(ich); dis = x(idi); q = x(iq); s = x(idxs);
80     Ep = p*dt; Ech = ch*dt; Edis = dis*dt; Eq = q*dt; % kWh
81     tol = 1e-6;
82     Ep(abs(Ep)<tol)=0; Ech(abs(Ech)<tol)=0; Edis(abs(Edis)<tol)=0; Eq(abs(Eq)<tol)=0;
83
84     % --- 表1: 指定时段购电量 (时刻起点=数据行号: 10:00->60, 12:00->72, ...) ---
85     rowT1 = [60 72 84 96 108 120];
86     nameT1 = {'10:00-10:10', '12:00-12:10', '14:00-14:10', ...
87              '16:00-16:10', '18:00-18:10', '20:00-20:10'};
88     fprintf('--- 表1 购电量 ---\n');
89     for k = 1:6
90         fprintf('%s: %.4f kWh\n', nameT1{k}, Ep(rowT1(k)));
91     end
92     fprintf('全天购电量: %.4f kWh\n', sum(Ep));
93
94     % --- 表2: 4h 分段充放电量 (行144=0:00+1-0:10+1即时钟0:00-0:10, 归入 0:00-4:00) ---
95     blocks = {[144, 1:23], 24:47, 48:71, 72:95, 96:119, 120:143};
96     bname = {'0:00-4:00', '4:00-8:00', '8:00-12:00', '12:00-16:00', '16:00-20:00', '20:00-24:00'};
97     Ebch = zeros(6,1); Ebdis = zeros(6,1);
98     fprintf('--- 表2 充放电量 ---\n');
99     for k = 1:6
100         Ebch(k) = sum(Ech(blocks{k})); Ebdis(k) = sum(Edis(blocks{k}));
101         fprintf('%s: 充电 %.4f kWh 放电 %.4f kWh\n', bname{k}, Ebch(k), Ebdis(k));
102     end
103     s000 = s(144); s2400 = s(144); % 时钟0:00/24:00 在第143、144行之间
104     fprintf('0:00 储电量: %.4f kWh 24:00 储电量: %.4f kWh\n', s000, s2400);
105     fprintf('弃光总量: %.4f kWh, 储电量范围 [%.2f, %.2f] kWh\n', sum(Eq), min(s), max(s));
106     fprintf('功率平衡最大残差: %.2e kW (应=0)\n', max(abs(p + pv + dis - L - ch - q)));
107
108     %% 写结果文件 (按模板版式重建)
109     C1 = cell(145,2);
110     C1(1,:) = {'时间段', '购电量'};
111     for i = 1:T
112         C1{i+1,1} = [timetag(10*i), '-', timetag(10*(i+1))];
113         C1{i+1,2} = round(Ep(i), 4);
114     end
115     C2 = cell(7,5);

```

```

116     C2(1,:) = {'时间段','充电量','放电量','时刻','储电量'};
117     for k = 1:6
118         C2{k+1,1} = bname{k};
119         C2{k+1,2} = round(Ebch(k), 4);
120         C2{k+1,3} = round(Ebdis(k), 4);
121     end
122     C2{2,4} = '0:00'; C2{2,5} = round(s000, 4);
123     C2{3,4} = '24:00'; C2{3,5} = round(s2400, 4);
124     if exist(fout,'file'), delete(fout); end
125     writecell(C1, fout, 'Sheet', '计划购电量');
126     writecell(C2, fout, 'Sheet', '充放电电量');
127     fprintf('已写出 %s\n', fout);
128
129     if ic == 1 % 保存情形A轨迹用于绘图
130         pA = p; chA = ch; disA = dis; sA = s;
131     end
132 end
133
134 %% ----- 10. 绘图(情形A, 论文用图) -----
135 tt = (1:T)/6; % 小时(近似)
136 figure('Position',[100 100 900 720]);
137 subplot(4,1,1); stairs(tt,c); ylabel('电价(元/kWh)'); grid on; title('电价曲线');
138 subplot(4,1,2); stairs(tt,pA); hold on; stairs(tt,L); ylabel('功率(kW)');
139 legend('购电功率','小区负载'); grid on; title('购电功率与负载');
140 subplot(4,1,3); bar(tt,[chA,-disA],1,'stacked'); ylabel('功率(kW)');
141 legend('充电','放电(取负)'); grid on; title('储能充放电功率');
142 subplot(4,1,4); stairs(0:T, sA); ylabel('储电量(kWh)'); xlabel('时间(10min格)');
143 hold on; yline(Smin,'--'); yline(Smax,'--'); grid on; title('储电量变化(情形A)');
144 saveas(gcf, fullfile(thisdir, 'q1_fig.png'));
145
146 %% ----- 局部函数 -----
147 function s = timetag(m) % 分钟数 -> 'H:MM' 或 'H:MM+1'
148 if m >= 1440
149     s = sprintf('%d:%02d+1', floor((m-1440)/60), mod(m-1440,60));
150 else
151     s = sprintf('%d:%02d', floor(m/60), mod(m,60));
152 end
153 end

```

3.1.2 问题二：problem2.m (w1 预测 + 余量 + 日前 LP + 日内纠偏，全年滚动)

```

1 %% CUMCM2026 C题 问题2 —— 全年滚动计划购电策略 (最终策略)
2 % 策略: w1预测(上周同星期实际) + 余量(负载×1.05, 光伏×0.98)
3 % 日前: 日LP计划(日循环 S(144)=S(0)=S0, S0=前日实际末储电量; 1月1日S0=6000)
4 % 日内: 计划购电照付不议; 实际偏差储能限额内调节(缺口: 减充电→加放电→紧急购电5倍;
5 %       富余: 减放电→加充电→弃光)
6 % 参考解(Python/HiGHS): 总费用 14,546,504 元 = 计划 13,676,311 + 紧急 870,192 元,
7 %       紧急购电量 145,134 kWh, 紧急天数 102/334.
8 warning('off','all');
9 clear; clc;
10 thisdir = fileparts(mfilename('fullpath'));
11
12 %% ----- 1. 读取数据 -----
13 r1 = readmatrix(fullfile(thisdir, '..', '附件', '附件1.xlsx'), 'Range', 'B2:D145');
14 price = r1(:,1); L_typ = r1(:,2); PV_typ = r1(:,3);
15 LD = readmatrix(fullfile(thisdir, '..', '附件', '附件2.xlsx'), 'Sheet', '小区负载', 'Range', 'B2:E0366');
16 PVD = readmatrix(fullfile(thisdir, '..', '附件', '附件2.xlsx'), 'Sheet', '光伏发电实际功率', 'Range', 'B2:E0366');
17 NDAY = size(LD,1); assert(NDAY==365 && size(LD,2)==144, '附件2尺寸异常');
18 T = 144; dt = 1/6;
19
20 %% ----- 2. 参数 -----

```

```

21 ETA_C = 0.9; ETA_D = 0.9; PMAx = 5000; SMIN = 1200; SMAx = 10800; EMULT = 5;
22 GAMMA = 1.05; DELTA = 0.98; % 余量系数(扫描选定)
23 ip = 1:T; ich = T+1:2*T; idi = 2*T+1:3*T; iq = 3*T+1:4*T; idxs = 4*T+1:5*T+1;
24 nx = 5*T + 1;
25 feb1 = 32; % 2月1日是第32天
26
27 % LP 公共部分
28 fobj = zeros(nx,1); fobj(ip) = price*dt;
29 Ii = [1:T, 1:T, 1:T, 1:T, T+(1:T), T+(1:T), T+(1:T), T+(1:T)];
30 Jj = [ip, idi, ich, iq, idxs(2:T+1), idxs(1:T), ich, idi];
31 Vv = [ones(1,T), ones(1,T), -ones(1,T), -ones(1,T), ...
32      ones(1,T), -ones(1,T), -ETA_C*dt*ones(1,T), (dt/ETA_D)*ones(1,T)];
33 AeqBase = sparse(Ii, Jj, Vv, 2*T, nx);
34 lb = zeros(nx,1); ub = inf(nx,1);
35 ub(ich) = PMAx; ub(idi) = PMAx; lb(idxs) = SMIN; ub(idxs) = SMAx;
36 opts = optimoptions('linprog','Algorithm','dual-simplex','Display','off');
37
38 %% ----- 3. 全年滚动 -----
39 plans_p = zeros(NDAY,T); Emerg = zeros(NDAY,T);
40 ch_r = zeros(NDAY,T); dis_r = zeros(NDAY,T);
41 Sall = zeros(NDAY,T+1); plan_cost = zeros(NDAY,1);
42 SO = 6000; % 附录1: 2025-1-1 0:00
43 for d = 1:NDAY
44     % --- w1 预测 ---
45     if d == 1, Lhat = L_typ; PVhat = PV_typ;
46     elseif d >= 8, Lhat = LD(d-7,:); PVhat = PVD(d-7,:);
47     else, Lhat = LD(d-1,:); PVhat = PVD(d-1,:);
48     end
49     % --- 日前 LP ---
50     beq = [Lhat*GAMMA - PVhat*DELTA; zeros(T,1)];
51     rS = sparse(1,nx); rS(idxs(1)) = 1;
52     rE = sparse(1,nx); rE(idxs(T+1)) = 1;
53     Aeq = [AeqBase; rS; rE]; beq = [beq; SO; SO];
54     [x, fval, exitflag] = linprog(fobj, [], [], Aeq, beq, lb, ub, opts);
55     assert(exitflag==1, '第%d天计划LP未收敛', d);
56     ps = x(ip); chs = x(ich); diss = x(idi);
57     % --- 日内模拟 ---
58     e = zeros(T,1); cr = zeros(T,1); dr = zeros(T,1); S = zeros(T+1,1); S(1) = SO;
59     for t = 1:T
60         dis_max = min(PMAx, max(S(t)-SMIN,0)*ETA_D/dt);
61         ch_max = min(PMAx, max(SMAx-S(t),0)/ETA_C/dt);
62         dis = min(diss(t), dis_max); ch = min(chs(t), ch_max);
63         resid = (LD(d,t) - ps(t) - PVD(d,t)) - (dis - ch);
64         if resid > 0
65             r = min(ch, resid); ch = ch - r; resid = resid - r;
66             r = min(dis_max - dis, resid); dis = dis + r; resid = resid - r;
67             e(t) = max(resid, 0);
68         else
69             sur = -resid;
70             r = min(dis, sur); dis = dis - r; sur = sur - r;
71             r = min(ch_max - ch, sur); ch = ch + r; sur = sur - r;
72         end
73         cr(t) = ch; dr(t) = dis;
74         S(t+1) = S(t) + ETA_C*ch*dt - dis*dt/ETA_D;
75     end
76     plans_p(d,:) = ps'; Emerg(d,:) = e'; ch_r(d,:) = cr'; dis_r(d,:) = dr';
77     Sall(d,:) = S'; plan_cost(d) = fval;
78     SO = S(T+1);
79 end
80
81 %% ----- 4. 全年统计 -----
82 E_p = sum(plans_p(feb1:end,:), 'all')*dt;
83 C_p = sum(plan_cost(feb1:end));
84 E_e = sum(Emerg(feb1:end,:), 'all')*dt;

```

```

85 C_e = sum(Emerg(feb1:end,:).*price'.*EMULT,'all')*dt;
86 fprintf('==== 问题2 最终策略 =====\n');
87 fprintf('计划购电量 %.1f kWh, 计划购电费 %.2f 元\n', E_p, C_p);
88 fprintf('紧急购电量 %.1f kWh, 紧急购电费 %.2f 元\n', E_e, C_e);
89 fprintf('总费用 %.2f 元, 紧急天数 %d/334\n', C_p+C_e, nnz(sum(Emerg(feb1:end,:),2)>1e-9));
90 fprintf('储电量范围 [%.1f, %.1f] kWh\n', min(Sall,[],'all'), max(Sall,[],'all'));
91
92 %% ----- 5. 表3 指定日期 -----
93 blocks = {[144, 1:23], 24:47, 48:71, 72:95, 96:119, 120:143};
94 bname = {'0:00-4:00','4:00-8:00','8:00-12:00','12:00-16:00','16:00-20:00','20:00-24:00'};
95 rowT1 = [60 72 84 96 108 120];
96 nameT1 = {'10:00-10:10','12:00-12:10','14:00-14:10','16:00-16:10','18:00-18:10','20:00-20:10'};
97 dvec = datetime(2025,1,1) + days(0:364);
98 target = [datetime(2025,3,20), datetime(2025,6,21), datetime(2025,9,23), datetime(2025,12,21)];
99 for td = target
100     d = find(dvec == td);
101     Ep = plans_p(d,:)*dt;
102     fprintf('\n--- %s ---\n', datestr(td,'yyyy.mm.dd'));
103     fprintf('全天计划购电量 %.2f kWh, 计划购电费 %.2f 元\n', sum(Ep), plan_cost(d));
104     for k = 1:6, fprintf(' %s: %.4f\n', nameT1{k}, Ep(rowT1(k))); end
105     for k = 1:6
106         if k == 1
107             ebch = ch_r(d-1,144)*dt + sum(ch_r(d,1:23))*dt;
108             ebdis = dis_r(d-1,144)*dt + sum(dis_r(d,1:23))*dt;
109         else
110             ebch = sum(ch_r(d,blocks{k}))*dt; ebdis = sum(dis_r(d,blocks{k}))*dt;
111         end
112         fprintf(' %s: 充电 %.4f 放电 %.4f\n', bname{k}, ebch, ebdis);
113     end
114     s000 = 6000; if d > 1, s000 = Sall(d-1,144); end
115     fprintf(' s(0:00)=%.2f, s(24:00)=%.2f\n', s000, Sall(d,144));
116     % 紧急窗口
117     e = Emerg(d,:); t = 1;
118     if all(e<=1e-9), fprintf(' 紧急购电: 无\n'); end
119     while t <= T
120         if e(t) > 1e-9
121             j = t; while j<T && e(j+1)>1e-9, j = j+1; end
122             fprintf(' 紧急购电 %s-%s: %.4f kWh\n', timetag(10*t), timetag(10*(j+1)), sum(e(t:j))*dt);
123             t = j+1;
124         else, t = t + 1; end
125     end
126 end
127
128 %% ----- 6. 写 result2.xlsx -----
129 fout = fullfile(thisdir, 'result2.xlsx');
130 if exist(fout,'file'), delete(fout); end
131 C1 = cell(335, 147);
132 C1(1,:) = [{'日期\时间'}, arrayfun(@(i) [timetag(10*i),'-',timetag(10*(i+1))], 1:143, 'UniformOutput',false), ...
133     {'0:00-0:10+1','全天购电量','全天购电费'}];
134 C1{1,43} = '7:0-7:10'; % 保留官方模板第43列的 typo 以逐字一致
135 for i = 1:334
136     d = feb1 - 1 + i;
137     Ep = plans_p(d,:)*dt;
138     C1{i+1,1} = dvec(d);
139     for t = 1:144, C1{i+1,1+t} = round(Ep(t),4); end
140     C1{i+1,146} = round(sum(Ep),4);
141     C1{i+1,147} = round(plan_cost(d),4);
142 end
143 writecell(C1, fout, 'Sheet', '计划购电量');
144
145 C2 = cell(1+334*6, 6);
146 C2(1,:) = {'日期','时间段','充电量','放电量','时刻','储电量'};
147 for i = 1:334
148     d = feb1 - 1 + i;

```

```

149     s000 = 6000; if d > 1, s000 = Sall(d-1,144); end
150     for k = 1:6
151         r = (i-1)*6 + k + 1;
152         if k == 1, C2{r,1} = dvec(d); end
153         C2{r,2} = bname{k};
154         if k == 1
155             C2{r,3} = round(ch_r(d-1,144)*dt + sum(ch_r(d,1:23))*dt, 4);
156             C2{r,4} = round(dis_r(d-1,144)*dt + sum(dis_r(d,1:23))*dt, 4);
157         else
158             C2{r,3} = round(sum(ch_r(d,blocks{k}))*dt, 4);
159             C2{r,4} = round(sum(dis_r(d,blocks{k}))*dt, 4);
160         end
161         if k == 1, C2{r,5} = '0:00'; C2{r,6} = round(s000,4); end
162         if k == 2, C2{r,5} = '24:00'; C2{r,6} = round(Sall(d,144),4); end
163     end
164 end
165 writecell(C2, fout, 'Sheet', '充放电量');
166
167 C3 = cell(100000,3); nrow = 0;
168 C3(1,:) = {'日期', '购电时间段', '购电量'}; nrow = 1;
169 for i = 1:334
170     d = febl - 1 + i; e = Emerg(d,:); t = 1; first = true;
171     while t <= T
172         if e(t) > 1e-9
173             j = t; while j<T && e(j+1)>1e-9, j = j+1; end
174             nrow = nrow + 1;
175             if first, C3{nrow,1} = dvec(d); first = false; end
176             C3{nrow,2} = [timetag(10*t), '-', timetag(10*(j+1))];
177             C3{nrow,3} = round(sum(e(t:j))*dt, 4);
178             t = j+1;
179         else, t = t + 1; end
180     end
181 end
182 writecell(C3(1:nrow,:), fout, 'Sheet', '紧急购电量');
183 fprintf('\n已写出 %s\n', fout);
184
185 %% ----- 局部函数 -----
186 function s = timetag(m)
187 if m >= 1440
188     s = sprintf('%d:%02d+1', floor((m-1440)/60), mod(m-1440,60));
189 else
190     s = sprintf('%d:%02d', floor(m/60), mod(m,60));
191 end
192 end

```

3.1.3 问题三：problem3.m（预报标定 + 残差修正 + 双向调整 LP，全年滚动）

```

1 %% CUMCM2026 C题 问题3 —— 计划+滚动调整购电策略（最终策略 V4FR: V4F+末端备用>=4000）
2 % 标定光伏(逐时DLS[官方报,w1],30天滚动) + 双向调整(超额1.5倍总价/违约0.5倍) + 日内储能调节 + 紧急5倍
3 % 参考解(Python/HiGHS): 总费用 13,771,900 元级 (V4FR, 精确值见 summary3.json)
4 % 调整: 上调 638,431 kWh / 下调 457,921 kWh; 紧急购电量 91,359 kWh, 紧急天数 222/334
5 clear; clc;
6 thisdir = fileparts(mfilename('fullpath'));
7
8 %% ----- 1. 读取数据 -----
9 r1 = readmatrix(fullfile(thisdir, '..', '附件', '附件1.xlsx'), 'Range', 'B2:D145');
10 price = r1(:,1); L_typ = r1(:,2); PV_typ = r1(:,3);
11 LD = readmatrix(fullfile(thisdir, '..', '附件', '附件2.xlsx'), 'Sheet', '小区负载', 'Range', 'B2:E0366');
12 PVD = readmatrix(fullfile(thisdir, '..', '附件', '附件2.xlsx'), 'Sheet', '光伏发电实际功率', 'Range', 'B2:E0366');
13 raw3 = readcell(fullfile(thisdir, '..', '附件', '附件3.xlsx'), 'Range', 'A2:Z1461');
14 NDAY = 365; T = 144; dt = 1/6;

```

```

15 PVh = reshape(PVD', 6, 24, 365); PVh = squeeze(mean(PVh,1)); % 365×24 整点实际
16 Lh = reshape(LD', 6, 24, 365); Lh = squeeze(mean(Lh,1));
17 FORE = zeros(365, 24, 4); % (天, 预报k小时, 发布: 0/6/12/18)
18 d = 0;
19 for i = 1:size(raw3,1)
20     cc = raw3{i,1};
21     nonempty = ~(isempty(cc) || (ischar(cc) && isempty(strtrim(cc))) || ...
22         (isstring(cc) && ismissing(cc)) || (isnumeric(cc) && isnan(cc)));
23     if nonempty, d = d + 1; end
24     issue = sscanf(char(raw3{i,2}), '%d:%s'); % char() 兼容 string/char 两种读取类型
25     ii = find(issue == [0 6 12 18]);
26     FORE(d, :, ii) = cellfun(@double, raw3{i, 3:26});
27 end
28
29 %% ----- 2. 参数 -----
30 ETA_C = 0.9; ETA_D = 0.9; PMAX = 5000; SMIN = 1200; SMAX = 10800;
31 EMULT = 5; ADJ_PRE = 0.5; % 上/下调相对基础价的边际费率(超额总价1.5c/违约0.5c)
32 GAMMA = 1.02; DELTA = 0.98; LAM_PV = 0.1; LAM_L = 0.3; % 费用网格寻优
33 ADJ_T = [36 72 108]; % 调整起始区间(1基): 6:00/12:00/18:00
34 ISSUE = [6 12 18];
35 feb1 = 32;
36 opts = optimoptions('linprog','Algorithm','dual-simplex','Display','off');
37
38 %% ----- 3. 全年滚动 -----
39 plans_p = zeros(NDAY,T); adj_p = zeros(NDAY,T);
40 Emerg = zeros(NDAY,T); ch_r = zeros(NDAY,T); dis_r = zeros(NDAY,T);
41 Sall = zeros(NDAY,T+1); plan_cost = zeros(NDAY,1);
42 for d = 1:NDAY
43     % --- 0:00 估计 ---
44     h_est = pv_hour_est_issue(d, 0, FORE, PVh, NDAY); % 0:00发布
45     PVh10 = to_10min(d, h_est, PVD, PVh, T);
46     Lh10 = load_est(d, LD, L_typ);
47     Lh_h = load_hour_est(d, Lh, L_typ);
48     if d == 1, S0 = 6000; else, S0 = Sall(d-1, T+1); end
49     [ps, chs, diss] = lp_plan(Lh10*GAMMA, PVh10*DELTA, S0, price, ETA_C, ETA_D, PMAX, SMIN, SMAX, T, opts);
50     plans_p(d,:) = ps'; plan_cost(d) = sum(ps.*price)*dt;
51     p_adj = ps; S = zeros(T+1,1); S(1) = S0;
52     bounds = [1 ADJ_T T+1];
53     for si = 1:4
54         a = bounds(si); b = bounds(si+1) - 1;
55         if si > 1
56             % --- 调整时刻 a: 最新预报标定 + 当日残差更新 ---
57             h_est2 = pv_hour_est_issue(d, ISSUE(si-1), FORE, PVh, NDAY);
58             h_end = floor((a-1)*10/60); % 已实现的时钟小时(1..h_end)
59             res_pv = 0; npv = 0; res_l = 0;
60             for h = 1:h_end
61                 col = h + 1; % PVh/Lh 列 = 时钟小时+1
62                 if h_est(h+1) > 50 || (d >= 8 && PVh(d-7,col) > 50)
63                     res_pv = res_pv + PVh(d,col) - h_est(h+1); npv = npv + 1;
64                 end
65                 res_l = res_l + Lh(d,col) - Lh_h(col);
66             end
67             if npv > 0, h_est2 = h_est2 + LAM_PV * res_pv / npv; end
68             h_est2 = max(h_est2, 0);
69             PVh10 = to_10min(d, h_est2, PVD, PVh, T);
70             Lh10(a:T) = Lh10(a:T) + LAM_L * res_l / max(h_end,1);
71             [p_new, ch_new, dis_new] = lp_adjust(a, Lh10*GAMMA, PVh10*DELTA, S(a), ...
72                 plans_p(d,a:T)', price, ETA_C, ETA_D, PMAX, SMIN, SMAX, T, ADJ_PRE, EMULT, opts);
73             p_adj(a:T) = p_new; chs(a:T) = ch_new; diss(a:T) = dis_new;
74         end
75         [e, cr, dr, S] = simulate_seg(p_adj, chs, diss, LD(d,:)', PVD(d,:)', S, a, b, ETA_C, ETA_D, PMAX, SMIN, SMAX, T);
76         Emerg(d,a:b) = e'; ch_r(d,a:b) = cr'; dis_r(d,a:b) = dr';
77     end
78     adj_p(d,:) = p_adj';

```

```

79     Sall(d,:) = S';
80 end
81
82 %% ----- 4. 年度统计 -----
83 up = max(adj_p - plans_p, 0); dn = max(plans_p - adj_p, 0);
84 sl = febl:NDAY;
85 base = sum(adj_p(sl,:).*price','all')*dt;
86 fee_adj = sum((ADJ_PRE*up(sl,:) + ADJ_PRE*dn(sl,:)).*price','all')*dt;
87 C_e = sum(Emerg(sl,:).*price'.*EMULT,'all')*dt;
88 fprintf('==== 问题3 最终策略 V4FR =====\n');
89 fprintf('计划购电费(0:00口径) %.2f 元\n', sum(plan_cost(sl)));
90 fprintf('按调整量实付 %.2f 元, 违约/溢价费 %.2f 元\n', base, fee_adj);
91 fprintf('紧急购电费 %.2f 元, 紧急购电量 %.1f kWh\n', C_e, sum(Emerg(sl,:), 'all')*dt);
92 fprintf('调整下调电量 %.1f kWh, 上调电量 %.1f kWh\n', sum(dn(sl,:), 'all')*dt, sum(up(sl,:), 'all')*dt);
93 fprintf('总费用 %.2f 元, 紧急天数 %d/334\n', base+fee_adj+C_e, nnz(sum(Emerg(sl,:), 2)>1e-9));
94 fprintf('储电量范围 [%1f, %1f]\n', min(Sall, [], 'all'), max(Sall, [], 'all'));
95
96 %% ----- 5. 表3 指定日期 -----
97 blocks = {[144, 1:23], 24:47, 48:71, 72:95, 96:119, 120:143};
98 bname = {'0:00-4:00', '4:00-8:00', '8:00-12:00', '12:00-16:00', '16:00-20:00', '20:00-24:00'};
99 rowT1 = [60 72 84 96 108 120];
100 nameT1 = {'10:00-10:10', '12:00-12:10', '14:00-14:10', '16:00-16:10', '18:00-18:10', '20:00-20:10'};
101 dvec = datetime(2025,1,1) + days(0:364);
102 target = [datetime(2025,3,20), datetime(2025,6,21), datetime(2025,9,23), datetime(2025,12,21)];
103 for td = target
104     d = find(dvec == td);
105     Ea = adj_p(d,:)*dt; Ep = plans_p(d,:)*dt;
106     fprintf('\n--- %s ---\n', datestr(td, 'yyyy.mm.dd'));
107     fprintf('计划购电量 %.2f -> 调整购电量 %.2f kWh, 计划购电费 %.2f 元\n', sum(Ep), sum(Ea), plan_cost(d));
108     for k = 1:6, fprintf(' %s: %.4f\n', nameT1{k}, Ea(rowT1(k))); end
109     for k = 1:6
110         if k == 1
111             ebch = ch_r(d-1,144)*dt + sum(ch_r(d,1:23))*dt;
112             ebdis = dis_r(d-1,144)*dt + sum(dis_r(d,1:23))*dt;
113         else
114             ebch = sum(ch_r(d,blocks{k}))*dt; ebdis = sum(dis_r(d,blocks{k}))*dt;
115         end
116         fprintf(' %s: 充电 %.4f 放电 %.4f\n', bname{k}, ebch, ebdis);
117     end
118     s000 = 6000; if d > 1, s000 = Sall(d-1,144); end
119     fprintf(' s(0:00)=%.2f, s(24:00)=%.2f\n', s000, Sall(d,144));
120     e = Emerg(d,:); t = 1;
121     if all(e<=1e-9), fprintf(' 紧急购电: 无\n'); end
122     while t <= T
123         if e(t) > 1e-9
124             j = t; while j<T && e(j+1)>1e-9, j = j+1; end
125             fprintf(' 紧急购电 %s-%s: %.4f kWh\n', timetag(10*t), timetag(10*(j+1)), sum(e(t:j))*dt);
126             t = j+1;
127         else, t = t + 1; end
128     end
129 end
130
131 %% ----- 6. 写 result3.xlsx -----
132 fout = fullfile(thisdir, 'result3.xlsx');
133 if exist(fout, 'file'), delete(fout); end
134 hdr = [{ '日期\时间' }, arrayfun(@(i) [timetag(10*i), '-', timetag(10*(i+1))], 1:143, 'UniformOutput', false), ...
135     {'0:00-0:10+1', '全天购电量', '全天购电费'}];
136 hdr{43} = '7:0-7:10'; % 保留官方模板 typo
137 for sheet = 1:2
138     if sheet == 1, PP = plans_p; else, PP = adj_p; end
139     C1 = cell(335, 147); C1(1,:) = hdr;
140     for i = 1:334
141         d = febl - 1 + i;
142         Ep = PP(d,:)*dt;

```

```

143     C1{i+1,1} = dvec(d);
144     for t = 1:T, C1{i+1,1+t} = round(Ep(t),4); end
145     C1{i+1,146} = round(sum(Ep),4);
146     if sheet == 1, C1{i+1,147} = round(plan_cost(d),4);
147     else, C1{i+1,147} = round(sum(PP(d,:).*price')*dt,4); end
148 end
149 if sheet == 1, writecell(C1, fout, 'Sheet', '计划购电量');
150 else, writecell(C1, fout, 'Sheet', '调整购电量'); end
151 end
152 C2 = cell(1+334*6, 6);
153 C2(1,:) = {'日期','时间段','充电量','放电量','时刻','储电量'};
154 for i = 1:334
155     d = febl - 1 + i;
156     s000 = 6000; if d > 1, s000 = Sall(d-1,144); end
157     for k = 1:6
158         r = (i-1)*6 + k + 1;
159         if k == 1, C2{r,1} = dvec(d); end
160         C2{r,2} = bname{k};
161         if k == 1
162             C2{r,3} = round(ch_r(d-1,144)*dt + sum(ch_r(d,1:23))*dt, 4);
163             C2{r,4} = round(dis_r(d-1,144)*dt + sum(dis_r(d,1:23))*dt, 4);
164         else
165             C2{r,3} = round(sum(ch_r(d,blocks{k}))*dt, 4);
166             C2{r,4} = round(sum(dis_r(d,blocks{k}))*dt, 4);
167         end
168         if k == 1, C2{r,5} = '0:00'; C2{r,6} = round(s000,4); end
169         if k == 2, C2{r,5} = '24:00'; C2{r,6} = round(Sall(d,144),4); end
170     end
171 end
172 writecell(C2, fout, 'Sheet', '充放电量');
173 C3 = cell(100000,3); nrow = 1;
174 C3(1,:) = {'日期','购电时间段','购电量'};
175 for i = 1:334
176     d = febl - 1 + i; e = Emerg(d,:); t = 1; first = true;
177     while t <= T
178         if e(t) > 1e-9
179             j = t; while j<T && e(j+1)>1e-9, j = j+1; end
180             nrow = nrow + 1;
181             if first, C3{nrow,1} = dvec(d); first = false; end
182             C3{nrow,2} = [timetag(10*t),'-',timetag(10*(j+1))];
183             C3{nrow,3} = round(sum(e(t:j))*dt, 4);
184             t = j+1;
185         else, t = t + 1; end
186     end
187 end
188 writecell(C3(1:nrow,:), fout, 'Sheet', '紧急购电量');
189 fprintf('\n已写出 %s\n', fout);
190
191 %% ===== 局部函数 =====
192 function s = timetag(m)
193 if m >= 1440, s = sprintf('%d:%02d+1', floor((m-1440)/60), mod(m-1440,60));
194 else, s = sprintf('%d:%02d', floor(m/60), mod(m,60)); end
195 end
196
197 function h_est = pv_hour_est_issue(d, issue, FORE, PVh, NDAY)
198 % 逐时OLS[官方issue时刻报, w1上周同星期] (30天滚动训练, 用0:00报训练)
199 % h_est(j): j=h+1 对应预报小时 h (h=1..24, 时钟 mod(h,24)); PVh 列 = 时钟小时+1
200 ii = find(issue == [0 6 12 18]);
201 h_est = zeros(25,1);
202 for h = 1:24
203     k = h - issue;
204     if k >= 1 && k <= 24, off = FORE(d, k, ii); else, off = 0; end
205     col = mod(h,24) + 1; % 时钟 mod(h,24) 的 PVh 列
206     if d >= 8, w1 = PVh(d-7, col); else, w1 = 0; end

```

```

207     tr = max(8, d-30):d-1;
208     if numel(tr) >= 5
209         X = zeros(numel(tr),2); y = zeros(numel(tr),1);
210         for j = 1:numel(tr)
211             dd = tr(j);
212             X(j,:) = [FORE(dd, h, 1), PVh(dd-7, col)];
213             y(j) = PVh(dd, col);
214         end
215         beta = [ones(numel(tr),1), X] \ y;
216         est = beta(1) + beta(2)*off + beta(3)*w1;
217         if ~isfinite(est), est = 0.5*off + 0.5*w1; end
218     else
219         est = 0.5*off + 0.5*w1;
220     end
221     h_est(h+1) = max(est, 0);
222 end
223 end
224
225 function out = to_10min(d, h_est, PVD, PVh, T)
226 % 整点估计 → 144区间功率; 区间t的预报小时 FH=floor(t/6)(0..24), PVh列=mod(FH,24)+1
227 out = zeros(T,1);
228 for t = 1:T
229     FH = floor(t/6);
230     col = mod(FH,24) + 1;
231     if d >= 8 && PVh(d-7,col) > 1e-6, shape = PVD(d-7,t)/PVh(d-7,col); else, shape = 1; end
232     out(t) = h_est(FH+1) * shape;
233 end
234 end
235
236 function L = load_est(d, LD, L_typ)
237 if d == 1, L = L_typ; elseif d >= 8, L = LD(d-7,:); else, L = LD(d-1,:); end
238 end
239
240 function Lh_h = load_hour_est(d, Lh, L_typ)
241 if d == 1, Lh_h = mean(reshape(L_typ,6,24),1); elseif d >= 8, Lh_h = Lh(d-7,:); else, Lh_h = Lh(d-1,:); end
242 end
243
244 function [p, ch, dis] = lp_plan(Lh10, Ph10, S0, price, ETA_C, ETA_D, PMAX, SMIN, SMAX, T, opts)
245 n = T; o = 4*n; m = 5*n+1; dt = 1/6;
246 ip = 1:n; ich = n+1:2*n; idi = 2*n+1:3*n; iq = 3*n+1:4*n; is = o+1:m;
247 Ii = [1:n, 1:n, 1:n, 1:n, n+(1:n), n+(1:n), n+(1:n), n+(1:n), 2*n+1, 2*n+2];
248 Jj = [ip, idi, ich, iq, is(2:n+1), is(1:n), ich, idi, is(1), is(n+1)];
249 Vv = [ones(1,n), ones(1,n), -ones(1,n), -ones(1,n), ones(1,n), -ones(1,n), ...
250       -ETA_C*dt*ones(1,n), (dt/ETA_D)*ones(1,n), 1, 1];
251 Aeq = sparse(Ii, Jj, Vv, 2*n+2, m);
252 beq = [Lh10 - Ph10; zeros(n,1); S0; S0];
253 f = zeros(m,1); f(ip) = price*dt;
254 lb = zeros(m,1); ub = inf(m,1);
255 ub(ich) = PMAX; ub(idi) = PMAX; lb(is) = SMIN; ub(is) = SMAX;
256 [x, ~, exitflag] = linprog(f, [], [], Aeq, beq, lb, ub, opts);
257 assert(exitflag==1, '计划LP不可行');
258 p = x(ip); ch = x(ich); dis = x(idi);
259 end
260
261 function [p, ch, dis] = lp_adjust(a, Lh10, Ph10, S_start, p_plan_seg, price, ...
262     ETA_C, ETA_D, PMAX, SMIN, SMAX, T, ADJ_PRE, EMULT, opts)
263 % 双向调整, 末端自由, 带5c安全阀; u±边际费率均为 ADJ_PRE=0.5c
264 n = T - a + 1; o = 4*n; m = 5*n+1; M = m + 3*n; dt = 1/6;
265 iu1 = m+1:m+n; iu2 = m+n+1:m+2*n; ie = m+2*n+1:M; % u+ / u- / e 三块索引
266 ip = 1:n; ich = n+1:2*n; idi = 2*n+1:3*n; iq = 3*n+1:4*n; is = o+1:m;
267 nrow = 3*n + 1;
268 Ii = [1:n, 1:n, 1:n, 1:n, 1:n, n+(1:n), n+(1:n), n+(1:n), n+(1:n), 2*n+1];
269 Jj = [ip, idi, ie, ich, iq, is(2:n+1), is(1:n), ich, idi, is(1)];
270 Vv = [ones(1,n), ones(1,n), ones(1,n), -ones(1,n), -ones(1,n), ones(1,n), -ones(1,n), ...

```

```

271     -ETA_C*dt*ones(1,n), (dt/ETA_D)*ones(1,n), 1];
272 rows_u = 2*n+1 + (1:n);
273 Ii = [Ii, rows_u, rows_u, rows_u];
274 Jj = [Jj, ip, iu1, iu2];
275 Vv = [Vv, ones(1,n), -ones(1,n), ones(1,n)];
276 Aeq = sparse(Ii, Jj, Vv, nrow, M);
277 beq = [Lh10(a:T) - Ph10(a:T); zeros(n,1); S_start; p_plan_seg];
278 f = zeros(M,1);
279 f(ip) = price(a:T)*dt; f(iu1) = ADJ_PRE*price(a:T)*dt; f(iu2) = ADJ_PRE*price(a:T)*dt; f(ie) = EMULT*price(a:T)*dt;
280 lb = zeros(M,1); ub = inf(M,1);
281 ub(ich) = PMAX; ub(idi) = PMAX; lb(is) = SMIN; ub(is) = SMAX;
282 Aub = sparse(1, M); Aub(1, is(n+1)) = -1; bub = -4000; % 末端最低备用 S(24:00)>=4000
283 [x, ~, exitflag] = linprog(f, Aub, bub, Aeq, beq, lb, ub, opts);
284 assert(exitflag==1, '调整LP不可行 a=%d', a);
285 p = x(ip); ch = x(ich); dis = x(idi);
286 end
287
288 function [e, cr, dr, S] = simulate_seg(ps, chs, diss, L, PV, S, a, b, ETA_C, ETA_D, PMAX, SMIN, SMAX, T)
289 dt = 1/6;
290 e = zeros(b-a+1,1); cr = zeros(b-a+1,1); dr = zeros(b-a+1,1);
291 for t = a:b
292     dis_max = min(PMAX, max(S(t)-SMIN,0)*ETA_D/dt);
293     ch_max = min(PMAX, max(SMAX-S(t),0)/ETA_C/dt);
294     dis = min(diss(t), dis_max); ch = min(chs(t), ch_max);
295     resid = (L(t) - ps(t) - PV(t)) - (dis - ch);
296     if resid > 0
297         r = min(ch, resid); ch = ch - r; resid = resid - r;
298         r = min(dis_max - dis, resid); dis = dis + r; resid = resid - r;
299         e(t-a+1) = max(resid, 0);
300     else
301         sur = -resid;
302         r = min(dis, sur); dis = dis - r; sur = sur - r;
303         r = min(ch_max - ch, sur); ch = ch + r; sur = sur - r;
304     end
305     cr(t-a+1) = ch; dr(t-a+1) = dis;
306     S(t+1) = S(t) + ETA_C*ch*dt - dis*dt/ETA_D;
307 end
308 end

```

3.1.4 问题四：problem4.m（波动电价下的问题二、三策略重算）

```

1 %% CUMCM2026 C题 问题4 —— 波动电价下重算问题2/问题3
2 % Q4-2 = 问题2策略(w1预测, =1.05, =0.98) × 附件4逐日电价 -> result4-2.xlsx
3 % Q4-3 = 问题3策略(V4FR: 备用>=4000, =1.02, =1.00) × 附件4逐日电价 -> result4-3.xlsx
4 % 参考解(Python/HiGHS): Q4-2 总费用 15,263,635.75 元; Q4-3 总费用 14,393,752 元级(V4FR+备用, 精确见summary4.json)
5 % (恒定电价对照: Q2 14,546,503.97; Q3 13,825,637.87)
6 clear; clc;
7 thisdir = fileparts(mfilename('fullpath'));
8
9 %% ----- 1. 读取数据 -----
10 r1 = readmatrix(fullfile(thisdir, '..', '附件', '附件1.xlsx'), 'Range', 'B2:D145');
11 price1 = r1(:,1); L_typ = r1(:,2); PV_typ = r1(:,3);
12 LD = readmatrix(fullfile(thisdir, '..', '附件', '附件2.xlsx'), 'Sheet', '小区负载', 'Range', 'B2:E0366');
13 PVD = readmatrix(fullfile(thisdir, '..', '附件', '附件2.xlsx'), 'Sheet', '光伏发电实际功率', 'Range', 'B2:E0366');
14 raw3 = readcell(fullfile(thisdir, '..', '附件', '附件3.xlsx'), 'Range', 'A2:Z1461');
15 PM = readmatrix(fullfile(thisdir, '..', '附件', '附件4.xlsx'), 'Range', 'B2:E0366'); % 365×144 波动电价
16 NDAY = 365; T = 144; dt = 1/6;
17 assert(size(PM,1)==365 && size(PM,2)==144, '附件4尺寸异常');
18 PVh = reshape(PVD, 6, 24, 365); PVh = squeeze(mean(PVh,1));
19 Lh = reshape(LD, 6, 24, 365); Lh = squeeze(mean(Lh,1));
20 FORE = zeros(365, 24, 4);

```

```

21 d = 0;
22 for i = 1:size(raw3,1)
23     cc = raw3{i,1};
24     nonempty = ~(isempty(cc) || (ischar(cc) && isempty(strtrim(cc))) || ...
25         (isstring(cc) && ismissing(cc)) || (isnumeric(cc) && isnan(cc)));
26     if nonempty, d = d + 1; end
27     issue = sscanf(char(raw3{i,2}), '%d:%s');
28     ii = find(issue == [0 6 12 18]);
29     FORE(d, :, ii) = cellfun(@double, raw3(i, 3:26));
30 end
31 fprintf('附件4电价: 均值 %.4f, 范围 [%.4f, %.4f], 日内价差均值 %.4f\n', ...
32     mean(PM, 'all'), min(PM, [], 'all'), max(PM, [], 'all'), mean(max(PM, [], 2) - min(PM, [], 2)));
33
34 %% ----- 2. 共享参数 -----
35 ETA_C = 0.9; ETA_D = 0.9; PMAX = 5000; SMIN = 1200; SMAX = 10800;
36 EMULT = 5; ADJ_PRE = 0.5;
37 LAM_PV = 0.1; LAM_L = 0.3; % 费用网格寻优
38 ADJ_T = [36 72 108]; ISSUE = [6 12 18];
39 feb1 = 32;
40 opts = optimoptions('linprog','Algorithm','dual-simplex','Display','off');
41 blocks = {[144, 1:23], 24:47, 48:71, 72:95, 96:119, 120:143};
42 bname = {'0:00-4:00','4:00-8:00','8:00-12:00','12:00-16:00','16:00-20:00','20:00-24:00'};
43 dvec = datetime(2025,1,1) + days(0:364)';
44
45 %% ===== 第一部分: Q4-2 =====
46 GAMMA2 = 1.05; DELTA2 = 0.98;
47 plans_p = zeros(NDAY,T); Emerg = zeros(NDAY,T);
48 ch_r = zeros(NDAY,T); dis_r = zeros(NDAY,T);
49 Sall = zeros(NDAY,T+1); plan_cost = zeros(NDAY,1);
50 SO = 6000;
51 for d = 1:NDAY
52     if d == 1, Lhat = L_typ; PVhat = PV_typ;
53     elseif d >= 8, Lhat = LD(d-7,:); PVhat = PVD(d-7,:);
54     else, Lhat = LD(d-1,:); PVhat = PVD(d-1,:);
55     end
56     [ps, chs, diss] = lp_plan(Lhat*GAMMA2, PVhat*DELTA2, SO, PM(d,:), ...
57         ETA_C, ETA_D, PMAX, SMIN, SMAX, T, opts);
58     plans_p(d,:) = ps'; plan_cost(d) = sum(ps.*PM(d,:))*dt;
59     [e, cr, dr, S] = simulate_seg(ps, chs, diss, LD(d,:), PVD(d,:), ...
60         [SO; zeros(T,1)], 1, T, ETA_C, ETA_D, PMAX, SMIN, SMAX, T);
61     Emerg(d,:) = e'; ch_r(d,:) = cr'; dis_r(d,:) = dr'; Sall(d,:) = S';
62     SO = S(T+1);
63 end
64 sl = feb1:NDAY;
65 E_p2 = sum(plans_p(sl,:), 'all')*dt; C_p2 = sum(plan_cost(sl));
66 E_e2 = sum(Emerg(sl,:), 'all')*dt; C_e2 = sum(Emerg(sl,:).*PM(sl,:).*EMULT, 'all')*dt;
67 fprintf('\n===== Q4-2 (问题2策略 × 波动电价) =====\n');
68 fprintf('计划购电量 %.1f kWh, 计划购电费 %.2f 元\n', E_p2, C_p2);
69 fprintf('紧急购电量 %.1f kWh, 紧急购电费 %.2f 元, 天数 %d/334\n', E_e2, C_e2, nnz(sum(Emerg(sl,:), 2) > 1e-9));
70 fprintf('总费用 %.2f 元 (恒定电价Q2: 14546503.97, 差 %+.2f)\n', C_p2+C_e2, C_p2+C_e2-14546503.97);
71 R2 = struct('plans_p', plans_p, 'Emerg', Emerg, 'ch_r', ch_r, 'dis_r', dis_r, 'Sall', Sall, 'plan_cost', plan_cost);
72
73 %% ===== 第二部分: Q4-3 =====
74 GAMMA3 = 1.02; DELTA3 = 1.00; % 波动电价下重寻优: =1.00
75 plans_p = zeros(NDAY,T); adj_p = zeros(NDAY,T);
76 Emerg = zeros(NDAY,T); ch_r = zeros(NDAY,T); dis_r = zeros(NDAY,T);
77 Sall = zeros(NDAY,T+1); plan_cost = zeros(NDAY,1);
78 for d = 1:NDAY
79     h_est = pv_hour_est_issue(d, 0, FORE, PVh);
80     PVh10 = to_10min(d, h_est, PVD, PVh, T);
81     Lh10 = load_est(d, LD, L_typ); Lh_h = load_hour_est(d, Lh, L_typ);
82     if d == 1, SO = 6000; else, SO = Sall(d-1, T+1); end
83     [ps, chs, diss] = lp_plan(Lh10*GAMMA3, PVh10*DELTA3, SO, PM(d,:), ...
84         ETA_C, ETA_D, PMAX, SMIN, SMAX, T, opts);

```

```

85 plans_p(d,:) = ps'; plan_cost(d) = sum(ps.*PM(d,:))*dt;
86 p_adj = ps; S = zeros(T+1,1); S(1) = S0;
87 bounds = [1 ADJ_T T+1];
88 for si = 1:4
89     a = bounds(si); b = bounds(si+1) - 1;
90     if si > 1
91         h_est2 = pv_hour_est_issue(d, ISSUE(si-1), FORE, PVh);
92         h_end = floor((a-1)*10/60);
93         res_pv = 0; npv = 0; res_l = 0;
94         for h = 1:h_end
95             col = h + 1;
96             if h_est(h+1) > 50 || (d >= 8 && PVh(d-7,col) > 50)
97                 res_pv = res_pv + PVh(d,col) - h_est(h+1); npv = npv + 1;
98             end
99             res_l = res_l + Lh(d,col) - Lh_h(col);
100         end
101         if npv > 0, h_est2 = h_est2 + LAM_PV * res_pv / npv; end
102         h_est2 = max(h_est2, 0);
103         PVh10 = to_10min(d, h_est2, PVD, PVh, T);
104         Lh10(a:T) = Lh10(a:T) + LAM_L * res_l / max(h_end,1);
105         [p_new, ch_new, dis_new] = lp_adjust(a, Lh10*GAMMA3, PVh10*DELTA3, S(a), ...
106             plans_p(d,a:T)', PM(d,:)', ETA_C, ETA_D, PMAX, SMIN, SMAX, T, ADJ_PRE, EMULT, opts);
107         p_adj(a:T) = p_new; chs(a:T) = ch_new; diss(a:T) = dis_new;
108     end
109     [e, cr, dr, S] = simulate_seg(p_adj, chs, diss, LD(d,:)', PVD(d,:)', S, a, b, ETA_C, ETA_D, PMAX, SMIN, SMAX, T);
110     Emerg(d,a:b) = e'; ch_r(d,a:b) = cr'; dis_r(d,a:b) = dr';
111 end
112 adj_p(d,:) = p_adj';
113 Sall(d,:) = S';
114 end
115 up = max(adj_p - plans_p, 0); dn = max(plans_p - adj_p, 0);
116 base3 = sum(adj_p(s1,:).*PM(s1,:),'all')*dt;
117 fee3 = sum((ADJ_PRE*up(s1,:) + ADJ_PRE*dn(s1,:)).*PM(s1,:),'all')*dt;
118 C_e3 = sum(Emerg(s1,:).*PM(s1,:).*EMULT,'all')*dt;
119 fprintf('\n===== Q4-3 (问题3策略 × 波动电价) =====\n');
120 fprintf('按调整量实付 %.2f 元, 违约/溢价费 %.2f 元\n', base3, fee3);
121 fprintf('紧急购电费 %.2f 元, 紧急购电量 %.1f kWh, 天数 %d/334\n', C_e3, sum(Emerg(s1,:),'all')*dt, nnz(sum(Emerg(s1,:),2)>1e-9));
122 fprintf('调整: 上调 %.1f kWh, 下调 %.1f kWh\n', sum(up(s1,:),'all')*dt, sum(dn(s1,:),'all')*dt);
123 fprintf('总费用 %.2f 元 (恒定电价Q3: 13825637.87, 差 %.2f)\n', base3+fee3+C_e3, base3+fee3+C_e3-13825637.87);
124 R3 = struct('plans_p',plans_p,'adj_p',adj_p,'Emerg',Emerg,'ch_r',ch_r,'dis_r',dis_r,'Sall',Sall,'plan_cost',plan_cost);
125
126 %% ----- 表3 指定日期 -----
127 rowT1 = [60 72 84 96 108 120];
128 nameT1 = {'10:00-10:10','12:00-12:10','14:00-14:10','16:00-16:10','18:00-18:10','20:00-20:10'};
129 target = [datetime(2025,3,20), datetime(2025,6,21), datetime(2025,9,23), datetime(2025,12,21)];
130 for td = target
131     d = find(dvec == td);
132     fprintf('\n--- %s ---\n', datestr(td,'yyyy.mm.dd'));
133     Ea2 = R2.plans_p(d,:)*dt;
134     fprintf('Q4-2: 计划购电量 %.2f kWh, 购电费 %.2f 元\n', sum(Ea2), R2.plan_cost(d));
135     Ep3 = R3.plans_p(d,:)*dt; Ea3 = R3.adj_p(d,:)*dt;
136     settle3 = sum(Ea3.*PM(d,:)) + 0.5*sum(abs(Ea3-Ep3).*PM(d,:)); % Ea3/Ep3已是kWh, 不再乘dt
137     fprintf('Q4-3: 计划 %.2f -> 调整 %.2f kWh, 计划费 %.2f -> 结算 %.2f 元\n', ...
138         sum(Ep3), sum(Ea3), R3.plan_cost(d), settle3);
139     for k = 1:6, fprintf(' 调整后 %s: %.4f\n', nameT1{k}, Ea3(rowT1(k))); end
140     s000 = 6000; if d > 1, s000 = R3.Sall(d-1,144); end
141     fprintf(' s(0:00)=%.2f, s(24:00)=%.2f\n', s000, R3.Sall(d,144));
142 end
143
144 %% ----- 写 result4-2.xlsx / result4-3.xlsx -----
145 write_result4(fullfile(thisdir,'result4-2.xlsx'), R2, false, PM, dvec, feb1, blocks, bname, T, dt);
146 write_result4(fullfile(thisdir,'result4-3.xlsx'), R3, true, PM, dvec, feb1, blocks, bname, T, dt);
147 fprintf('\n已写出 result4-2.xlsx, result4-3.xlsx\n');
148

```

```

149 %% ===== 局部函数 =====
150 function write_result4(fout, R, has_adj, PM, dvec, febl, blocks, bname, T, dt)
151 if exist(fout, 'file'), delete(fout); end
152 hdr = [{'日期\时间'}, arrayfun(@(i) [timetag(10*i), '-', timetag(10*(i+1))], 1:143, 'UniformOutput', false), ...
153        {'0:00-0:10+1', '全天购电量', '全天购电费'}];
154 hdr{43} = '7:0-7:10';
155 C1 = cell(335, 147); C1(1,:) = hdr;
156 for i = 1:334
157     d = febl - 1 + i;
158     Ep = R.plans_p(d,:)*dt;
159     C1{i+1,1} = dvec(d);
160     for t = 1:T, C1{i+1,1+t} = round(Ep(t),4); end
161     C1{i+1,146} = round(sum(Ep),4);
162     C1{i+1,147} = round(R.plan_cost(d),4);
163 end
164 writecell(C1, fout, 'Sheet', '计划购电量');
165 if has_adj
166     C1b = cell(335, 147); C1b(1,:) = hdr;
167     for i = 1:334
168         d = febl - 1 + i;
169         Ea = R.adj_p(d,:)*dt;
170         C1b{i+1,1} = dvec(d);
171         for t = 1:T, C1b{i+1,1+t} = round(Ea(t),4); end
172         C1b{i+1,146} = round(sum(Ea),4);
173         C1b{i+1,147} = round(sum(R.adj_p(d,:).*PM(d,:)*dt,4);
174     end
175     writecell(C1b, fout, 'Sheet', '调整购电量');
176 end
177 C2 = cell(1+334*6, 6);
178 C2(1,:) = {'日期', '时间段', '充电量', '放电量', '时刻', '储电量'};
179 for i = 1:334
180     d = febl - 1 + i;
181     s000 = 6000; if d > 1, s000 = R.Sall(d-1,144); end
182     for k = 1:6
183         r = (i-1)*6 + k + 1;
184         if k == 1, C2{r,1} = dvec(d); end
185         C2{r,2} = bname{k};
186         if k == 1
187             C2{r,3} = round(R.ch_r(d-1,144)*dt + sum(R.ch_r(d,1:23))*dt, 4);
188             C2{r,4} = round(R.dis_r(d-1,144)*dt + sum(R.dis_r(d,1:23))*dt, 4);
189         else
190             C2{r,3} = round(sum(R.ch_r(d,blocks{k}))*dt, 4);
191             C2{r,4} = round(sum(R.dis_r(d,blocks{k}))*dt, 4);
192         end
193         if k == 1, C2{r,5} = '0:00'; C2{r,6} = round(s000,4); end
194         if k == 2, C2{r,5} = '24:00'; C2{r,6} = round(R.Sall(d,144),4); end
195     end
196 end
197 writecell(C2, fout, 'Sheet', '充放电量');
198 C3 = cell(100000,3); nrow = 1;
199 C3(1,:) = {'日期', '购电时间段', '购电量'};
200 for i = 1:334
201     d = febl - 1 + i; e = R.Emerg(d,:); t = 1; first = true;
202     while t <= T
203         if e(t) > 1e-9
204             j = t; while j<T && e(j+1)>1e-9, j = j+1; end
205             nrow = nrow + 1;
206             if first, C3{nrow,1} = dvec(d); first = false; end
207             C3{nrow,2} = [timetag(10*t), '-', timetag(10*(j+1))];
208             C3{nrow,3} = round(sum(e(t:j))*dt, 4);
209             t = j+1;
210         else, t = t + 1; end
211     end
212 end

```

```

213 writecell(C3(1:nrow,:), fout, 'Sheet', '紧急购电量');
214 end
215
216 function s = timetag(m)
217 if m >= 1440, s = sprintf('%d:%02d+1', floor((m-1440)/60), mod(m-1440,60));
218 else, s = sprintf('%d:%02d', floor(m/60), mod(m,60)); end
219 end
220
221 function h_est = pv_hour_est_issue(d, issue, FORE, PVh)
222 ii = find(issue == [0 6 12 18]);
223 h_est = zeros(25,1);
224 for h = 1:24
225     k = h - issue;
226     if k >= 1 && k <= 24, off = FORE(d, k, ii); else, off = 0; end
227     col = mod(h,24) + 1;
228     if d >= 8, w1 = PVh(d-7, col); else, w1 = 0; end
229     tr = max(8, d-30):d-1;
230     if numel(tr) >= 5
231         X = zeros(numel(tr),2); y = zeros(numel(tr),1);
232         for j = 1:numel(tr)
233             dd = tr(j);
234             X(j,:) = [FORE(dd, h, 1), PVh(dd-7, col)];
235             y(j) = PVh(dd, col);
236         end
237         beta = [ones(numel(tr),1), X] \ y;
238         est = beta(1) + beta(2)*off + beta(3)*w1;
239         if ~isfinite(est), est = 0.5*off + 0.5*w1; end
240     else
241         est = 0.5*off + 0.5*w1;
242     end
243     h_est(h+1) = max(est, 0);
244 end
245 end
246
247 function out = to_10min(d, h_est, PVD, PVh, T)
248 out = zeros(T,1);
249 for t = 1:T
250     FH = floor(t/6);
251     col = mod(FH,24) + 1;
252     if d >= 8 && PVh(d-7,col) > 1e-6, shape = PVD(d-7,t)/PVh(d-7,col); else, shape = 1; end
253     out(t) = h_est(FH+1) * shape;
254 end
255 end
256
257 function L = load_est(d, LD, L_typ)
258 if d == 1, L = L_typ; elseif d >= 8, L = LD(d-7,:); else, L = LD(d-1,:); end
259 end
260
261 function Lh_h = load_hour_est(d, Lh, L_typ)
262 if d == 1, Lh_h = mean(reshape(L_typ,6,24),1); elseif d >= 8, Lh_h = Lh(d-7,:); else, Lh_h = Lh(d-1,:); end
263 end
264
265 function [p, ch, dis] = lp_plan(Lh10, Ph10, S0, price_vec, ETA_C, ETA_D, PMAx, SMIN, SMAX, T, opts)
266 n = T; o = 4*n; m = 5*n+1; dt = 1/6;
267 ip = 1:n; ich = n+1:2*n; idi = 2*n+1:3*n; iq = 3*n+1:4*n; is = o+1:m;
268 Ii = [1:n, 1:n, 1:n, 1:n, n+(1:n), n+(1:n), n+(1:n), n+(1:n), 2*n+1, 2*n+2];
269 Jj = [ip, idi, ich, iq, is(2:n+1), is(1:n), ich, idi, is(1), is(n+1)];
270 Vv = [ones(1,n), ones(1,n), -ones(1,n), -ones(1,n), ones(1,n), -ones(1,n), ...
271     -ETA_C*dt*ones(1,n), (dt/ETA_D)*ones(1,n), 1, 1];
272 Aeq = sparse(Ii, Jj, Vv, 2*n+2, m);
273 beq = [Lh10 - Ph10; zeros(n,1); S0; S0];
274 f = zeros(m,1); f(ip) = price_vec*dt;
275 lb = zeros(m,1); ub = inf(m,1);
276 ub(ich) = PMAx; ub(idi) = PMAx; lb(is) = SMIN; ub(is) = SMAX;

```

```

277 [x, ~, exitflag] = linprog(f, [], [], Aeq, beq, lb, ub, opts);
278 assert(exitflag==1, '计划LP不可行');
279 p = x(ip); ch = x(ich); dis = x(idi);
280 end
281
282 function [p, ch, dis] = lp_adjust(a, Lh10, Ph10, S_start, p_plan_seg, price_vec, ...
283     ETA_C, ETA_D, PMAX, SMIN, SMAX, T, ADJ_PRE, EMULT, opts)
284 % 双向调整, 末端自由, 带5c安全阀; u±边际费率均为 ADJ_PRE=0.5c
285 n = T - a + 1; o = 4*n; m = 5*n+1; M = m + 3*n; dt = 1/6;
286 iu1 = m+1:m+n; iu2 = m+n+1:m+2*n; ie = m+2*n+1:M;
287 ip = 1:n; ich = n+1:2*n; idi = 2*n+1:3*n; iq = 3*n+1:4*n; is = o+1:m;
288 nrow = 3*n + 1;
289 Ii = [1:n, 1:n, 1:n, 1:n, 1:n, n+(1:n), n+(1:n), n+(1:n), n+(1:n), 2*n+1];
290 Jj = [ip, idi, ie, ich, iq, is(2:n+1), is(1:n), ich, idi, is(1)];
291 Vv = [ones(1,n), ones(1,n), ones(1,n), -ones(1,n), -ones(1,n), ones(1,n), -ones(1,n), ...
292     -ETA_C*dt*ones(1,n), (dt/ETA_D)*ones(1,n), 1];
293 rows_u = 2*n+1 + (1:n);
294 Ii = [Ii, rows_u, rows_u, rows_u];
295 Jj = [Jj, ip, iu1, iu2];
296 Vv = [Vv, ones(1,n), -ones(1,n), ones(1,n)];
297 Aeq = sparse(Ii, Jj, Vv, nrow, M);
298 beq = [Lh10(a:T) - Ph10(a:T); zeros(n,1); S_start; p_plan_seg];
299 f = zeros(M,1);
300 f(ip) = price_vec(a:T)*dt; f(iu1) = ADJ_PRE*price_vec(a:T)*dt; f(iu2) = ADJ_PRE*price_vec(a:T)*dt; f(ie) = EMULT*price_vec(a:T)
    *dt;
301 lb = zeros(M,1); ub = inf(M,1);
302 ub(ich) = PMAX; ub(idi) = PMAX; lb(is) = SMIN; ub(is) = SMAX;
303 Aub = sparse(1, M); Aub(1, is(n+1)) = -1; bub = -4000; % 末端最低备用 S(24:00)>=4000
304 [x, ~, exitflag] = linprog(f, Aub, bub, Aeq, beq, lb, ub, opts);
305 assert(exitflag==1, '调整LP不可行 a=%d', a);
306 p = x(ip); ch = x(ich); dis = x(idi);
307 end
308
309 function [e, cr, dr, S] = simulate_seg(ps, chs, diss, L, PV, S, a, b, ETA_C, ETA_D, PMAX, SMIN, SMAX, T)
310 dt = 1/6;
311 e = zeros(b-a+1,1); cr = zeros(b-a+1,1); dr = zeros(b-a+1,1);
312 for t = a:b
313     dis_max = min(PMAX, max(S(t)-SMIN,0)*ETA_D/dt);
314     ch_max = min(PMAX, max(SMAX-S(t),0)/ETA_C/dt);
315     dis = min(diss(t), dis_max); ch = min(chs(t), ch_max);
316     resid = (L(t) - ps(t) - PV(t)) - (dis - ch);
317     if resid > 0
318         r = min(ch, resid); ch = ch - r; resid = resid - r;
319         r = min(dis_max - dis, resid); dis = dis + r; resid = resid - r;
320         e(t-a+1) = max(resid, 0);
321     else
322         sur = -resid;
323         r = min(dis, sur); dis = dis - r; sur = sur - r;
324         r = min(ch_max - ch, sur); ch = ch + r; sur = sur - r;
325     end
326     cr(t-a+1) = ch; dr(t-a+1) = dis;
327     S(t+1) = S(t) + ETA_C*ch*dt - dis*dt/ETA_D;
328 end
329 end

```

3.2 Python 程序

3.2.1 问题一: solve_q1_two_cases.py (日循环线性规划, 双情形)

```
#!/usr/bin/env python3
```

```

2  # -*- coding: utf-8 -*-
3  """
4  CUMCM 2026 C题 问题1 —— 双情形求解
5  情形A: S(0) 自由优化 (稳态循环, 仅要求 S(0:00)=S(24:00)) -> result1.xlsx
6  情形B: S(0)=S(24:00)=6000 kWh 固定 (附录1初始值解读) -> result1_fix6000.xlsx
7  约定: 附件1 第 i 行(时刻戳)对应该时刻起点的 10 min 区间。
8  """
9  import json
10 from pathlib import Path
11 import numpy as np
12 import openpyxl
13 from scipy.optimize import linprog
14
15 BASE = Path(__file__).resolve().parents[1]
16 SRC = BASE / "附件" / "附件1.xlsx"
17
18 wb = openpyxl.load_workbook(SRC, data_only=True)
19 rows = [r for r in wb["Sheet1"].iter_rows(min_row=2, values_only=True)]
20 assert len(rows) == 144
21 c = np.array([float(r[1]) for r in rows])
22 L = np.array([float(r[2]) for r in rows])
23 pv = np.array([float(r[3]) for r in rows])
24
25 T, dt = 144, 1.0 / 6.0
26 ETA_C = ETA_D = 0.9
27 PMAX = 5000.0
28 SMIN, SMAX = 1200.0, 10800.0
29 ip, ich, idi, iq, is_ = 0, T, 2 * T, 3 * T, 4 * T
30 nx = 5 * T + 1
31
32 def solve(s0_fixed=None):
33     f = np.zeros(nx); f[ip:ip + T] = c * dt
34     Aeq, beq = [], []
35     for t in range(T):
36         r = np.zeros(nx)
37         r[ip + t] = 1; r[idi + t] = 1; r[ich + t] = -1; r[iq + t] = -1
38         Aeq.append(r); beq.append(L[t] - pv[t])
39     for t in range(T):
40         r = np.zeros(nx)
41         r[is_ + t + 1] = 1; r[is_ + t] = -1
42         r[ich + t] = -ETA_C * dt; r[idi + t] = dt / ETA_D
43         Aeq.append(r); beq.append(0.0)
44     r = np.zeros(nx); r[is_] = 1; r[is_ + T] = -1 # S(144)=S(0)
45     Aeq.append(r); beq.append(0.0)
46     if s0_fixed is not None: # S(0)=s0_fixed
47         r = np.zeros(nx); r[is_] = 1
48         Aeq.append(r); beq.append(s0_fixed)
49     bnd = ([ (0, None) ] * T + [ (0, PMAX) ] * 2 * T +
50            [ (0, None) ] * T + [ (SMIN, SMAX) ] * (T + 1))
51     res = linprog(f, A_eq=np.array(Aeq), b_eq=np.array(beq),
52                  bounds=bnd, method="highs")
53     assert res.status == 0, res.message
54     return res
55
56 def postprocess(res):
57     x = res.x
58     p, ch, dis = x[ip:ip+T], x[ich:ich+T], x[idi:idi+T]
59     q, s = x[iq:iq+T], x[is_:is_+T+1]
60     Ep, Ech, Edis, Eq = p*dt, ch*dt, dis*dt, q*dt
61     for a in (Ep, Ech, Edis, Eq):
62         a[np.abs(a) < 1e-6] = 0.0
63     bal = np.abs(p + pv + dis - L - ch - q).max()
64     assert bal < 1e-6 and s.min() >= SMIN-1e-6 and s.max() <= SMAX+1e-6
65     return Ep, Ech, Edis, Eq, s

```

```

66
67 idx_t1 = [59, 71, 83, 95, 107, 119]
68 names_t1 = ["10:00-10:10", "12:00-12:10", "14:00-14:10",
69             "16:00-16:10", "18:00-18:10", "20:00-20:10"]
70 blocks = [[143] + list(range(0, 23))] + [list(range(a, b))
71         for a, b in [(23, 47), (47, 71), (71, 95), (95, 119), (119, 143)]]
72 bnames = ["0:00-4:00", "4:00-8:00", "8:00-12:00",
73           "12:00-16:00", "16:00-20:00", "20:00-24:00"]
74
75 def tag(m):
76     plus = "+1" if m >= 1440 else ""
77     m %= 1440
78     return f"{m//60}:{m%60:02d}{plus}"
79
80 def write_xlsx(path, Ep, Ech, Edis, s):
81     out = openpyxl.Workbook()
82     ws1 = out.active; ws1.title = "计划购电量"
83     ws1.append(["时间段", "购电量"])
84     for i in range(144):
85         ws1.append([f"{tag(10*(i+1))}-{tag(10*(i+2))}", round(float(Ep[i]), 4)])
86     ws2 = out.create_sheet("充放电电量")
87     ws2.append(["时间段", "充电量", "放电量", "时刻", "储电量"])
88     for j, (n, b) in enumerate(zip(bnames, blocks)):
89         r = [n, round(float(Ech[b].sum()), 4), round(float(Edis[b].sum()), 4)]
90         if j == 0: r += ["0:00", round(float(s[143]), 4)]
91         elif j == 1: r += ["24:00", round(float(s[143]), 4)]
92         else: r += [None, None]
93         ws2.append(r)
94     out.save(path)
95
96 def report(name, res, Ep, Ech, Edis, Eq, s):
97     print(f"\n{' '*56}\n {name}\n{' '*56}")
98     print(f"全天购电费: {res.fun:.4f} 元  全天购电量: {Ep.sum():.4f} kWh")
99     print(f"0:00/24:00 储电量: {s[143]:.4f} kWh  弃光: {Eq.sum():.4f} kWh")
100    for n, i in zip(names_t1, idx_t1):
101        print(f" {n}: {Ep[i]:.4f}")
102    for n, b in zip(bnames, blocks):
103        print(f" {n}: 充电 {Ech[b].sum():9.4f} 放电 {Edis[b].sum():9.4f}")
104    # 关键时刻轨迹 (s[k]=第k段结束储电量, 对应时钟 10(k+1) 分钟)
105    anchors = {"0:00": 143, "6:00": 35, "8:00": 47, "10:00": 59,
106              "14:30": 86, "18:00": 107, "20:50": 124, "22:00": 131}
107    tr = " 轨迹: " + " ".join(f"{k}={s[v]:.0f}" for k, v in anchors.items())
108    print(tr)
109    return {"cost": res.fun, "purchase": float(Ep.sum()), "s000": float(s[143]),
110           "curtail": float(Eq.sum()),
111           "t1": {n: float(Ep[i]) for n, i in zip(names_t1, idx_t1)},
112           "t2": {n: [float(Ech[b].sum()), float(Edis[b].sum())]
113                  for n, b in zip(bnames, blocks)}}
114
115 # ---- 情形 A: S(0) 自由 ----
116 resA = solve()
117 EpA, EchA, EdisA, EqA, sA = postprocess(resA)
118 sumA = report("情形 A: S(0) 自由优化", resA, EpA, EchA, EdisA, EqA, sA)
119 write_xlsx(BASE / "Q1" / "result1.xlsx", EpA, EchA, EdisA, sA)
120
121 # ---- 情形 B: S(0)=6000 固定 ----
122 resB = solve(s0_fixed=6000.0)
123 EpB, EchB, EdisB, EqB, sB = postprocess(resB)
124 sumB = report("情形 B: S(0)=S(24:00)=6000 kWh 固定", resB, EpB, EchB, EdisB, EqB, sB)
125 write_xlsx(BASE / "Q1" / "result1_fix6000.xlsx", EpB, EchB, EdisB, sB)
126
127 print(f"\n费用差: {resB.fun - resA.fun:.4f} 元 "
128       f"({(resB.fun - resA.fun)/resA.fun*100:.4f}%)")
129 json.dump({"A_free": sumA, "B_fix6000": sumB},

```

```

130         open(BASE / "Q1" / "summary1_two_cases.json", "w"),
131         ensure_ascii=False, indent=1)
132 print("已写出 result1.xlsx (情形A) 与 result1_fix6000.xlsx (情形B)")

```

3.2.2 问题二：solve_q2.py (w1 预测 + 余量 + 日前 LP + 日内纠偏，全年滚动)

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  CUMCM 2026 C题 问题2 —— 最终策略: w1预测(上周同星期) + 余量(=1.05, =0.98)
5  流程: 日前(0:00) w1预测→日LP计划(S(144)=S(0)=S0)→日内实际模拟(储能限额内调节,
6  缺口经 减充电→加放电→紧急购电5倍; 富余经 减放电→加充电→弃光)。
7  约定: 每行时刻戳=区间起点, 第d行覆盖 0:10(d)+0:10(d+1); 储电量取实际轨迹。
8  输出: result2.xlsx (2025.2.1-12.31), summary2.json, q2_data.npz
9  """
10 import json
11 from pathlib import Path
12 import numpy as np
13 import openpyxl
14 from scipy.optimize import linprog
15
16 BASE = Path(__file__).resolve().parents[1]
17
18 c1 = openpyxl.load_workbook(BASE / "附件" / "附件1.xlsx", data_only=True)["Sheet1"]
19 rows1 = list(c1.iter_rows(min_row=2, values_only=True))
20 price = np.array([float(r[1]) for r in rows1])
21 L_typ = np.array([float(r[2]) for r in rows1])
22 PV_typ = np.array([float(r[3]) for r in rows1])
23
24 def load_cols(path, sheet):
25     ws = openpyxl.load_workbook(path, data_only=True)[sheet]
26     rows = list(ws.iter_rows(min_row=2, values_only=True))
27     return [r[0] for r in rows], np.array([float(v) for v in r[1:145]] for r in rows)
28
29 dates, LD = load_cols(BASE / "附件" / "附件2.xlsx", "小区负载")
30 _, PVD = load_cols(BASE / "附件" / "附件2.xlsx", "光伏发电实际功率")
31 NDAY = 365
32 T, dt = 144, 1.0 / 6.0
33 ETA_C = ETA_D = 0.9
34 PMAX, SMIN, SMAX = 5000.0, 1200.0, 10800.0
35 EMULT = 5.0
36 ip, ich, idi, iq, is_ = 0, T, 2 * T, 3 * T, 4 * T
37 nx = 5 * T + 1
38 feb1 = 31
39
40 GAMMA, DELTA = 1.05, 0.98 # 余量系数(扫描选定)
41
42 def forecast(d):
43     """w1: 上周同星期实际; 首周回退持续性; 1月1日用附件1典型日"""
44     if d == 0: return L_typ, PV_typ
45     if d >= 7: return LD[d-7], PVD[d-7]
46     return LD[d-1], PVD[d-1]
47
48 def plan_lp(Lhat, PVhat, S0, price_vec=None):
49     pr = price if price_vec is None else price_vec
50     f = np.zeros(nx); f[ip:ip+T] = pr * dt
51     Aeq, beq = [], []
52     Lh, Ph = Lhat * GAMMA, PVhat * DELTA
53     for t in range(T):
54         r = np.zeros(nx)
55         r[ip+t] = 1; r[idi+t] = 1; r[ich+t] = -1; r[iq+t] = -1

```

```

56     Aeq.append(r); beq.append(Lh[t] - Ph[t])
57     for t in range(T):
58         r = np.zeros(nx)
59         r[is_+t+1] = 1; r[is_+t] = -1
60         r[ich+t] = -ETA_C*dt; r[idi+t] = dt/ETA_D
61         Aeq.append(r); beq.append(0.0)
62     for col, val in [(is_, S0), (is_+T, S0)]:
63         r = np.zeros(nx); r[col] = 1
64         Aeq.append(r); beq.append(val)
65     bnd = [(0, None)*T + [(0, PMAX)]*2*T + [(0, None)]*T + [(SMIN, SMAX)]*(T+1))
66     res = linprog(f, A_eq=np.array(Aeq), b_eq=np.array(beq), bounds=bnd, method="highs")
67     assert res.status == 0, f"计划LP失败: {res.message}"
68     x = res.x
69     return x[ip:ip+T], x[ich:ich+T], x[idi:idi+T], res.fun
70
71 def simulate(ps, chs, diss, L, PV, S0):
72     e = np.zeros(T); ch_r = np.zeros(T); dis_r = np.zeros(T)
73     S = np.empty(T+1); S[0] = S0
74     for t in range(T):
75         dis_max = min(PMAX, max(S[t]-SMIN, 0.0)*ETA_D/dt)
76         ch_max = min(PMAX, max(SMAX-S[t], 0.0)/ETA_C/dt)
77         dis = min(diss[t], dis_max); ch = min(chs[t], ch_max)
78         resid = (L[t] - ps[t] - PV[t]) - (dis - ch)
79         if resid > 0:
80             r = min(ch, resid); ch -= r; resid -= r
81             r = min(dis_max - dis, resid); dis += r; resid -= r
82             e[t] = max(resid, 0.0)
83         else:
84             sur = -resid
85             r = min(dis, sur); dis -= r; sur -= r
86             r = min(ch_max - ch, sur); ch += r; sur -= r
87             ch_r[t], dis_r[t] = ch, dis
88             S[t+1] = S[t] + ETA_C*ch*dt - dis*dt/ETA_D
89     return e, ch_r, dis_r, S
90
91 # ----- 全年滚动 -----
92 def run_year(prices=None):
93     plans_p = np.zeros((NDAY, T)); plans_ch = np.zeros((NDAY, T)); plans_dis = np.zeros((NDAY, T))
94     Emerg = np.zeros((NDAY, T)); ch_r = np.zeros((NDAY, T)); dis_r = np.zeros((NDAY, T))
95     Sall = np.zeros((NDAY, T+1)); plan_cost = np.zeros(NDAY)
96     S0 = 6000.0
97     for d in range(NDAY):
98         Lhat, PVhat = forecast(d)
99         pv_d = None if prices is None else prices[d]
100         ps, chs, diss, pc = plan_lp(Lhat, PVhat, S0, pv_d)
101         e, cr, dr, S = simulate(ps, chs, diss, LD[d], PVD[d], S0)
102         plans_p[d], plans_ch[d], plans_dis[d] = ps, chs, diss
103         Emerg[d], ch_r[d], dis_r[d], Sall[d], plan_cost[d] = e, cr, dr, S, pc
104         S0 = S[T]
105     return dict(plans_p=plans_p, plans_ch=plans_ch, plans_dis=plans_dis,
106               Emerg=Emerg, ch_r=ch_r, dis_r=dis_r, Sall=Sall, plan_cost=plan_cost)
107
108 if __name__ == "__main__":
109     _R = run_year()
110     plans_p, plans_ch, plans_dis = _R["plans_p"], _R["plans_ch"], _R["plans_dis"]
111     Emerg, ch_r, dis_r, Sall, plan_cost = _R["Emerg"], _R["ch_r"], _R["dis_r"], _R["Sall"], _R["plan_cost"]
112
113     rep = slice(feb1, NDAY)
114     E_p = plans_p[rep].sum()*dt
115     C_p = float(plan_cost[rep].sum())
116     E_e = Emerg[rep].sum()*dt
117     C_e = float((Emerg[rep]*price*EMULT).sum()*dt)
118     C_tot = C_p + C_e
119     edays = int((Emerg[rep].sum(1) > 1e-9).sum())

```

```

120 print("===== 问题2 最终策略 (w1预测, =1.05, =0.98) =====")
121 print(f"报告期 2025.2.1-12.31:")
122 print(f" 计划购电量 {E_p:,.1f} kWh, 计划购电费 {C_p:,.2f} 元")
123 print(f" 紧急购电量 {E_e:,.1f} kWh, 紧急购电费 {C_e:,.2f} 元")
124 print(f" 总费用 {C_tot:,.2f} 元, 紧急购电天数 {edays}/334")
125 print(f" 实际储电量范围 [{Sall.min():.1f}, {Sall.max():.1f}] kWh")
126 assert Sall.min() >= SMIN-1e-6 and Sall.max() <= SMAX+1e-6
127 viol = sum(1 for d in range(NDAY) for t in range(T)
128             if Emerg[d, t] > 1e-9 and not (ch_r[d, t] < 1e-9 and
129                 (dis_r[d, t] > PMAX-1e-6 or Sall[d, t+1] < SMIN+1e-6)))
130 print(f" 紧急购电合理性违规: {viol} 个")
131
132 # ----- 表3指定日期 -----
133 def tag(m):
134     plus = "+1" if m >= 1440 else ""
135     m %= 1440
136     return f"{m//60}:{m%60:02d}{plus}"
137
138 # 表2 钟点日时段块: "0:00-4:00" = 前一日末段(0:00-0:10) + 当日第1-23段
139 def bsum(arr, d, k):
140     if k == 0:
141         return float(arr[d - 1][143] + arr[d][0:23].sum())
142     seg = [(23, 47), (47, 71), (71, 95), (95, 119), (119, 143)][k - 1]
143     return float(arr[d][seg[0]:seg[1]].sum())
144 bnames = ["0:00-4:00", "4:00-8:00", "8:00-12:00", "12:00-16:00", "16:00-20:00", "20:00-24:00"]
145 idx_t1 = [59, 71, 83, 95, 107, 119]
146 names_t1 = ["10:00-10:10", "12:00-12:10", "14:00-14:10",
147             "16:00-16:10", "18:00-18:10", "20:00-20:10"]
148
149 def fmt_date(d):
150     x = dates[d]
151     return f"{x.year}.{x.month}.{x.day}"
152
153 def emerg_windows(e):
154     wins = []; t = 0
155     while t < T:
156         if e[t] > 1e-9:
157             j = t
158             while j+1 < T and e[j+1] > 1e-9: j += 1
159             wins.append([f"{tag(10*(t+1))}-{tag(10*(j+2))}", float(e[t:j+1].sum()*dt)])
160             t = j+1
161         else: t += 1
162     return wins
163
164 table3 = {}
165 for d in range(NDAY):
166     if fmt_date(d) in ["2025.3.20", "2025.6.21", "2025.9.23", "2025.12.21"]:
167         Ep = plans_p[d]*dt
168         Ech, Edis = ch_r[d]*dt, dis_r[d]*dt
169         e = Emerg[d]*dt
170         wins = emerg_windows(Emerg[d])
171         table3[fmt_date(d)] = {
172             "plan_t1": {n: float(Ep[i]) for n, i in zip(names_t1, idx_t1)},
173             "plan_total": float(Ep.sum()),
174             "plan_cost": float(plan_cost[d]),
175             "blocks": {n: [bsum(ch_r * dt, d, k), bsum(dis_r * dt, d, k)]
176                         for k, n in enumerate(bnames)},
177             "s000": float(Sall[d-1][143] if d > 0 else 6000.0),
178             "s2400": float(Sall[d][143]),
179             "emerg_windows": wins,
180             "emerg_total": float(e.sum()),
181         }
182
183 print("\n===== 表3 指定日期(最终策略) =====")

```

```

184 for k, v in table3.items():
185     print(f"\n--- {k} ---")
186     print(f"  全天计划购电量 {v['plan_total']:.2f} kWh, 计划购电费 {v['plan_cost']:.2f} 元")
187     print(f"  表1: ", {n: round(x, 2) for n, x in v["plan_t1"].items()})
188     print(f"  表2: ", {n: [round(a, 2), round(b, 2)] for n, (a, b) in v["blocks"].items()})
189     print(f"  s(0:00)={v['s000']:.2f}, s(24:00)={v['s2400']:.2f}")
190     print(f"  紧急购电: {[w, round(x, 2)] for w, x in v['emerg_windows']} if v['emerg_windows'] else '无' (合计 {v['emerg_total']:.2f} kWh)")
191
192 json.dump({"annual": {"E_plan": E_p, "C_plan": C_p, "E_emerg": E_e, "C_emerg": C_e,
193                     "C_total": C_tot, "emerg_days": edays},
194          "table3": table3},
195         open(BASE / "Q2" / "summary2.json", "w"), ensure_ascii=False, indent=1)
196
197 # ----- 写 result2.xlsx -----
198 out = openpyxl.Workbook()
199 ws1 = out.active; ws1.title = "计划购电量"
200 hdr = ["日期\\时间"] + [f"{tag(10*(i+1))}-{tag(10*(i+2))}" for i in range(143)]
201 hdr += ["0:00-0:10+1", "全天购电量", "全天购电费"]
202 hdr[42] = "7:0-7:10" # 保留官方模板第43列的 typo 以逐字一致
203 ws1.append(hdr)
204 for d in range(feb1, NDAY):
205     Ep = plans_p[d]*dt
206     ws1.append([dates[d]] + [round(float(x), 4) for x in Ep] +
207                [round(float(Ep.sum()), 4), round(float(plan_cost[d]), 4)])
208
209 ws2 = out.create_sheet("充放电电量")
210 ws2.append(["日期", "时间段", "充电量", "放电量", "时刻", "储电量"])
211 import datetime as _dt
212 for d in range(feb1, NDAY):
213     Ech, Edis = ch_r[d]*dt, dis_r[d]*dt
214     s000 = Sall[d-1][143] if d > 0 else 6000.0
215     s2400 = Sall[d][143]
216     for k, n in enumerate(bnames):
217         r = [dates[d] if k == 0 else None, n,
218              round(bsum(ch_r * dt, d, k), 4), round(bsum(dis_r * dt, d, k), 4)]
219         if k == 0: r += [_dt.time(0, 0), round(float(s000), 4)] # 与模板同为 time 对象
220         elif k == 1: r += ["24:00", round(float(s2400), 4)]
221         else: r += [None, None]
222     ws2.append(r)
223
224 ws3 = out.create_sheet("紧急购电量")
225 ws3.append(["日期", "购电时间段", "购电量"])
226 for d in range(feb1, NDAY):
227     first = True
228     for w, energy in emerg_windows(Emerg[d]):
229         ws3.append([dates[d] if first else None, w, round(energy, 4)])
230     first = False
231 out.save(BASE / "Q2" / "result2.xlsx")
232 print("\n已写出 result2.xlsx")
233
234 np.savez(BASE / "Q2" / "q2_data.npz", plans_p=plans_p, plans_ch=plans_ch,
235         plans_dis=plans_dis, Emerg=Emerg, Sall=Sall, plan_cost=plan_cost,
236         ch_r=ch_r, dis_r=dis_r, allow_pickle=True)
237 print("已保存 q2_data.npz")

```

3.2.3 问题二：q2_experiments.py（预测方法与余量参数的网格寻优实验）

```

1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """Q2 策略寻优实验：预测方法 × 余量系数 的 全年总费用对比"""

```

```

4 import numpy as np
5 from pathlib import Path
6 import openpyxl
7 from scipy.optimize import linprog
8
9 BASE = Path(__file__).resolve().parents[1]
10 c1 = openpyxl.load_workbook(BASE / "附件" / "附件1.xlsx", data_only=True)["Sheet1"]
11 rows1 = list(c1.iter_rows(min_row=2, values_only=True))
12 price = np.array([float(r[1]) for r in rows1])
13 L_typ = np.array([float(r[2]) for r in rows1])
14 PV_typ = np.array([float(r[3]) for r in rows1])
15
16 def load_cols(path, sheet):
17     ws = openpyxl.load_workbook(path, data_only=True)[sheet]
18     rows = list(ws.iter_rows(min_row=2, values_only=True))
19     return [r[0] for r in rows], np.array([float(v) for v in r[1:145]] for r in rows)
20
21 dates, LD = load_cols(BASE / "附件" / "附件2.xlsx", "小区负载")
22 _, PVD = load_cols(BASE / "附件" / "附件2.xlsx", "光伏发电实际功率")
23 NDAY = 365
24 T, dt = 144, 1.0 / 6.0
25 ETA_C = ETA_D = 0.9
26 PMAX, SMIN, SMAX = 5000.0, 1200.0, 10800.0
27 EMULT = 5.0
28 ip, ich, idi, iq, is_ = 0, T, 2 * T, 3 * T, 4 * T
29 nx = 5 * T + 1
30 febl = 31
31
32 def plan_lp(Lhat, PVhat, S0, lscale=1.0, pscale=1.0):
33     f = np.zeros(nx); f[ip:ip+T] = price * dt
34     Aeq, beq = [], []
35     Lh, Ph = Lhat * lscale, PVhat * pscale
36     for t in range(T):
37         r = np.zeros(nx)
38         r[ip+t] = 1; r[idi+t] = 1; r[ich+t] = -1; r[iq+t] = -1
39         Aeq.append(r); beq.append(Lh[t] - Ph[t])
40     for t in range(T):
41         r = np.zeros(nx)
42         r[is_+t+1] = 1; r[is_+t] = -1
43         r[ich+t] = -ETA_C*dt; r[idi+t] = dt/ETA_D
44         Aeq.append(r); beq.append(0.0)
45     for col, val in [(is_, S0), (is_+T, S0)]:
46         r = np.zeros(nx); r[col] = 1
47         Aeq.append(r); beq.append(val)
48     bnd = [(0, None)]*T + [(0, PMAX)]*2*T + [(0, None)]*T + [(SMIN, SMAX)]*(T+1)
49     res = linprog(f, A_eq=np.array(Aeq), b_eq=np.array(beq), bounds=bnd, method="highs")
50     assert res.status == 0
51     x = res.x
52     return x[ip:ip+T], x[ich:ich+T], x[idi:idi+T], res.fun
53
54 def simulate(ps, chs, diss, L, PV, S0):
55     e = np.zeros(T)
56     S = np.empty(T+1); S[0] = S0
57     for t in range(T):
58         dis_max = min(PMAX, max(S[t]-SMIN, 0.0)*ETA_D/dt)
59         ch_max = min(PMAX, max(SMAX-S[t], 0.0)/ETA_C/dt)
60         dis = min(diss[t], dis_max); ch = min(chs[t], ch_max)
61         resid = (L[t] - ps[t] - PV[t]) - (dis - ch)
62         if resid > 0:
63             r = min(ch, resid); ch -= r; resid -= r
64             r = min(dis_max - dis, resid); dis += r; resid -= r
65             e[t] = max(resid, 0.0)
66         else:
67             sur = -resid

```

```

68         r = min(dis, sur); dis -= r; sur -= r
69         r = min(ch_max - ch, sur); ch += r; sur -= r
70         S[t+1] = S[t] + ETA_C*ch*dt - dis*dt/ETA_D
71     return e, S
72
73 def forecast(d, method):
74     if d == 0:
75         return L_typ, PV_typ
76     if method == "pers":
77         return LD[d-1], PVD[d-1]
78     if method == "w1":
79         # 上周同星期
80         return (LD[d-7], PVD[d-7]) if d >= 7 else (LD[d-1], PVD[d-1])
81     if method.startswith("wk"):
82         # 最近k个同星期均值
83         k = int(method[2:])
84         idx = [d-7*i for i in range(1, k+1) if d-7*i >= 0]
85         if not idx: return LD[d-1], PVD[d-1]
86         return LD[idx].mean(0), PVD[idx].mean(0)
87     if method == "ma7":
88         k = max(1, d-7)
89         return LD[k:d].mean(0), PVD[k:d].mean(0)
90     raise ValueError(method)
91
92 def run(method, lscale=1.0, pscale=1.0, d0=0, d1=365):
93     S0 = 6000.0
94     Cp = np.zeros(NDAY); Ce = np.zeros(NDAY); Ee = np.zeros(NDAY)
95     for d in range(NDAY):
96         Lhat, PVhat = forecast(d, method)
97         ps, chs, diss, pc = plan_lp(Lhat, PVhat, S0, lscale, pscale)
98         e, S = simulate(ps, chs, diss, LD[d], PVD[d], S0)
99         Cp[d] = pc
100        Ce[d] = (e*price*EMULT).sum()*dt
101        Ee[d] = e.sum()*dt
102        S0 = S[T]
103
104    sl = slice(max(d0, feb1), d1)
105    return Cp[sl].sum() + Ce[sl].sum(), Cp[sl].sum(), Ce[sl].sum(), Ee[sl].sum()
106
107 import itertools, time
108 methods = ["pers", "w1", "wk2", "wk4", "ma7"]
109 scales = [(1.0, 1.0), (1.05, 1.0), (1.10, 1.0), (1.05, 0.95), (1.10, 0.90)]
110 print(f'预测':<6}{ ' 负载':>6}{ ' 光伏':>6}{ '总费用(万元)':>14}{ '计划费(万元)':>14}{ '紧急费(万元)':>14}{ '紧急量(MWh)':>13})
111 results = {}
112 for m, (g, dd) in itertools.product(methods, scales):
113     t0 = time.time()
114     Ctot, Cp, Ce, Ee = run(m, g, dd)
115     results[(m, g, dd)] = (Ctot, Cp, Ce, Ee)
116     print(f"{m:<6}{g:>6.2f}{dd:>6.2f}{Ctot/1e4:>14.1f}{Cp/1e4:>14.1f}{Ce/1e4:>14.1f}{Ee/1e3:>13.1f}  ({time.time()-t0:.0f}s)",
117           flush=True)
118
119 best = min(results.items(), key=lambda kv: kv[1][0])
120 print("\n最优组合:", best[0], f"总费用 {best[1][0]:.0f} 元")
121
122 # 前后半段稳健性: 用最优参数看 Feb-Aug vs Sep-Dec
123 (m, g, dd), _ = best
124 C1 = run(m, g, dd, 31, 243)[0] # Feb-Aug
125 C2 = run(m, g, dd, 243, 365)[0] # Sep-Dec
126 print(f"最优参数分段: 2-8月 {C1/1e4:.1f} 万元, 9-12月 {C2/1e4:.1f} 万元")

```

3.2.4 问题二: q2_perfect_foresight.py (完美预测理论下界)

```

1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-

```

```

3 """
4 Q2 完美预测费用下界 (图2"完美预测"柱的生成脚本)
5 口径: 与最终策略同框架(含储能, 计划LP日循环 S(144)=S(0)=S0 逐日继承),
6 但预测=当天实际值(完美预测), 故无紧急购电。为费用理论下界。
7 """
8 import importlib.util
9 from pathlib import Path
10 import numpy as np
11
12 BASE = Path(__file__).resolve().parents[1]
13 spec = importlib.util.spec_from_file_location("q2", BASE / "q2" / "solve_q2.py")
14 q2 = importlib.util.module_from_spec(spec)
15 spec.loader.exec_module(q2)
16
17 C = 0.0
18 S0 = 6000.0
19 for d in range(q2.feb1, q2.NDAY):
20     # 预测=当天实际(完美), 余量 = 1(无需余量), 日循环 S(144)=S(0)=S0
21     g, dd = q2.GAMMA, q2.DELTA
22     q2.GAMMA, q2.DELTA = 1.0, 1.0
23     _, _, pc = q2.plan_lp(q2.LD[d], q2.PVD[d], S0)
24     q2.GAMMA, q2.DELTA = g, dd
25     C += pc
26
27 print(f"完美预测费用下界(2025.2.1-12.31, 含储能, 无紧急购电): {C:.2f} 元")
28 # 参考: 最终策略 14,546,503.97 元 -> 预测误差残余代价 = C_final - C

```

3.2.5 问题三: solve_q3.py (预报标定 + 残差修正 + 双向调整 LP, 策略变体)

```

1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 CUMCM 2026 C题 问题3 —— 计划+滚动调整购电策略
5 流程(每天d):
6     0:00 光伏估计=逐时OLS[官方0:00报, w1] (30天滚动); 负载=w1; 计划LP(S(144)=S(0)=S0) → p_plan
7     6:12/18:00 光伏估计=逐时OLS[官方最新报, w1] + _pv · 当日已实现残差;
8     负载估计 += _L · 当日已实现残差 → 调整LP(分段成本 上调1.5c/下调0.5c) → p_adj
9     日内 储能贪心调节(同Q2), 缺口→紧急购电(5c)
10    结算: C = Σ[c · p_adj + 0.5c · (p_plan - p_adj)] + 1.5c · (p_adj - p_plan) · dt + E5c · e · dt
11    变体: V0=Q2式(w1光伏) V1=原始0:00报 V2=原始报+调整 V3=标定无调整 V4=完整(交付)
12    约定: 时刻戳=区间起点; 调整区间起点(0基): 6:00+35, 12:00+71, 18:00+107
13 """
14 import json
15 from pathlib import Path
16 import numpy as np
17 import openpyxl
18 from scipy.optimize import linprog
19
20 BASE = Path(__file__).resolve().parents[1]
21
22 c1 = openpyxl.load_workbook(BASE / "附件" / "附件1.xlsx", data_only=True)["Sheet1"]
23 rows1 = list(c1.iter_rows(min_row=2, values_only=True))
24 price = np.array([float(r[1]) for r in rows1])
25 L_typ = np.array([float(r[2]) for r in rows1])
26 PV_typ = np.array([float(r[3]) for r in rows1])
27
28 def load_cols(path, sheet):
29     ws = openpyxl.load_workbook(path, data_only=True)[sheet]
30     rows = list(ws.iter_rows(min_row=2, values_only=True))
31     return [r[0] for r in rows], np.array([float(v) for v in r[1:145]] for r in rows)
32

```

```

33 dates, LD = load_cols(BASE / "附件" / "附件2.xlsx", "小区负载")
34 _, PVD = load_cols(BASE / "附件" / "附件2.xlsx", "光伏发电实际功率")
35 PVh = PVD.reshape(365, 24, 6).mean(2)
36 Lh = LD.reshape(365, 24, 6).mean(2)
37 ws3 = openpyxl.load_workbook(BASE / "附件" / "附件3.xlsx", data_only=True)["Sheet1"]
38 rows3 = list(ws3.iter_rows(min_row=2, values_only=True))
39 FORE = {}
40 d_ = -1
41 for r in rows3:
42     if r[0] != '' and r[0] is not None: d_ += 1
43     FORE[(d_, int(str(r[1]).split(':')[0]))] = np.array([float(v) for v in r[2:26]])
44
45 NDAY = 365
46 T, dt = 144, 1.0 / 6.0
47 ETA_C = ETA_D = 0.9
48 PMAX, SMIN, SMAX = 5000.0, 1200.0, 10800.0
49 EMULT = 5.0
50 # 结算: 超额部分按 1.5c 计价(=基础 c + 0.5c 溢价), 违约部分按 0.5c 计价
51 # LP 目标中 u/u 的系数均为 0.5c(相对基础价的边际)
52 ADJ_UP_PRE, ADJ_DN = 0.5, 0.5
53 LAM_PV, LAM_L = 0.1, 0.3 # 费用网格寻优(原MAE定标0.3/0.4超调)
54 ADJ_T = [35, 71, 107]
55 ISSUE_OF = {35: 6, 71: 12, 107: 18}
56 feb1 = 31
57 hour_of = np.array([(t + 1) // 6 for t in range(T)]) # 1..24
58
59 # ----- 估计模型 -----
60 def pv_hour_est(d, issue, calib):
61     """25个整点(1..24)的光伏功率估计"""
62     h_est = np.zeros(25)
63     f = FORE[(d, issue)]
64     for h in range(1, 25):
65         k = h - issue
66         off = f[k - 1] if 1 <= k <= 24 else 0.0
67         if not calib:
68             h_est[h] = off; continue
69         w1 = PVh[d - 7, h % 24] if d >= 7 else 0.0
70         tr = [dd for dd in range(max(7, d - 30), d)]
71         if len(tr) >= 5:
72             X = np.array([FORE[(dd, 0)][h - 1], PVh[dd - 7, h % 24] for dd in tr])
73             y = np.array([PVh[dd, h % 24] for dd in tr])
74             beta, *_ = np.linalg.lstsq(np.column_stack([np.ones(len(tr)), X]), y, rcond=None)
75             est = beta[0] + beta[1] * off + beta[2] * w1
76             if not np.isfinite(est): est = 0.5 * off + 0.5 * w1
77         else:
78             est = 0.5 * off + 0.5 * w1
79         h_est[h] = max(est, 0.0)
80     return h_est
81
82 def to_10min(d, h_est):
83     out = np.zeros(T)
84     for t in range(T):
85         h = hour_of[t] % 24
86         denom = PVh[d - 7, h] if d >= 7 else 0.0
87         shape = PVD[d - 7, t] / denom if (d >= 7 and denom > 1e-6) else 1.0
88         out[t] = h_est[hour_of[t]] * shape
89     return out
90
91 def load_est(d):
92     if d == 0: return L_typ.copy()
93     if d >= 7: return LD[d - 7].copy()
94     return LD[d - 1].copy()
95
96 def load_hour_est(d):

```

```

97     if d == 0: return L_typ.reshape(24, 6).mean(1)
98     if d >= 7: return Lh[d - 7]
99     return Lh[d - 1]
100
101 # ----- LP -----
102 def lp_plan(Lh10, Ph10, S0, price_vec=None):
103     """计划LP:  $\min \sum c \cdot p \cdot dt$ ,  $S(144)=S(0)=S0$ . 返回 p,ch,dis (kW, 144)
104     price_vec: 可选当日电价(波动电价); 缺省用附件1固定电价"""
105     pr = price if price_vec is None else price_vec
106     n = T; o = 4 * n; m = 5 * n + 1
107     A = np.zeros((2 * n + 2, m)); b = np.zeros(2 * n + 2)
108     for t in range(n):
109         A[t, t] = 1; A[t, 2 * n + t] = 1; A[t, n + t] = -1; A[t, 3 * n + t] = -1
110         b[t] = Lh10[t] - Ph10[t]
111         A[n + t, o + t + 1] = 1; A[n + t, o + t] = -1
112         A[n + t, n + t] = -ETA_C * dt; A[n + t, 2 * n + t] = dt / ETA_D
113     A[2 * n, o] = 1; b[2 * n] = S0
114     A[2 * n + 1, o + n] = 1; b[2 * n + 1] = S0
115     f = np.zeros(m); f[:n] = pr * dt
116     bnd = [(0, None)] * n + [(0, PMAX)] * 2 * n + [(0, None)] * n + [(SMIN, SMAX)] * (n + 1)
117     res = linprog(f, A_eq=A, b_eq=b, bounds=bnd, method="highs")
118     assert res.status == 0, "计划LP不可行"
119     x = res.x
120     return x[:n], x[n:2 * n], x[2 * n:3 * n]
121
122 def lp_adjust(a, Lh10, Ph10, S_start, S_end, p_plan_seg, down_only=False, free_terminal=False,
123             price_vec=None, s_min_end=0.0):
124     """调整LP: 区间a..143,  $\min \sum [c \cdot p_{adj} + 0.5c \cdot (u+u) + 5c \cdot e] \cdot dt$ 
125     (u 边际 0.5c 超额总价 1.5c; u 边际 0.5c 违约 0.5c), s(a)=S_start
126     e 为安全阀(紧急购电预期, 5c), 保证任何情形下可行;
127     down_only=True 时 p_adj p_plan; free_terminal=True 时末端仅受 [SMIN, SMAX] 约束
128     price_vec: 可选当日电价(波动电价); 缺省用附件1固定电价"""
129     pr = price if price_vec is None else price_vec
130     n = T - a; o = 4 * n; m = 5 * n + 1; M = m + 2 * n + n
131     iu, ie = m, m + 2 * n # u± 与 e 的偏移
132     n_eq = 3 * n + (1 if free_terminal else 2)
133     A = np.zeros((n_eq, M)); b = np.zeros(n_eq)
134     for t in range(n):
135         gt = a + t
136         A[t, t] = 1; A[t, 2 * n + t] = 1; A[t, ie + t] = 1
137         A[t, n + t] = -1; A[t, 3 * n + t] = -1
138         b[t] = Lh10[gt] - Ph10[gt]
139         A[n + t, o + t + 1] = 1; A[n + t, o + t] = -1
140         A[n + t, n + t] = -ETA_C * dt; A[n + t, 2 * n + t] = dt / ETA_D
141     A[2 * n, o] = 1; b[2 * n] = S_start
142     if not free_terminal:
143         A[2 * n + 1, o + n] = 1; b[2 * n + 1] = S_end
144     for i in range(n):
145         row = 2 * n + (1 if free_terminal else 2) + i
146         A[row, i] = 1
147         A[row, iu + i] = -1
148         A[row, ie + n + i] = 1
149         b[row] = p_plan_seg[i]
150     f = np.zeros(M)
151     f[:n] = pr[a:] * dt
152     f[iu:iu + n] = ADJ_UP_PRE * pr[a:] * dt
153     f[iu + n:iu + 2 * n] = ADJ_DN * pr[a:] * dt
154     f[ie:] = EMULT * pr[a:] * dt
155     ub_p = p_plan_seg if down_only else [None] * n
156     bnd = ([(0, ub_p[i]) for i in range(n)] + [(0, PMAX)] * 2 * n + [(0, None)] * n +
157            [(SMIN, SMAX)] * (n + 1) + [(0, None)] * 2 * n + [(0, None)] * n)
158     A_ub = b_ub = None
159     if s_min_end > 0:
160         A_ub = np.zeros((1, M)); A_ub[0, o + n] = -1

```

```

161     b_ub = [-s_min_end] # 末端最低备用 s(144) >= s_min_end
162     res = linprog(f, A_eq=A, b_eq=b, A_ub=A_ub, b_ub=b_ub, bounds=bnd, method="highs")
163     assert res.status == 0, f"调整LP不可行 a={a}"
164     x = res.x
165     return x[:n], x[n:2 * n], x[2 * n:3 * n]
166
167 def lp_storage_only(a, Lh10, Ph10, S_start, S_end, p_fix):
168     """购电计划固定时，重排余日储能计划:  $\min \sum q$  (弃光) 使平衡可行"""
169     n = T - a; o = 3 * n; m = 4 * n + 1
170     A = np.zeros((2 * n + 2, m)); b = np.zeros(2 * n + 2)
171     for t in range(n):
172         gt = a + t
173         A[t, 2 * n + t] = 1; A[t, n + t] = -1; A[t, 3 * n + t] = -1
174         b[t] = Lh10[gt] - Ph10[gt] - p_fix[t]
175         A[n + t, o + t + 1] = 1; A[n + t, o + t] = -1
176         A[n + t, n + t] = -ETA_C * dt; A[n + t, 2 * n + t] = dt / ETA_D
177     A[2 * n, o] = 1; b[2 * n] = S_start
178     A[2 * n + 1, o + n] = 1; b[2 * n + 1] = S_end
179     f = np.zeros(m); f[3 * n:o] = 1.0 # min 弃光
180     bnd = [(0, PMAX)] * 2 * n + [(0, None)] * n + [(SMIN, SMAX)] * (n + 1)
181     res = linprog(f, A_eq=A, b_eq=b, bounds=bnd, method="highs")
182     assert res.status == 0, f"储能重排LP不可行 a={a}"
183     x = res.x
184     return x[:n], x[n:2 * n]
185
186 # ----- 日内模拟 -----
187 def simulate_seg(ps, chs, diss, L, PV, S, a, b):
188     """区间[a,b] 模拟，S就地更新，返回 (紧急功率，实际充电，实际放电) 段数组"""
189     e = np.zeros(b - a); cr = np.zeros(b - a); dr = np.zeros(b - a)
190     for t in range(a, b):
191         dis_max = min(PMAX, max(S[t] - SMIN, 0.0) * ETA_D / dt)
192         ch_max = min(PMAX, max(SMAX - S[t], 0.0) / ETA_C / dt)
193         dis = min(diss[t], dis_max); ch = min(chs[t], ch_max)
194         resid = (L[t] - ps[t] - PV[t]) - (dis - ch)
195         if resid > 0:
196             r = min(ch, resid); ch -= r; resid -= r
197             r = min(dis_max - dis, resid); dis += r; resid -= r
198             e[t - a] = max(resid, 0.0)
199         else:
200             sur = -resid
201             r = min(dis, sur); dis -= r; sur -= r
202             r = min(ch_max - ch, sur); ch += r; sur -= r
203             cr[t - a], dr[t - a] = ch, dis
204             S[t + 1] = S[t] + ETA_C * ch * dt - dis * dt / ETA_D
205     return e, cr, dr
206
207 # ----- 全年滚动 -----
208 def run_year(mode, GAMMA=1.02, DELTA=0.98, THETA=0.0, prices=None, S_MIN_END=0.0):
209     """mode: V0 光伏无调整 | V1 原始预报 | V2 原始+调整 | V3 标定无调整
210         V4 完整调整 | V4T 阈值调整(偏离 THETA kW不调整) | V4D 仅下调"""
211     plans_p = np.zeros((NDAY, T)); adj_p = np.zeros((NDAY, T))
212     Emerg = np.zeros((NDAY, T)); ch_r = np.zeros((NDAY, T)); dis_r = np.zeros((NDAY, T))
213     Sall = np.zeros((NDAY, T + 1)); plan_cost = np.zeros(NDAY)
214     for d in range(NDAY):
215         if mode == "V0":
216             PVh10 = PVD[d - 7].copy() if d >= 7 else PV_typ.copy()
217         elif mode in ("V1", "V2"):
218             PVh10 = to_10min(d, pv_hour_est(d, 0, calib=False))
219         else:
220             PVh10 = to_10min(d, pv_hour_est(d, 0, calib=True))
221         Lh10 = load_est(d); Lh_h = load_hour_est(d)
222         S0 = 6000.0 if d == 0 else Sall[d - 1, T]
223         pv_d = None if prices is None else prices[d]
224         ps, chs, diss = lp_plan(Lh10 * GAMMA, PVh10 * DELTA, S0, pv_d)

```

```

225 plans_p[d] = ps
226 plan_cost[d] = float((ps * (price if pv_d is None else pv_d)).sum() * dt)
227 p_adj = ps.copy()
228 S = Sall[d]; S[0] = S0
229 do_adj = mode in ("V2", "V4", "V4F", "V4FR", "V4T", "V4D")
230 bounds = [0, ADJ_T[0], ADJ_T[1], ADJ_T[2], T]
231 for si in range(4):
232     a, b = bounds[si], bounds[si + 1]
233     if si > 0 and do_adj:
234         issue = ISSUE_OF[a]
235         calib = (mode != "V2")
236         h_est = pv_hour_est(d, issue, calib).copy()
237         # 当日已实现残差(整点级, 相对0:00估计)
238         h_end = (a * 10) // 60 # 已实现的最后整点
239         res_pv, npv, res_l = 0.0, 0, 0.0
240         est0 = pv_hour_est(d, 0, calib)
241         for h in range(1, h_end + 1):
242             hh = h % 24
243             if est0[h] > 50 or (d >= 7 and PVh[d - 7, hh] > 50):
244                 res_pv += PVh[d, hh] - est0[h]; npv += 1
245                 res_l += Lh[d, hh] - Lh_h[hh]
246             if npv > 0: h_est += LAM_PV * res_pv / npv
247             h_est = np.maximum(h_est, 0.0)
248             PVh10 = to_10min(d, h_est)
249             Lh10 = Lh10.copy(); Lh10[a:] += LAM_L * res_l / max(h_end, 1)
250             if mode == "V4D":
251                 p_new, ch_new, dis_new = lp_adjust(a, Lh10 * GAMMA, PVh10 * DELTA,
252                                                     S[a], S0, plans_p[d][a:],
253                                                     down_only=True, free_terminal=True,
254                                                     price_vec=pv_d)
255             elif mode == "V4T":
256                 p_lp, _, _ = lp_adjust(a, Lh10 * GAMMA, PVh10 * DELTA,
257                                         S[a], S0, plans_p[d][a:], price_vec=pv_d)
258                 mask = np.abs(p_lp - plans_p[d][a:]) > THETA
259                 p_new = np.where(mask, p_lp, plans_p[d][a:])
260                 ch_new, dis_new = lp_storage_only(a, Lh10 * GAMMA, PVh10 * DELTA,
261                                                  S[a], S0, p_new)
262             else:
263                 p_new, ch_new, dis_new = lp_adjust(a, Lh10 * GAMMA, PVh10 * DELTA,
264                                                     S[a], S0, plans_p[d][a:],
265                                                     free_terminal=(mode in ("V4F", "V4FR")),
266                                                     price_vec=pv_d,
267                                                     s_min_end=(S_MIN_END if mode == "V4FR" else 0.0))
268             p_adj[a:], chs[a:], diss[a:] = p_new, ch_new, dis_new
269             e, cr, dr = simulate_seg(p_adj, chs, diss, LD[d], PVD[d], S, a, b)
270             Emerg[d, a:b] = e; ch_r[d, a:b] = cr; dis_r[d, a:b] = dr
271         adj_p[d] = p_adj
272     return dict(plans_p=plans_p, adj_p=adj_p, Emerg=Emerg, ch_r=ch_r,
273               dis_r=dis_r, Sall=Sall, plan_cost=plan_cost)
274
275 def settle(R, sl, prices=None):
276     """费用结算:  $C = \sum [c \cdot p_{adj}]dt + \sum [0.5c \cdot u + 0.5c \cdot u]dt + \sum c \cdot e \cdot dt$ 
277     (等价于  $\sum [c \cdot \min(0.5c \cdot (\text{plan}-\text{adj}) + 1.5c \cdot (\text{adj}-\text{plan})]dt$  + 紧急)
278     prices: 可选(365×144)逐日电价矩阵(波动电价)"""
279     pp = R["plans_p"][sl]; pa = R["adj_p"][sl]; e = R["Emerg"][sl]
280     pr = price.reshape(1, T) if prices is None else prices[sl]
281     base = float((pa * pr).sum() * dt)
282     up = np.maximum(pa - pp, 0.0); dn = np.maximum(pp - pa, 0.0)
283     fee_adj = float(((ADJ_UP_PRE * up + ADJ_DN * dn) * pr).sum() * dt)
284     emerg = float((e * pr * EMULT).sum() * dt)
285     plan_nominal = float((pp * pr).sum() * dt)
286     return dict(total=base + fee_adj + emerg, base=base, fee_adj=fee_adj,
287               emerg=emerg, plan_nominal=plan_nominal,
288               E_emerg=float(e.sum() * dt),

```

```

289         E_adj_up=float(up.sum() * dt), E_adj_dn=float(dn.sum() * dt))
290
291 if __name__ == "__main__":
292     rep = slice(feb1, NDAY)
293     print(f"{'变体':<26}{'总费用(万)':>12}{'实付(万)':>11}{'调整费(万)':>11}{'紧急费(万)':>11}{'紧急MWh':>9}")
294     results = {}
295     for mode, name in [("V0", "V0 Q2式(w1光伏无调整)", ("V1", "V1 原始0:00报无调整"),
296                       ("V2", "V2 原始报+原始调整"), ("V3", "V3 标定无调整"),
297                       ("V4", "V4 完整策略")]:
298         R = run_year(mode)
299         s = settle(R, rep)
300         results[mode] = (R, s)
301         print(f"{name:<26}{s['total']/1e4:>12.1f}{s['base']/1e4:>11.1f}"
302               f"{s['fee_adj']/1e4:>11.1f}{s['emerg']/1e4:>11.1f}{s['E_emerg']/1e3:>9.1f}", flush=True)
303     np.save(BASE / "Q3" / "results_variants.npy", results, allow_pickle=True)

```

3.2.6 问题三：finalize_q3.py（最终策略 V4FR 年度仿真与结果输出）

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """Q3 最终策略 V4FR (=1.02, =0.98): 生成 result3.xlsx + summary3.json + q3_data.npz + 全部校验"""
4  import json
5  import numpy as np
6  import openpyxl
7  import solve_q3 as q
8
9  BASE = q.BASE
10 R = q.run_year("V4FR", GAMMA=1.02, DELTA=0.98, S_MIN_END=4000.0)
11 rep = slice(q.feb1, 365)
12 price, NDAY, T, dt = q.price, q.NDAY, q.T, q.dt
13 plans_p, adj_p = R["plans_p"], R["adj_p"]
14 Emerg, ch_r, dis_r, Sall = R["Emerg"], R["ch_r"], R["dis_r"], R["Sall"]
15
16 # ----- 校验 -----
17 assert Sall.min() >= 1200 - 1e-6 and Sall.max() <= 10800 + 1e-6, "储能越界"
18 assert (adj_p >= -1e-9).all()
19 up_total = float(np.maximum(adj_p - plans_p, 0).sum() * dt)
20 print(f" 上调电量 {up_total:,.1f} kWh (V4FR 双向调整)")
21 viol = sum(1 for d in range(NDAY) for t in range(T)
22           if Emerg[d, t] > 1e-9 and not (ch_r[d, t] < 1e-9 and
23           (dis_r[d, t] > 5000 - 1e-6 or Sall[d, t + 1] < 1200 + 1e-6)))
24 assert viol == 0, f"紧急购电语义违规 {viol}"
25 s = q.settle(R, rep)
26 print("===== 问题3 最终策略 V4FR (标定+双向调整+S(24:00) 4000备用, =1.02, =0.98) =====")
27 print(f"校验全部通过. 年度(2.1-12.31):")
28 print(f" 计划购电费(0:00计划口径) {s['plan_nominal']:,.2f} 元")
29 print(f" 按调整量实付 {s['base']:,.2f} 元, 调整违约/溢价费 {s['fee_adj']:,.2f} 元(上下调各0.5倍)")
30 print(f" 紧急购电费 {s['emerg']:,.2f} 元, 紧急购电量 {s['E_emerg']:,.1f} kWh")
31 print(f" 调整下调电量 {s['E_adj_dn']:,.1f} kWh, 上调电量 {s['E_adj_up']:,.1f} kWh")
32 print(f" 总费用 {s['total']:,.2f} 元")
33 edays = int((Emerg[rep].sum(1) > 1e-9).sum())
34 print(f" 紧急购电天数 {edays}/334")
35
36 # ----- 表3指定日期 -----
37 def tag(m):
38     plus = "+1" if m >= 1440 else ""
39     m %= 1440
40     return f"{m//60}:{m%60:02d}{plus}"
41
42 # 表2 钟点日时段块: "0:00-4:00" = 前一日末段(0:00-0:10) + 当日第1-23段
43 def bsum(arr, d, k):

```

```

44     if k == 0:
45         return float(arr[d - 1][143] + arr[d][0:23].sum())
46     seg = [(23, 47), (47, 71), (71, 95), (95, 119), (119, 143)][k - 1]
47     return float(arr[d][seg[0]:seg[1]].sum())
48 bnames = ["0:00-4:00", "4:00-8:00", "8:00-12:00", "12:00-16:00", "16:00-20:00", "20:00-24:00"]
49 idx_t1 = [59, 71, 83, 95, 107, 119]
50 names_t1 = ["10:00-10:10", "12:00-12:10", "14:00-14:10",
51             "16:00-16:10", "18:00-18:10", "20:00-20:10"]
52
53 def fmt_date(d):
54     x = q.dates[d]
55     return f"{x.year}.{x.month}.{x.day}"
56
57 def emerg_windows(e):
58     wins = []; t = 0
59     while t < T:
60         if e[t] > 1e-9:
61             j = t
62             while j + 1 < T and e[j + 1] > 1e-9: j += 1
63             wins.append([f"tag(10*(t+1))-tag(10*(j+2))", float(e[t:j+1].sum() * dt)])
64             t = j + 1
65         else: t += 1
66     return wins
67
68 table3 = {}
69 for d in range(NDAY):
70     if fmt_date(d) in ["2025.3.20", "2025.6.21", "2025.9.23", "2025.12.21"]:
71         Ep, Ea = plans_p[d] * dt, adj_p[d] * dt
72         Ech, Edis = ch_r[d] * dt, dis_r[d] * dt
73         e = Emerg[d] * dt
74         dev = np.abs(Ea - Ep)
75         fee_dn = float((dev * price * 0.5).sum())
76         table3[fmt_date(d)] = {
77             "plan_t1": {n: float(Ep[i]) for n, i in zip(names_t1, idx_t1)},
78             "adj_t1": {n: float(Ea[i]) for n, i in zip(names_t1, idx_t1)},
79             "plan_total": float(Ep.sum()), "adj_total": float(Ea.sum()),
80             "plan_cost": float(R["plan_cost"][d]),
81             "adj_settle_cost": float((Ea * price).sum() + fee_dn),
82             "blocks": {n: [bsum(ch_r * dt, d, k), bsum(dis_r * dt, d, k)]
83                          for k, n in enumerate(bnames)},
84             "s000": float(Sall[d - 1][143] if d > 0 else 6000.0),
85             "s2400": float(Sall[d][143]),
86             "emerg_windows": emerg_windows(Emerg[d]),
87             "emerg_total": float(e.sum()),
88         }
89
90 print("\n===== 表3 指定日期 =====")
91 for k, v in table3.items():
92     print(f"\n--- {k} ---")
93     print(f"    计划购电量 {v['plan_total']:.2f} kWh -> 调整购电量 {v['adj_total']:.2f} kWh")
94     print(f"    计划购电费 {v['plan_cost']:.2f} 元, 调整后结算(含违约金) {v['adj_settle_cost']:.2f} 元")
95     print("    表1(调整后):", {n: round(x, 2) for n, x in v["adj_t1"].items()})
96     print("    表2:", {n: [round(a, 2), round(b, 2)] for n, (a, b) in v["blocks"].items()})
97     print(f"    s(0:00)={v['s000']:.2f}, s(24:00)={v['s2400']:.2f}")
98     print(f"    紧急购电: {[w, round(x, 2)] for w, x in v['emerg_windows']} if v['emerg_windows'] else '无' (合计 {v['emerg_total']:.2f} kWh)")
99
100 json.dump({"annual": s, "emerg_days": edays, "table3": table3},
101           open(BASE / "Q3" / "summary3.json", "w"), ensure_ascii=False, indent=1)
102
103 # ----- 写 result3.xlsx -----
104 out = openpyxl.Workbook()
105 hdr = ["日期\\时间"] + [f"tag(10*(i+1))-tag(10*(i+2))" for i in range(143)]
106 hdr += ["0:00-0:10+1", "全天购电量", "全天购电费"]

```

```

107 hdr[42] = "7:0-7:10"          # 保留模板 typo
108
109 ws1 = out.active; ws1.title = "计划购电量"
110 ws1.append(hdr)
111 for d in range(q.feb1, NDAY):
112     Ep = plans_p[d] * dt
113     ws1.append([q.dates[d]] + [round(float(x), 4) for x in Ep] +
114               [round(float(Ep.sum()), 4), round(float(R["plan_cost"][d]), 4)])
115
116 ws1b = out.create_sheet("调整购电量")
117 ws1b.append(hdr.copy())
118 for d in range(q.feb1, NDAY):
119     Ea = adj_p[d] * dt
120     ws1b.append([q.dates[d]] + [round(float(x), 4) for x in Ea] +
121               [round(float(Ea.sum()), 4), round(float((adj_p[d] * price).sum() * dt), 4)])
122
123 import datetime as _dt
124 ws2 = out.create_sheet("充放电")
125 ws2.append(["日期", "时间段", "充电量", "放电", "时刻", "储电量"])
126 for d in range(q.feb1, NDAY):
127     Ech, Edis = ch_r[d] * dt, dis_r[d] * dt
128     s000 = Sall[d - 1][143] if d > 0 else 6000.0
129     s2400 = Sall[d][143]
130     for k, n in enumerate(bnames):
131         r = [q.dates[d] if k == 0 else None, n,
132              round(bsum(ch_r * dt, d, k), 4), round(bsum(dis_r * dt, d, k), 4)]
133         if k == 0: r += [_dt.time(0, 0), round(float(s000), 4)]
134         elif k == 1: r += ["24:00", round(float(s2400), 4)]
135         else: r += [None, None]
136     ws2.append(r)
137
138 ws3 = out.create_sheet("紧急购电量")
139 ws3.append(["日期", "购电时间段", "购电量"])
140 for d in range(q.feb1, NDAY):
141     first = True
142     for w, energy in emerg_windows(Emerg[d]):
143         ws3.append([q.dates[d] if first else None, w, round(energy, 4)])
144         first = False
145 out.save(BASE / "Q3" / "result3.xlsx")
146 print("\n已写出 result3.xlsx")
147
148 np.savez(BASE / "Q3" / "q3_data.npz", plans_p=plans_p, adj_p=adj_p, Emerg=Emerg,
149         ch_r=ch_r, dis_r=dis_r, Sall=Sall, plan_cost=R["plan_cost"], allow_pickle=True)
150 print("已保存 q3_data.npz")

```

3.2.7 问题四：solve_q4.py（波动电价下的问题二、三策略重算）

```

1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 CUMCM 2026 C题 问题4 —— 波动电价下重算问题2/问题3
5 电价：附件4 逐日曲线(365×144)，假设日前可知(日前市场公布)；
6 紧急购电=当时实际电价×5，调整结算=附件4实际价格。
7 Q4-2 = 问题2最终策略(w1预测, =1.05, =0.98) 换波动电价 → result4-2.xlsx
8 Q4-3 = 问题3最终策略(V4FR: 备用>=4000, =1.02, =1.00) 换波动电价 → result4-3.xlsx
9 """
10 import json
11 import importlib.util
12 from pathlib import Path
13 import numpy as np
14 import openpyxl

```

```

15
16 BASE = Path(__file__).resolve().parents[1]
17
18 def imp(path, name):
19     spec = importlib.util.spec_from_file_location(name, path)
20     m = importlib.util.module_from_spec(spec)
21     spec.loader.exec_module(m)
22     return m
23
24 q2 = imp(BASE / "Q2" / "solve_q2.py", "q2")
25 q3 = imp(BASE / "Q3" / "solve_q3.py", "q3")
26
27 # ----- 附件4 波动电价 -----
28 ws4 = openpyxl.load_workbook(BASE / "附件" / "附件4.xlsx", data_only=True)["Sheet1"]
29 rows4 = list(ws4.iter_rows(min_row=2, values_only=True))
30 dates4 = [r[0] for r in rows4]
31 PM = np.array([[float(v) for v in r[1:145]] for r in rows4]) # 365*144 逐日电价
32 assert PM.shape == (365, 144)
33 print(f"附件4电价: 均值 {PM.mean():.4f}, 范围 [{PM.min():.4f}, {PM.max():.4f}]")
34 print(f"日均电价: min {PM.mean(1).min():.4f} / max {PM.mean(1).max():.4f}; "
35       f"日内价差(max-min)均值 {(PM.max(1)-PM.min(1)).mean():.4f}")
36 # 与附件1固定电价比较
37 pc = q3.price
38 print(f"附件1固定电价: 均值 {pc.mean():.4f}, 日内价差 {pc.max()-pc.min():.4f}")
39
40 NDAY, T, dt = 365, 144, 1.0 / 6.0
41 feb1 = 31
42 rep = slice(feb1, NDAY)
43 dates = q3.dates
44
45 # ===== Q4-2: 问题2策略 × 波动电价 =====
46 R2 = q2.run_year(prices=PM)
47 E_p2 = R2["plans_p"][rep].sum() * dt
48 C_p2 = float(R2["plan_cost"][rep].sum())
49 E_e2 = R2["Emerg"][rep].sum() * dt
50 C_e2 = float((R2["Emerg"][rep] * PM[rep] * q2.EMULT).sum() * dt)
51 C2 = C_p2 + C_e2
52 ed2 = int((R2["Emerg"][rep].sum(1) > 1e-9).sum())
53 assert R2["Sall"].min() >= 1200 - 1e-6 and R2["Sall"].max() <= 10800 + 1e-6
54 print("\n==== Q4-2 (问题2策略 × 波动电价) =====")
55 print(f"计划购电量 {E_p2:,.1f} kWh, 计划购电费 {C_p2:,.2f} 元")
56 print(f"紧急购电量 {E_e2:,.1f} kWh, 紧急购电费 {C_e2:,.2f} 元, 天数 {ed2}/334")
57 print(f"总费用 {C2:,.2f} 元 (恒定电价问题2: 14,546,503.97 元, 差 {C2-14546503.97:+,.2f})")
58
59 # ===== Q4-3: 问题3策略 × 波动电价 =====
60 R3 = q3.run_year("V4FR", GAMMA=1.02, DELTA=1.00, prices=PM, S_MIN_END=4000.0)
61 s3 = q3.settle(R3, rep, prices=PM)
62 ed3 = int((R3["Emerg"][rep].sum(1) > 1e-9).sum())
63 assert R3["Sall"].min() >= 1200 - 1e-6 and R3["Sall"].max() <= 10800 + 1e-6
64 viol = sum(1 for d in range(NDAY) for t in range(T)
65            if R3["Emerg"][d, t] > 1e-9 and not (R3["ch_r"][d, t] < 1e-9 and
66            (R3["dis_r"][d, t] > 5000 - 1e-6 or R3["Sall"][d, t + 1] < 1200 + 1e-6)))
67 assert viol == 0, f"紧急购电语义违规 {viol}"
68 print("\n==== Q4-3 (问题3策略V4FR(备用>=4000, =1.00) × 波动电价) =====")
69 print(f"按调整量实付 {s3['base']:,.2f} 元, 调整违约/溢价费 {s3['fee_adj']:,.2f} 元")
70 print(f"紧急购电费 {s3['emerg']:,.2f} 元, 紧急购电量 {s3['E_emerg']:,.1f} kWh, 天数 {ed3}/334")
71 print(f"调整: 上调 {s3['E_adj_up']:,.1f} kWh, 下调 {s3['E_adj_dn']:,.1f} kWh")
72 print(f"总费用 {s3['total']:,.2f} 元 (恒定电价问题3: 13,825,637.87 元, 差 {s3['total']-13825637.87:+,.2f})")
73
74 # ----- 表3 指定日期 -----
75 def tag(m):
76     plus = "+1" if m >= 1440 else ""
77     m %= 1440
78     return f"{m//60}:{m%60:02d}{plus}"

```

```

79
80 # 表2 钟点日时段块: "0:00-4:00" = 前一日末段(0:00-0:10) + 当日第1-23段
81 def bsum(arr, d, k):
82     if k == 0:
83         return float(arr[d - 1][143] + arr[d][0:23].sum())
84     seg = [(23, 47), (47, 71), (71, 95), (95, 119), (119, 143)][k - 1]
85     return float(arr[d][seg[0]:seg[1]].sum())
86 bnames = ["0:00-4:00", "4:00-8:00", "8:00-12:00", "12:00-16:00", "16:00-20:00", "20:00-24:00"]
87 idx_t1 = [59, 71, 83, 95, 107, 119]
88 names_t1 = ["10:00-10:10", "12:00-12:10", "14:00-14:10",
89             "16:00-16:10", "18:00-18:10", "20:00-20:10"]
90
91 def fmt_date(d):
92     x = dates[d]
93     return f"{x.year}.{x.month}.{x.day}"
94
95 def emerg_windows(e):
96     wins = []; t = 0
97     while t < T:
98         if e[t] > 1e-9:
99             j = t
100             while j + 1 < T and e[j + 1] > 1e-9: j += 1
101             wins.append([f"{tag(10*(t+1))}-{tag(10*(j+2))}", float(e[t:j+1].sum() * dt)])
102             t = j + 1
103         else: t += 1
104     return wins
105
106 table3 = {}
107 for d in range(NDAY):
108     if fmt_date(d) in ["2025.3.20", "2025.6.21", "2025.9.23", "2025.12.21"]:
109         entry = {"date": fmt_date(d)}
110         # Q4-2
111         Ep2 = R2["plans_p"][d] * dt
112         entry["q42"] = {
113             "plan_t1": {n: float(Ep2[i]) for n, i in zip(names_t1, idx_t1)},
114             "plan_total": float(Ep2.sum()), "plan_cost": float(R2["plan_cost"][d]),
115             "blocks": {n: [bsum(R2["ch_r"] * dt, d, k), bsum(R2["dis_r"] * dt, d, k)]
116                        for k, n in enumerate(bnames)},
117             "s000": float(R2["Sall"][d - 1][143] if d > 0 else 6000.0),
118             "s2400": float(R2["Sall"][d][143]),
119             "emerg_windows": emerg_windows(R2["Emerg"][d]),
120             "emerg_total": float(R2["Emerg"][d].sum() * dt)}
121         # Q4-3
122         Ep3, Ea3 = R3["plans_p"][d] * dt, R3["adj_p"][d] * dt
123         dev = np.abs(Ea3 - Ep3)
124         entry["q43"] = {
125             "plan_t1": {n: float(Ep3[i]) for n, i in zip(names_t1, idx_t1)},
126             "adj_t1": {n: float(Ea3[i]) for n, i in zip(names_t1, idx_t1)},
127             "plan_total": float(Ep3.sum()), "adj_total": float(Ea3.sum()),
128             "plan_cost": float(R3["plan_cost"][d]),
129             "adj_settle_cost": float((Ea3 * PM[d]).sum() + 0.5 * (dev * PM[d]).sum()),
130             "blocks": {n: [bsum(R3["ch_r"] * dt, d, k), bsum(R3["dis_r"] * dt, d, k)]
131                        for k, n in enumerate(bnames)},
132             "s000": float(R3["Sall"][d - 1][143] if d > 0 else 6000.0),
133             "s2400": float(R3["Sall"][d][143]),
134             "emerg_windows": emerg_windows(R3["Emerg"][d]),
135             "emerg_total": float(R3["Emerg"][d].sum() * dt)}
136         table3[fmt_date(d)] = entry
137
138 print("\n===== 表3 指定日期(波动电价) =====")
139 for k, v in table3.items():
140     print(f"\n--- {k} ---")
141     a, b = v["q42"], v["q43"]
142     print(f" Q4-2: 计划购电量 {a['plan_total']:.2f} kWh, 购电费 {a['plan_cost']:.2f} 元, 紧急 {a['emerg_total']:.2f} kWh")

```

```

143     print(f" Q4-3: 计划 {b['plan_total']:.2f} -> 调整 {b['adj_total']:.2f} kWh, "
144           f"计划费 {b['plan_cost']:.2f} -> 结算 {b['adj_settle_cost']:.2f} 元, 紧急 {b['emerg_total']:.2f} kWh")
145
146 json.dump({"q42": {"E_plan": E_p2, "C_plan": C_p2, "E_emerg": E_e2,
147                  "C_emerg": C_e2, "C_total": C2, "emerg_days": ed2},
148           "q43": {"s3": s3, "emerg_days": ed3},
149           "price_stats": {"mean": float(PM.mean()), "min": float(PM.min()),
150                          "max": float(PM.max()),
151                          "daily_spread_mean": float((PM.max(1) - PM.min(1)).mean())},
152           "table3": table3},
153         open(BASE / "Q4" / "summary4.json", "w"), ensure_ascii=False, indent=1)
154
155 # ----- 写 result4-2.xlsx (同 result2 版式) -----
156 import datetime as _dt
157
158 def write_result4(fname, R, has_adj):
159     out = openpyxl.Workbook()
160     hdr = ["日期\\时间"] + [f"{tag(10*(i+1))}-{tag(10*(i+2))}" for i in range(143)]
161     hdr += ["0:00-0:10+1", "全天购电量", "全天购电费"]
162     hdr[42] = "7:0-7:10"
163     ws1 = out.active; ws1.title = "计划购电量"
164     ws1.append(hdr)
165     for d in range(feb1, NDAY):
166         Ep = R["plans_p"][d] * dt
167         ws1.append([dates[d]] + [round(float(x), 4) for x in Ep] +
168                   [round(float(Ep.sum()), 4), round(float(R["plan_cost"][d]), 4)])
169     if has_adj:
170         ws1b = out.create_sheet("调整购电量")
171         ws1b.append(hdr.copy())
172         for d in range(feb1, NDAY):
173             Ea = R["adj_p"][d] * dt
174             ws1b.append([dates[d]] + [round(float(x), 4) for x in Ea] +
175                        [round(float(Ea.sum()), 4), round(float((R["adj_p"][d] * PM[d]).sum() * dt), 4)])
176     ws2 = out.create_sheet("充放电电量")
177     ws2.append(["日期", "时间段", "充电量", "放电量", "时刻", "储电量"])
178     R_ch, R_dis = R["ch_r"] * dt, R["dis_r"] * dt
179     for d in range(feb1, NDAY):
180         s000 = R["Sall"][d - 1][143] if d > 0 else 6000.0
181         s2400 = R["Sall"][d][143]
182         for k, n in enumerate(bnames):
183             r = [dates[d] if k == 0 else None, n,
184                  round(bsum(R_ch, d, k), 4), round(bsum(R_dis, d, k), 4)]
185             if k == 0: r += [_dt.time(0, 0), round(float(s000), 4)]
186             elif k == 1: r += ["24:00", round(float(s2400), 4)]
187             else: r += [None, None]
188             ws2.append(r)
189     ws3 = out.create_sheet("紧急购电量")
190     ws3.append(["日期", "购电时间段", "购电量"])
191     for d in range(feb1, NDAY):
192         first = True
193         for w, energy in emerg_windows(R["Emerg"][d]):
194             ws3.append([dates[d] if first else None, w, round(energy, 4)])
195             first = False
196     out.save(BASE / "Q4" / fname)
197
198 write_result4("result4-2.xlsx", R2, has_adj=False)
199 write_result4("result4-3.xlsx", R3, has_adj=True)
200 print("\n已写出 result4-2.xlsx, result4-3.xlsx")
201
202 np.savez(BASE / "Q4" / "q4_data.npz",
203          R2_plans_p=R2["plans_p"], R2_Emerg=R2["Emerg"], R2_ch_r=R2["ch_r"],
204          R2_dis_r=R2["dis_r"], R2_Sall=R2["Sall"], R2_plan_cost=R2["plan_cost"],
205          R3_plans_p=R3["plans_p"], R3_adj_p=R3["adj_p"], R3_Emerg=R3["Emerg"],
206          R3_ch_r=R3["ch_r"], R3_dis_r=R3["dis_r"], R3_Sall=R3["Sall"],

```

```
207     R3_plan_cost=R3["plan_cost"], PM=PM, allow_pickle=True)
208     print("已保存 q4_data.npz")
```